RÉPUBLIQUE DU SÉNÉGAL
MINISTÈRE DE L'ENSEIGNEMENT SUPÉRIEUR, DE LA RECHERCHE ET DE L'INNOVATION

INSTITUT SUPÉRIEUR D'ENSEIGNEMENT PROFESSIONNEL AMADOU TRAWARÉ

PROJET PERSONNEL PROFESSIONNEL POUR L'OBTENTION DU DIPLÔME SUPÉRIEUR D'ENSEIGNEMENT PROFESSIONNEL (DiSEP)

Métier : Technicien Supérieur en Analyse de Performance Digitale

JAPPO DUNDU

Plateforme intégrée de coordination des ressources sanitaires : disponibilité des lits hospitaliers, dons de sang et services ambulanciers au Sénégal

Connecter • Sauver • Vivre

Date de soutenance : ……

Réalisé par :

1. **Pape Alioune Sene**
2. **Serigne Mbacké Faye**
3. **Ibrahima Khalilou Diallo**
4. **El Hadji Massogui Diop**

Encadreur :

M. Diagne

Année académique 2025 – 2026

REMERCIEMENTS

Au terme de ce projet, nous exprimons notre profonde gratitude à toutes les personnes qui ont contribué, de près ou de loin, à sa réalisation.

Nos remerciements s'adressent tout particulièrement à notre encadreur, **M. Diagne**, pour ses conseils méthodologiques, sa disponibilité et l'exigence bienveillante avec laquelle il a suivi chaque étape de ce travail.

Nous remercions la direction et l'ensemble du corps enseignant de l'**Institut Supérieur d'Enseignement Professionnel Amadou Trawaré (ISEP-AT)** de Diamniadio, qui nous ont transmis les connaissances techniques et la rigueur indispensables à ce projet, ainsi que les professionnels de santé et du Centre National de Transfusion Sanguine (CNTS) qui ont partagé leur expérience lors de nos échanges préliminaires.

Nous adressons enfin nos pensées aux donneurs bénévoles et aux équipes soignantes du Sénégal, dont l'engagement quotidien a donné son sens à ce projet, ainsi qu'à nos familles pour leur soutien constant tout au long de notre formation.

RESUME / ABSTRACT

Résumé

Au Sénégal, la prise en charge d'une urgence médicale dépend de trois ressources souvent dispersées : le sang disponible, les lits libres dans les hôpitaux et les ambulances. Leur disponibilité circule encore largement par téléphone, d'un établissement à l'autre, ce qui allonge les délais au moment où chaque minute compte. En 2024, 136 347 dons de sang ont été enregistrés dans le pays, pour un déficit estimé à 25 % par rapport aux besoins (MSAS, 2025).

Jappo Dundu (« Soigner ensemble » en wolof) est une plateforme numérique qui réunit ces trois ressources dans un même système. Elle comprend trois modules : **Jappo Sang** (demandes de sang urgentes, alerte des donneurs compatibles les plus proches, stocks, prévision des pénuries), **Jappo Lits** (capacités par service, admissions et sorties en temps réel, réservation et orientation des patients) et **Jappo Ambulances** (régulation SAMU, affectation de l'ambulance la plus proche, suivi GPS, pré-alerte de l'hôpital). Une application web sert les hôpitaux, le CNTS et la régulation ; une application mobile sert les donneurs et les ambulanciers.

La solution repose sur une API REST Django REST Framework documentée par Swagger (OpenAPI 3), une base PostgreSQL avec PostGIS pour les recherches géographiques, Django Channels et Redis pour le temps réel, React pour le web et Expo / React Native pour le mobile. Un modèle d'apprentissage automatique (régression quantile par gradient boosting, intervalles calibrés) prévoit les stocks de sang jusqu'à 30 jours. Le système est vérifié par 590 tests automatisés côté serveur, 104 côté web, 73 côté mobile et des scénarios de bout en bout, exécutés à chaque modification par une intégration continue.

Mots-clés : santé numérique, urgences médicales, don de sang, lits hospitaliers, ambulances, géolocalisation, temps réel, apprentissage automatique, Django REST Framework, React, Sénégal.

Abstract

In Senegal, handling a medical emergency depends on three scattered resources: available blood, free hospital beds and ambulances. Information about them still travels mostly by phone, which delays care when every minute counts. In 2024, 136,347 blood donations were recorded nationwide, with an estimated 25 % shortfall against actual needs (MSAS, 2025).

Jappo Dundu (“Caring together” in Wolof) is a digital platform that brings these three resources into a single system. It is organised into three modules: **Jappo Sang** (urgent blood requests, alerts to the nearest compatible donors, stock management, shortage forecasting), **Jappo Lits** (bed capacity per ward, real-time admissions and discharges, bed reservation and patient referral) and **Jappo Ambulances** (EMS dispatch, automatic assignment of the nearest ambulance, GPS tracking, hospital pre-alert). A web application serves hospitals, the national blood transfusion centre and dispatchers; a mobile application serves donors and ambulance crews.

The solution is built on a Django REST Framework API documented with Swagger (OpenAPI 3), a PostgreSQL/PostGIS database for geographic queries, Django Channels and Redis for real-time updates, React for the web and Expo / React Native for mobile. A machine-learning model (quantile gradient boosting with calibrated intervals) forecasts blood stocks up to 30 days ahead. The system is checked by 590 automated server-side tests, 104 web tests, 73 mobile tests and end-to-end scenarios, run on every change by continuous integration.

Keywords: digital health, medical emergencies, blood donation, hospital beds, ambulances, geolocation, real time, machine learning, Django REST Framework, React, Senegal.

LISTE DES FIGURES

Figure 1 : Architecture fonctionnelle : cheminement d'une requête, de l'interface à la base de données et retour 22
Figure 2 : Architecture logicielle en couches du backend Django 23
Figure 3 : Hiérarchie des acteurs (relations de généralisation) 24
Figure 4 : Cas d'utilisation de l'administrateur général 25
Figure 5 : Cas d'utilisation du gestionnaire des lits 26
Figure 6 : Cas d'utilisation du gestionnaire du sang 27
Figure 7 : Cas d'utilisation du régulateur SAMU 28
Figure 8 : Cas d'utilisation des utilisateurs mobiles 28
Figure 9 : Diagramme de cas d'utilisation global 30
Figure 10 : Séquence d'authentification et d'ouverture du temps réel 32
Figure 11 : Séquence de la demande de sang urgente jusqu'à la confirmation du don 33
Figure 12 : Séquence de l'admission d'un patient 34
Figure 13 : Séquence de la régulation d'une mission, de l'appel à l'arrivée à l'hôpital 35
Figure 14 : Séquence de la prévision des pénuries à la demande 36
Figure 15 : Diagramme de séquence global de Jappo Dundu 36
Figure 16 : Diagramme de classes de Jappo Dundu 38
Figure 17 : Page d'accueil publique de Jappo Dundu 45
Figure 18 : Écran de connexion de l'espace professionnel 46
Figure 19 : Refus d'une connexion avec un mot de passe erroné 47
Figure 20 : Accès refusé à un module hors du rôle de l'utilisateur 48
Figure 21 : Tableau de bord de supervision nationale 49
Figure 22 : Vue complète du tableau de bord national 50
Figure 23 : Formulaire de création d'un établissement 51
Figure 24 : Création d'un compte : contrôle de saisie puis formulaire valide 51
Figure 25 : Compte créé et rattaché à son établissement 52
Figure 26 : Prévisions de pénurie de sang à 7 jours 53
Figure 27 : Réponse de l'assistant Jappo IA, chiffrée et sourcée 54
Figure 28 : Tableau de bord du gestionnaire du sang (CNTS) 55
Figure 29 : Formulaire de demande de sang urgente 55
Figure 30 : Demande publiée et première vague d'alertes envoyée 56
Figure 31 : Réception d'une demande urgente dans l'application mobile 57
Figure 32 : Engagement de la donneuse et carte donneur présentée au centre 58
Figure 33 : Réponse de la donneuse reçue en temps réel par le CNTS 59
Figure 34 : Don confirmé : une poche collectée sur deux 60
Figure 35 : Suivi du donneur après le don 61
Figure 36 : Tableau de bord de l'administrateur des lits 61
Figure 37 : Capacités des services de l'établissement 62
Figure 38 : Réservation d'un lit pour un patient attendu 63
Figure 39 : Admission d'un patient aux urgences 63
Figure 40 : Recherche de lits libres dans les établissements proches 64
Figure 41 : Flotte SAMU : véhicules et interventions sur la carte 65
Figure 42 : Création d'une mission à partir d'un appel 65
Figure 43 : Régulation de la mission : affectation puis choix de l'hôpital 66
Figure 44 : Déroulement de la mission dans l'application de l'ambulancier 67
Figure 45 : Pré-alerte de l'hôpital : patient en approche 68
Figure 46 : Swagger UI : documentation de l'API Jappo Dundu 69

Figure 47 : Liste des points d'accès et schémas d'authentification 69
Figure 48 : Contrat d'une route et exécution d'un appel depuis Swagger 70
Figure 49 : Requête PostGIS de recherche de proximité (geo/postgis.py, simplifiée) 70
Figure 50 : Diagramme de déploiement en production 74
Figure 51 : Extrait du flux d'intégration continue (.github/workflows/ci.yml) 74
Figure 52 : Définition des rôles et des domaines (backend/security/roles.py, extrait) 81
Figure 53 : Admission sans condition de course (backend/lits/services.py, extrait)..... 81
Figure 54 : Configuration de drf-spectacular (backend/core/settings.py, extrait)..... 82

LISTE DES TABLEAUX

Tableau 1 : Sigles et abréviations utilisés dans le rapport 7
Tableau 2 : Répartition des responsabilités au sein de l'équipe 13
Tableau 3 : Étude comparative des solutions de santé numérique 16
Tableau 4 : Besoins fonctionnels de Jappo Dundu 18
Tableau 5 : Besoins techniques (non fonctionnels) 19
Tableau 6 : Itérations du projet, reconstituées à partir de l'historique Git 20
Tableau 7 : Fiche du cas « Créer une demande de sang urgente » 30
Tableau 8 : Fiche du cas « Affecter une ambulance à une mission » 31
Tableau 9 : Fiche du cas « Enregistrer l'admission d'un patient » 31
Tableau 10 : Dictionnaire des principales entités 39
Tableau 11 : Performances du modèle de prévision (données simulées, 99 770 exemples de test) 71
Tableau 12 : Campagnes de tests automatisés et résultats 71
Tableau 13 : Plan de tests fonctionnels et résultats observés 72
Tableau 14 : Environnements du projet 73
Tableau 15 : Bilan des objectifs spécifiques 76
Tableau 16 : Difficultés rencontrées et solutions apportées 77
Tableau 17 : Documentation technique consultée 79
Tableau 18 : Matrice simplifiée des droits par rôle 81
Tableau 19 : Comptes de démonstration (mot de passe : Password123!) 82

LISTE DES SIGLES ET ABREVIATIONS

Tableau 1 : Sigles et abréviations utilisés dans le rapport

Sigle	Signification
ABAC	Attribute-Based Access Control (contrôle d'accès par attributs : établissement, région)
API	Application Programming Interface (interface de programmation)
ASGI	Asynchronous Server Gateway Interface (interface serveur asynchrone de Python)
CI/CD	Continuous Integration / Continuous Delivery (intégration et livraison continues)
CNTS	Centre National de Transfusion Sanguine
CQR	Conformalized Quantile Regression (régression quantile conformalisée)
CSP	Content Security Policy (politique de sécurité du contenu)
DHIS2	District Health Information Software 2
DiSEP	Diplôme Supérieur d'Enseignement Professionnel
DRF	Django REST Framework
FHIR	Fast Healthcare Interoperability Resources (norme d'échange de données de santé)
GPS	Global Positioning System
ISEP-AT	Institut Supérieur d'Enseignement Professionnel Amadou Trawaré
JWT	JSON Web Token (jeton d'authentification signé)
MAE	Mean Absolute Error (erreur absolue moyenne)
MSAS	Ministère de la Santé et de l'Action Sociale
OSRM	Open Source Routing Machine (calcul d'itinéraires routiers)
OTP / TOTP	One-Time Password / Time-based One-Time Password (code à usage unique)
PPP	Projet Personnel Professionnel
RBAC	Role-Based Access Control (contrôle d'accès par rôle)
REST	Representational State Transfer
SAMU	Service d'Aide Médicale Urgente
SIH	Système d'Information Hospitalier
SMS	Short Message Service
SOS	Alerte de détresse déclenchée par un citoyen
UML	Unified Modeling Language
WS	WebSocket (canal de communication bidirectionnel en temps réel)

TABLE DES MATIERES

Remerciements 1
Résumé / Abstract 2
Résumé 2
Abstract 2
Liste des figures 4
Liste des tableaux 6
Liste des sigles et abréviations 7
Table des matières 8
Introduction 10
Contexte 10
Problématique 10
Objectifs 11
Organisation et plan 12
Chapitre 1 : Analyse des besoins 13
1. Présentation du cadre 13
1.1 L'établissement de formation 13
1.2 Le projet et l'équipe 13
1.3 Les acteurs du domaine 14
2. Étude de l'existant / état de l'art 14
2.1 Présentation des solutions existantes 14
2.2 Étude comparative 16
2.3 Présentation théorique des technologies 17
3. Spécifications des besoins 18
3.1 Besoins fonctionnels 18
3.2 Besoins techniques 19
Chapitre 2 : Conception et choix techniques 20
1. Méthodologie 20
2. Schéma d'architecture globale 21
3. Modélisation 23
3.1 Les acteurs 23
3.2 Diagrammes de cas d'utilisation par acteur 24
3.3 Diagramme de cas d'utilisation global 29
3.4 Description textuelle des cas d'utilisation principaux 30
3.5 Diagrammes de séquence par acteur 32
3.6 Diagramme de séquence global 36

3.7 Diagramme de classes 37
4. Choix technique 39
4.1 Outils de développement 40
4.2 Technologies et frameworks 41
Chapitre 3 : Réalisation, tests et déploiement 45
1. Réalisation technique 45
1.1 Authentification et contrôle d'accès 45
1.2 Supervision nationale (administrateur général) 48
1.3 Module Jappo Sang 54
1.4 Module Jappo Lits 61
1.5 Module Jappo Ambulances 64
1.6 Backend et documentation de l'API (Swagger) 68
1.7 Module de prévision des pénuries 70
2. Qualité et tests 71
2.1 Stratégie de tests 71
2.2 Plan de tests fonctionnels 72
3. Déploiement 73
3.1 Gestion des environnements 73
3.2 Conteneurisation et architecture de production 73
3.3 Intégration continue 74
Conclusion 76
Bilan technique et fonctionnel 76
Bilan personnel 76
Perspectives 77
Bibliographie et webographie 78
Ouvrages et articles scientifiques 78
Rapports et textes officiels 78
Documentation technique (consultée en octobre et novembre 2026) 79
Annexes 81
Annexe 1 : Rôles et matrice des droits d'accès 81
Annexe 2 : Extrait de code — contrôle d'accès par rôle 81
Annexe 3 : Extrait de code — admission atomique d'un patient 81
Annexe 4 : Extrait de configuration — documentation Swagger 82
Annexe 5 : Comptes de démonstration 82

INTRODUCTION

Contexte

Les technologies numériques occupent une place croissante dans les systèmes de santé : elles facilitent le partage de l'information, l'optimisation des ressources et la coordination entre les acteurs de la chaîne de soins (OMS, 2021). Au Sénégal, ce contexte est favorable : selon l'Autorité de Régulation des Télécommunications et des Postes, le pays comptait 21 125 421 abonnements Internet au quatrième trimestre 2024, l'Internet mobile représentant 96,46 % du parc (ARTP, 2025).

Le système de santé sénégalais reste pourtant confronté à des tensions structurelles. Le pays comptait 18 126 390 habitants lors du recensement de 2023 (ANSD, 2024), et la répartition des ressources médicales y demeure inégale entre les régions (MSAS, 2022). En matière de transfusion, 136 347 dons ont été enregistrés en 2024, pour un déficit estimé à 25 % par rapport aux besoins réels (MSAS, 2025). Lors d'une urgence (hémorragie de la délivrance, accident de la route, crise drépanocytaire), la recherche d'une poche de sang, d'un lit de réanimation ou d'une ambulance disponible se fait encore largement par téléphone, établissement par établissement.

C'est dans ce contexte que s'inscrit **Jappo Dundu**, « Soigner ensemble » en wolof : une plateforme qui interconnecte les établissements de santé, les centres de transfusion, le SAMU et les citoyens donneurs de sang autour d'une information partagée et mise à jour en temps réel.

Problématique

Les observations menées auprès de professionnels de santé et les données publiques convergent vers quatre difficultés :

- **une information dispersée** : chaque hôpital connaît ses propres lits et stocks, mais personne ne dispose d'une vue d'ensemble en temps réel ;
- **une mobilisation des donneurs lente** : lorsqu'un groupe sanguin manque, la recherche de donneurs compatibles repose sur des appels et des relais informels ;
- **une orientation des patients à l'aveugle** : une ambulance peut être dirigée vers un établissement déjà saturé, faute de connaître les lits réellement libres ;

- **des pénuries subies plutôt qu'anticipées** : sans exploitation de l'historique des stocks, les ruptures sont constatées au lieu d'être prévues.

D'où la question centrale de ce projet :

> *Comment concevoir et développer une plateforme numérique sécurisée qui coordonne, en temps réel, les dons de sang, les capacités hospitalières et les ambulances, afin de réduire les délais de prise en charge des urgences médicales au Sénégal ?*

Objectifs

Objectif général

Concevoir, développer et livrer une plateforme intégrée de coordination des ressources sanitaires (sang, lits, ambulances), accessible sur le web aux établissements et sur mobile aux donneurs et aux ambulanciers, exploitant la géolocalisation, le temps réel et l'apprentissage automatique.

Objectifs spécifiques

1. **Jappo Sang** : permettre de publier une demande de sang urgente et d'alerter automatiquement les donneurs compatibles et aptes les plus proches, puis de suivre leurs réponses jusqu'à la confirmation du don.
2. **Jappo Lits** : donner une visibilité en temps réel sur les lits libres par service, enregistrer admissions et sorties, réserver un lit pour un patient en route et trouver les lits libres les plus proches.
3. **Jappo Ambulances** : créer une mission à partir d'un appel, affecter l'ambulance la plus proche par temps de trajet, suivre sa position et pré-alerter l'hôpital de destination.
4. **Pilotage** : offrir à chaque rôle un tableau de bord adapté et prévoir les pénuries de sang à 7 jours à l'aide d'un modèle d'apprentissage automatique.
5. **Qualité et sécurité** : garantir un contrôle d'accès strict par rôle et par établissement, documenter l'API (Swagger) et vérifier le système par des tests automatisés et une intégration continue.

Organisation et plan

Ce rapport suit le canevas du Projet Personnel Professionnel de l'ISEP-AT. Le **chapitre 1** analyse les besoins : il présente le cadre du projet, étudie les solutions existantes et spécifie les besoins fonctionnels et techniques. Le **chapitre 2** expose la conception : méthodologie, architecture globale, modélisation UML (cas d'utilisation, séquences, classes) et choix techniques. Le **chapitre 3** présente la réalisation à travers les parcours de chaque module illustrés par des captures d'écran, la stratégie de tests et le déploiement. La conclusion dresse le bilan technique, fonctionnel et personnel et ouvre des perspectives.

CHAPITRE 1 : ANALYSE DES BESOINS

Ce chapitre situe le projet dans son cadre, analyse les solutions existantes en santé numérique pour en dégager les forces et les limites, puis spécifie les besoins fonctionnels et techniques auxquels Jappo Dundu répond.

1. Présentation du cadre

1.1 L'établissement de formation

Ce projet est réalisé à l'**Institut Supérieur d'Enseignement Professionnel Amadou Trawaré (ISEP-AT)** de Diamniadio, établissement public placé sous la tutelle du Ministère de l'Enseignement Supérieur, de la Recherche et de l'Innovation. Les ISEP forment des techniciens supérieurs directement opérationnels et délivrent le Diplôme Supérieur d'Enseignement Professionnel (DiSEP). Le Projet Personnel Professionnel (PPP) en est l'épreuve de synthèse : l'équipe y conduit un projet complet, de l'analyse du besoin jusqu'au déploiement, dans le métier de **Technicien Supérieur en Analyse de Performance Digitale**.

1.2 Le projet et l'équipe

Le sujet retenu est la **conception et le développement d'une plateforme intégrée de coordination des ressources sanitaires** : disponibilité des lits hospitaliers, dons de sang et services ambulanciers. Le travail est mené par une équipe de quatre apprenants, chacun responsable d'un domaine, selon la règle « API d'abord » : le backend définit le format des échanges et les interfaces web et mobile le consomment.

Tableau 2 : Répartition des responsabilités au sein de l'équipe

Membre	Domaine	Responsabilités principales
Pape Alioune Sene	Chef d'équipe, mobile	Initialisation du dépôt et de l'architecture ; application mobile Expo (donneurs, ambulanciers) ; notifications push ; interopérabilité SIH ; récupération de compte.
Ibrahima Khalilou Diallo	Backend et base de données	Modèles et logique métier des modules sang, lits, ambulances et utilisateurs ; recherche géographique PostGIS ; sérialisation des données.
Serigne Mbacké Faye	Frontend web	Application React : routage, composants partagés, modules de gestion, cartes Leaflet, tableaux de bord et connexion temps réel.

Membre	Domaine	Responsabilités principales
El Hadji Massogui Diop	IA, DevOps et sécurité	Sécurité de l'API (JWT, rôles, limitation de débit), temps réel (Channels, Redis), modèle de prévision des pénuries, intégration continue, déploiement et documentation Swagger.

Source : README du dépôt et en-têtes « Auteur » des fichiers source.

Le code est versionné sur GitHub : l'historique compte **70 commits** entre le 10 septembre et le 31 octobre 2026. Chaque fichier du backend indique en en-tête le membre responsable, ce qui assure la traçabilité des contributions.

1.3 Les acteurs du domaine

La coordination d'une urgence fait intervenir plusieurs acteurs dont les besoins diffèrent :

- **les hôpitaux et cliniques**, qui gèrent leurs capacités d'accueil par service (urgences, réanimation, maternité…) et expriment des besoins en sang ;
- **le CNTS et les centres régionaux de transfusion**, qui coordonnent les stocks de produits sanguins et la mobilisation des donneurs ;
- **le SAMU (numéro 15)**, qui régule les appels d'urgence et engage les ambulances ;
- **les ambulanciers**, sur le terrain, qui doivent connaître leur mission et l'hôpital de destination ;
- **les citoyens donneurs de sang**, dont la générosité doit être sollicitée au bon moment et au bon endroit ;
- **le Ministère de la Santé**, qui a besoin d'une vision nationale consolidée pour piloter le réseau.

2. Étude de l'existant / état de l'art

2.1 Présentation des solutions existantes

Les systèmes d'information hospitaliers (SIH)

Les SIH gèrent les activités administratives, médicales et financières d'un établissement (Shortliffe et Cimino, 2014). Ils améliorent la gestion interne et la traçabilité des actes, mais restent centrés sur un seul établissement, sans coordination interinstitutionnelle en temps réel (Haux et al., 2004).

DHIS2

DHIS2 est une plateforme open source de collecte, d'analyse et de visualisation des données sanitaires, développée par le centre HISP de l'Université d'Oslo et utilisée à l'échelle nationale par plus de 75 pays (Braa et Sahay, 2017 ; DHIS2, 2024). Une méta-synthèse portant sur onze pays en souligne les forces (accès aux données, aide à la décision) et les défis (qualité de saisie, besoin de formation) (Dehnavieh et al., 2019). DHIS2 est conçu pour l'agrégation statistique, non pour la gestion opérationnelle des urgences : il ne recherche pas de donneurs, ne régule pas d'ambulances et n'affiche pas les lits libres en direct.

OpenMRS

OpenMRS est un dossier médical électronique open source déployé dans plus de quarante pays d'Afrique et d'Asie (Verma et al., 2021 ; Mamlin et al., 2006). Gratuit et modulaire, il est orienté vers le suivi clinique individuel et ne propose pas de coordination collective des ressources d'urgence.

LifeBank (Nigeria)

LifeBank met en relation établissements de santé et banques de sang et assure la livraison de produits sanguins, avec un impact documenté au Nigeria (Trelstad et al., 2020 ; Faheem et Dutta, 2023). Sa spécialisation porte sur la chaîne logistique du sang ; elle ne couvre ni les lits hospitaliers ni la régulation des ambulances.

Les outils de gestion des banques de sang et des ambulances

Les logiciels de banque de sang assurent l'enregistrement des donneurs et la traçabilité des poches (OMS, 2020), sans géolocalisation des donneurs ni lien avec les capacités hospitalières. Les travaux sur la régulation ambulancière montrent l'intérêt d'un système d'information géographique pour réduire les temps d'intervention (Peleg et Pliskin, 2004), mais dans des contextes éloignés du Sénégal et sans intégration du sang et des lits.

La situation au Sénégal

La coordination transfusionnelle relève du CNTS et la régulation des urgences du SAMU national. Les échanges préliminaires de l'équipe avec des professionnels de santé ont fait ressortir un besoin d'interconnexion entre ces acteurs. Parmi les solutions étudiées, aucune ne réunit dans un même outil la gestion du sang, des capacités hospitalières et du transport sanitaire.

2.2 Étude comparative

Le tableau suivant compare quatre solutions selon les critères fonctionnels qui structurent la coordination d'une urgence.

Tableau 3 : Étude comparative des solutions de santé numérique

Critère	DHIS2	OpenMRS	LifeBank	Jappo Dundu
Dossier médical du patient	Partiel	✓	✗	✗ (hors périmètre)
Stocks de sang par groupe	Partiel	✗	✓	✓
Alerte des donneurs compatibles proches	✗	✗	Partiel	✓
Lits disponibles en temps réel	Partiel	Partiel	✗	✓
Régulation et suivi GPS des ambulances	✗	✗	✗	✓
Notifications push et SMS	Partiel	✗	Partiel	✓
Tableau de bord national	✓	Partiel	✗	✓
Prévision des pénuries (apprentissage automatique)	Partiel	✗	✗	✓ (données simulées)
Interopérabilité (API, FHIR, export DHIS2)	✓	✓	✗	✓
Contrôle d'accès par rôle et périmètre	✓	✓	Partiel	✓
Maturité et déploiements documentés	✓✓	✓✓	✓	Prototype
Adaptation au contexte sénégalais	Non spécifique	Non spécifique	Non spécifique	Ciblée

Légende : ✓✓ fonctionnalité avancée et documentée ; ✓ couverte ; Partiel couverture partielle ; ✗ non couverte. Sources : documentation des éditeurs et articles cités.

DHIS2 et OpenMRS surpassent Jappo Dundu en maturité et en retour d'expérience ; le projet ne vise pas à les remplacer mais à combler un vide différent : la **coordination opérationnelle en temps réel** de la chaîne des urgences. LifeBank démontre la viabilité d'une solution africaine ciblée, sans dimension hospitalière ni transport. L'avantage de Jappo Dundu tient à l'intégration de ces trois ressources et à son adaptation au contexte local : groupes sanguins et règles d'aptitude au don, quatorze régions administratives, numéros sénégalais, calendrier de Ramadan, de la Tabaski et du Grand Magal dans les prévisions.

2.3 Présentation théorique des technologies

Architecture client-serveur et API REST

Le style REST (Representational State Transfer) organise un service web autour de **ressources** identifiées par des URL (demandes de sang, capacités, missions) et manipulées par les méthodes HTTP standard : GET pour lire, POST pour créer ou déclencher une action, PATCH et PUT pour modifier, DELETE pour supprimer. Le serveur est sans état : chaque requête porte son propre jeton d'authentification, ce qui permet de servir indifféremment l'application web, l'application mobile et les logiciels hospitaliers.

Authentification par jeton JWT

Un **JSON Web Token** est un jeton signé par le serveur qui contient l'identité de l'utilisateur et une date d'expiration. Après la connexion, le client reçoit un jeton d'accès de courte durée (30 minutes) et un jeton de rafraîchissement (7 jours) ; le premier accompagne chaque requête dans l'en-tête `Authorization: Bearer`. La rotation et la mise en liste noire des jetons de rafraîchissement permettent de fermer une session à tout moment.

Communication temps réel par WebSocket

Le protocole WebSocket ouvre un canal permanent et bidirectionnel entre le navigateur et le serveur. Contrairement au modèle requête-réponse, le serveur peut **pousser** une information dès qu'elle change : un lit libéré, une nouvelle demande de sang ou la position d'une ambulance apparaissent instantanément sur tous les écrans concernés.

Systèmes d'information géographique et PostGIS

PostGIS ajoute à PostgreSQL des types et des fonctions géographiques. La fonction `ST_DWithin` sélectionne les objets situés à moins d'une distance donnée sur la sphère terrestre, et l'opérateur `<->`, appuyé sur un index spatial GiST, trie les résultats du plus proche au plus éloigné sans parcourir toute la table. C'est le fondement de la recherche des donneurs, des lits et des ambulances les plus proches.

Apprentissage automatique pour la prévision

La prévision des stocks de sang est un problème de **régression** : prédire une quantité future à partir de l'historique. Le *gradient boosting* combine de nombreux arbres de décision simples, chacun corrigeant les erreurs des précédents. En **régression quantile**, le modèle prédit non seulement une valeur centrale mais aussi des bornes (quantiles 10 % et 90 %) ; la **calibration**

conformelle (CQR) ajuste ces bornes sur des données jamais vues pour garantir qu'environ 80 % des valeurs réelles tombent dans l'intervalle annoncé.

3. Spécifications des besoins

3.1 Besoins fonctionnels

Les besoins fonctionnels sont regroupés par module. Chacun porte un identifiant repris dans le plan de tests du chapitre 3.

Tableau 4 : Besoins fonctionnels de Jappo Dundu

N°	Module	Besoin fonctionnel
BF01	Authentification	Se connecter par identifiant, e-mail ou téléphone et mot de passe ; double authentification par code TOTP lorsqu'elle est activée ; se déconnecter.
BF02	Authentification	Restreindre chaque écran et chaque donnée au rôle de l'utilisateur et à son établissement ou sa région.
BF03	Supervision	Superviser en temps réel le réseau national : lits, demandes de sang, ambulances, priorités et carte opérationnelle.
BF04	Supervision	Gérer les établissements (création, modification, désactivation) et les comptes du personnel avec leur rôle et leur rattachement.
BF05	Jappo Sang	Créer une demande de sang urgente (groupe, produit, nombre de poches, urgence, rayon de recherche).
BF06	Jappo Sang	Alerter automatiquement, par vagues, les donneurs compatibles, aptes et disponibles les plus proches (notification push, SMS en repli).
BF07	Jappo Sang	Permettre au donneur de répondre à une alerte et à l'établissement de suivre les réponses puis de confirmer le don.
BF08	Jappo Sang	Retrouver un donneur par sa carte (QR code) ou son téléphone ; enregistrer un don spontané.
BF09	Jappo Sang	Tenir l'inventaire des stocks de poches par groupe et consulter les prévisions de pénurie à 7 jours.
BF10	Jappo Lits	Consulter les capacités par service, enregistrer admissions et sorties, ajuster et confirmer les chiffres.
BF11	Jappo Lits	Réserver un lit pour un patient attendu et rechercher les établissements proches ayant des lits libres.
BF12	Jappo Ambulances	Créer une mission à partir d'un appel et lui affecter l'ambulance disponible la plus proche par temps de trajet.
BF13	Jappo Ambulances	Choisir l'hôpital de destination selon les lits libres et pré-alerter cet hôpital, qui accepte ou refuse le patient.
BF14	Jappo Ambulances	Permettre à l'ambulancier de suivre sa mission, d'en faire progresser les étapes et de partager sa position GPS.
BF15	Application mobile	Créer un compte et un profil donneur, consulter son aptitude, sa carte, son historique, ses badges et les centres proches ; déclencher une alerte SOS.

N°	Module	Besoin fonctionnel
BF16	Assistance	Interroger l'assistant Jappo IA en langage naturel et obtenir une réponse chiffrée, datée et sourcée.
BF17	Interopérabilité	Exposer une API documentée et permettre aux logiciels hospitaliers de synchroniser lits et stocks par clé d'API.

3.2 Besoins techniques

Les besoins non fonctionnels encadrent la qualité attendue du système.

Tableau 5 : Besoins techniques (non fonctionnels)

Exigence	Description et réponse apportée
Sécurité	Mots de passe hachés (PBKDF2) d'au moins 8 caractères ; jetons JWT à durée courte avec rotation ; contrôle d'accès vérifié côté serveur à chaque requête (rôle relu en base, périmètre établissement ou région) ; limitation du débit des requêtes ; en-têtes de sécurité (CSP stricte, X-Frame-Options, HSTS en production) ; journal d'audit des actions sensibles.
Confidentialité	Respect de la loi sénégalaise n° 2008-12 sur les données personnelles : la position d'un donneur sert uniquement à trouver les demandes proches et n'est jamais transmise aux hôpitaux ; les listes de donneurs proches sont anonymes.
Performance	Recherches géographiques indexées (GiST, tri KNN) ; diffusion temps réel au lieu de rechargements ; calculs de prévision exécutés en tâche de fond pour ne jamais bloquer l'API.
Disponibilité et mode dégradé	Repli par SMS sans Internet ; relecture périodique des données si le flux temps réel est interrompu ; dernière liste d'alertes conservée sur le téléphone.
Scalabilité	API sans état derrière un proxy inverse ; temps réel distribué par Redis entre processus ; déploiement conteneurisé (Docker Compose).
Accessibilité et ergonomie	Interfaces en français, adaptées au rôle ; libellés accessibles aux lecteurs d'écran ; thème clair et sombre ; respect de la préférence « mouvement réduit » ; écrans mobiles utilisables d'une main.
Interopérabilité	API REST documentée (OpenAPI 3 / Swagger) ; exports FHIR R4 et DHIS2 ; connecteur SIH par clé d'API.
Maintenabilité	Code modulaire par domaine, tests automatisés (backend, web, mobile, bout en bout) et intégration continue à chaque modification.

Ces besoins étant établis, le chapitre suivant présente la démarche, l'architecture et la modélisation retenues pour y répondre.

CHAPITRE 2 : CONCEPTION ET CHOIX TECHNIQUES

Ce chapitre présente la démarche de projet suivie, l'architecture globale du système, sa modélisation UML puis les technologies et outils retenus.

1. Méthodologie

L'équipe a suivi une **démarche itérative et incrémentale inspirée d'Agile Scrum**. Plutôt qu'un cycle en V, où tout est spécifié avant d'être développé, chaque itération livre un incrément utilisable, présenté et corrigé avant la suivante. Ce choix répond à deux contraintes du projet : des besoins qui se sont précisés au fil des échanges avec les professionnels de santé, et une équipe de quatre personnes travaillant en parallèle sur le backend, le web et le mobile.

Les principes appliqués sont les suivants :

- **API d'abord** : le backend fixe le contrat des échanges (format JSON, aujourd'hui documenté par Swagger) ; le web et le mobile avancent en parallèle sur ce contrat ;
- **un responsable par domaine**, nommé dans l'en-tête de chaque fichier ;
- **un dépôt unique** sur GitHub, des messages de commit normalisés (`feat`, `fix`, `docs`…) et une **intégration continue** qui rejoue tests, typage, analyse statique et construction à chaque envoi ;
- **une version « cœur du sujet »** présentée par défaut : les fonctions avancées restent dans le code et dans les tests, derrière un interrupteur de configuration.

Tableau 6 : Itérations du projet, reconstituées à partir de l'historique Git

Itération	Période	Incrément livré
1. Socle	10 – 29 sept. 2026	Initialisation du backend Django ; sécurité (JWT, rôles), temps réel et infrastructure du modèle de prévision ; cœur métier (sang, lits, ambulances, utilisateurs) et recherche géographique PostGIS.
2. Interfaces	4 – 6 oct. 2026	Application web des hôpitaux et de la régulation ; application mobile des donneurs et notifications push ; prévisions asynchrones, tickets WebSocket, CSP ; intégration continue ; administrateurs de domaine et tableaux de bord ; carte donneur à QR code.
3. Parcours métier	12 – 20 oct. 2026	Fidélisation des donneurs (points, badges), envoi de SMS, audit des rôles ; réservation de lits ; espace ambulancier, gestion de la flotte, régulation complète d'une mission et alerte SOS citoyenne.

Itération	Période	Incrément livré
4. Consolidation	25 – 31 oct. 2026	Mode sombre et parcours mobiles ; fonctions avancées (double authentification, journal d'audit, alertes critiques, exports FHIR et DHIS2) ; version « cœur du sujet » ; refonte visuelle et assistant Jappo IA.
5. Documentation et recette	Nov. 2026	Documentation OpenAPI / Swagger de l'API, modélisation UML, campagne de tests et de captures, rapport.

2. Schéma d'architecture globale

Jappo Dundu adopte une **architecture client-serveur en couches**, organisée autour d'une API REST unique. Trois clients la consomment : l'application web (personnel de santé), l'application mobile (donneurs et ambulanciers) et, de machine à machine, les logiciels hospitaliers. Le serveur applicatif Django applique les règles métier et les droits d'accès, s'appuie sur PostgreSQL / PostGIS pour les données et sur Redis pour diffuser les événements en temps réel.

La figure suivante suit le cheminement complet d'une requête, avec un code couleur par couche : présentation (bleu), exposition (orange), application (vert), données (violet) et services externes (gris). La requête traverse le proxy Caddy, est authentifiée et autorisée, puis traitée par le service métier qui lit ou écrit en base. Une fois la transaction validée, l'événement est publié dans Redis et poussé par WebSocket vers les autres écrans concernés ; les destinataires hors application reçoivent une notification push ou un SMS.

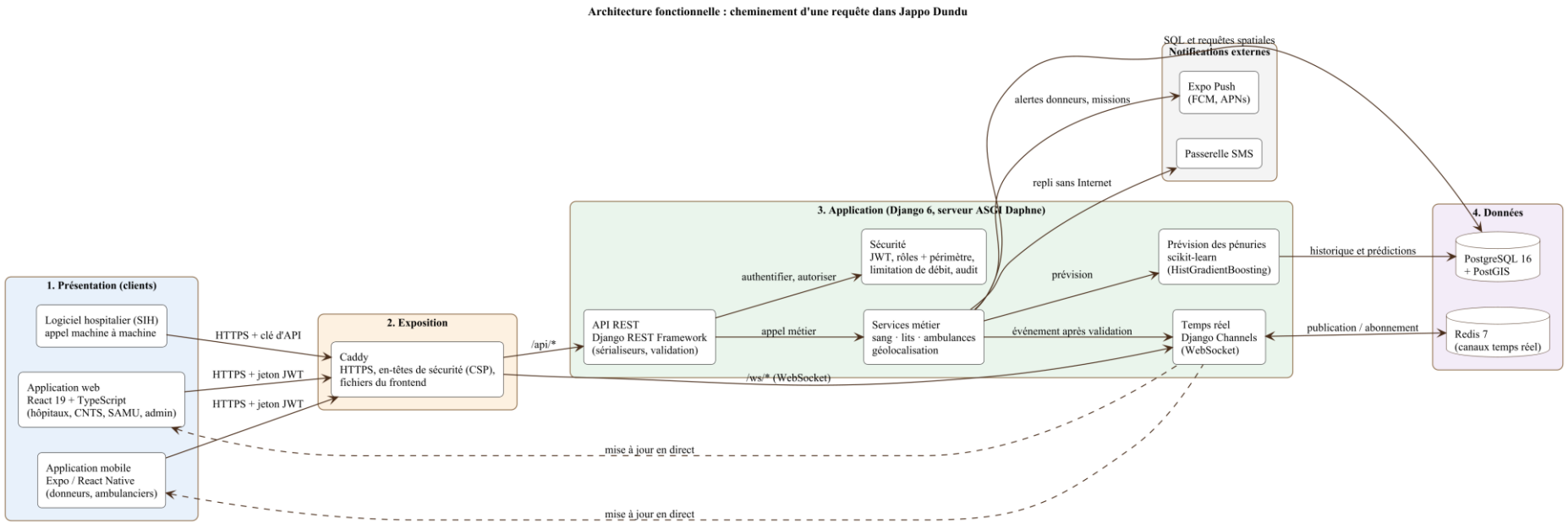


Figure 1 : Architecture fonctionnelle : cheminement d'une requête, de l'interface à la base de données et retour

Côté serveur, Django suit le motif **MVT** (Modèle – Vue – Template), adapté à une API : les *templates* HTML sont remplacés par des sérialiseurs JSON. Le code est organisé en couches aux responsabilités distinctes, des routes jusqu'à la base de données.

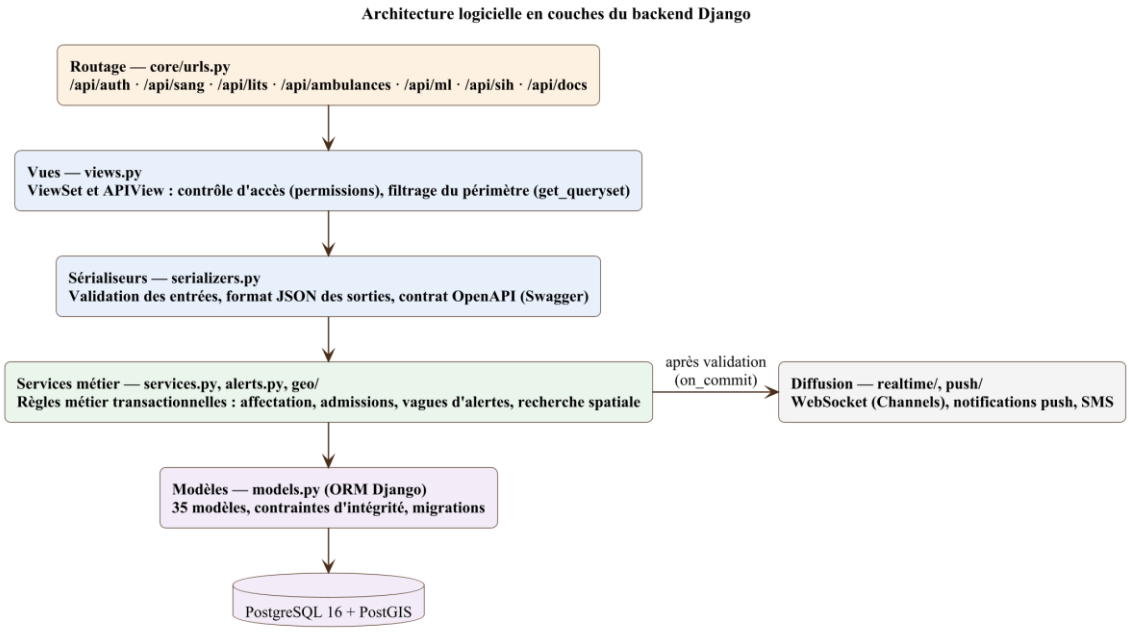


Figure 2 : Architecture logicielle en couches du backend Django

Cette séparation a une conséquence importante pour la sécurité : le contrôle d'accès n'est jamais délégué à l'interface. Une vue vérifie le rôle (permission) **et** filtre les données au périmètre de l'utilisateur (méthode `get_queryset`) ; un utilisateur qui forgerait une requête vers une ressource d'un autre établissement reçoit une erreur 403 ou 404.

3. Modélisation

La modélisation utilise le langage **UML**. Conformément au guide de rédaction, les diagrammes de cas d'utilisation et de séquence sont d'abord présentés par acteur, puis synthétisés dans un diagramme global ; le diagramme de classes est unique. Tous les diagrammes sont écrits en PlantUML (dossier `docs/uml` du dépôt) et exportés en image haute résolution.

3.1 Les acteurs

Huit profils interagissent avec le système. Les acteurs humains se répartissent en utilisateurs web (personnel de santé) et utilisateurs mobiles ; parmi les premiers, l'**administrateur général** correspond au super administrateur et les trois **administrateurs de domaine** (lits, sang, SAMU) aux responsables de structure, rattachés à un établissement. Le logiciel hospitalier (SIH) est un acteur machine, authentifié par clé d'API.

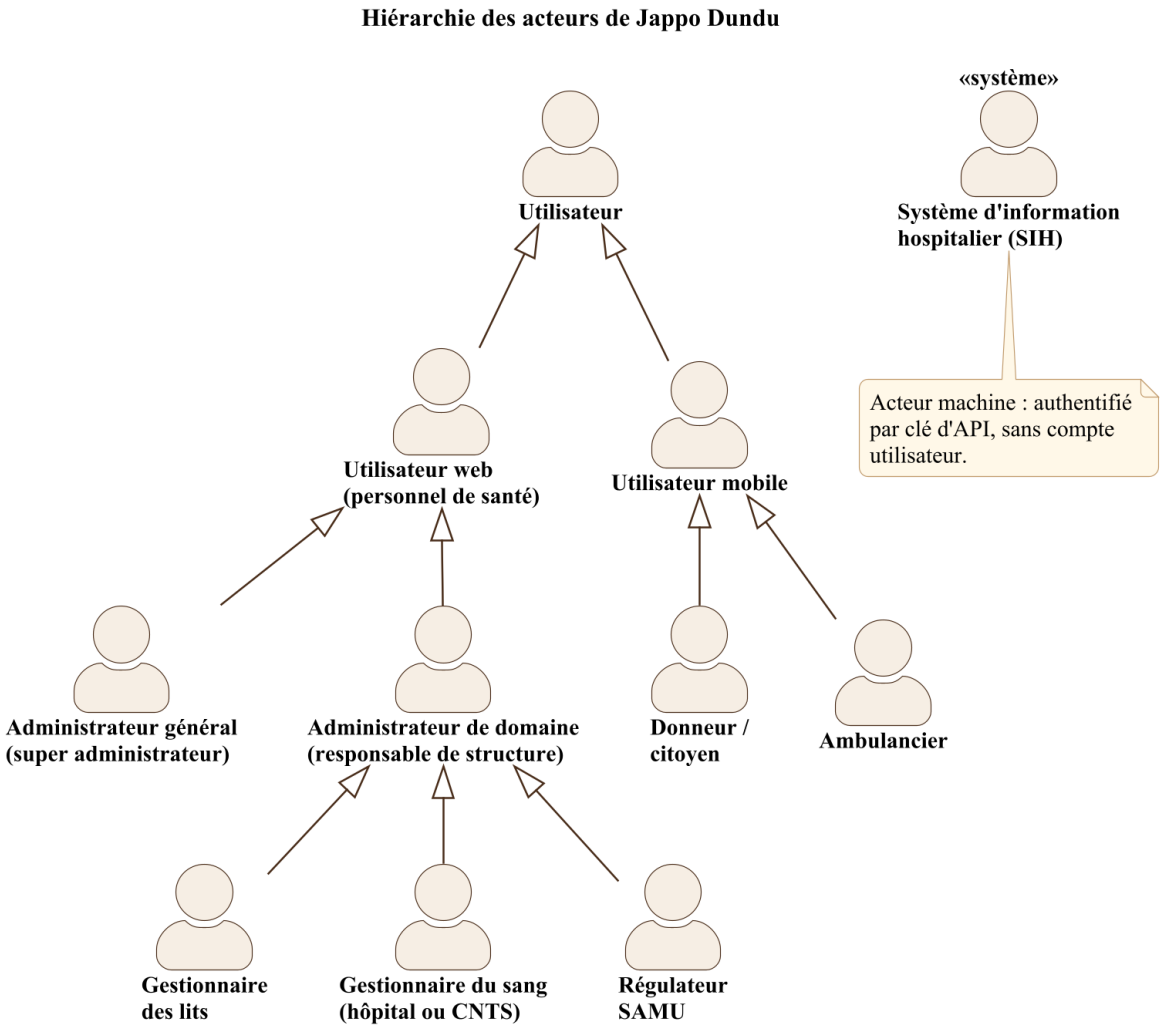


Figure 3 : Hiérarchie des acteurs (relations de généralisation)

3.2 Diagrammes de cas d'utilisation par acteur

Dans les diagrammes suivants, la relation `<<include>>` désigne un cas toujours exécuté par le cas de base (par exemple « S'authentifier ») et `<<extend>>` un cas optionnel qui l'enrichit sous condition (par exemple « Saisir le code de double authentification » lorsqu'elle est activée).

Administrateur général (super administrateur)

L'administrateur général pilote le réseau national : il supervise en temps réel l'ensemble des ressources, gère les établissements et les comptes du personnel, consulte les prévisions nationales, gère les clés d'API des logiciels hospitaliers et interroge l'assistant Jappo IA.

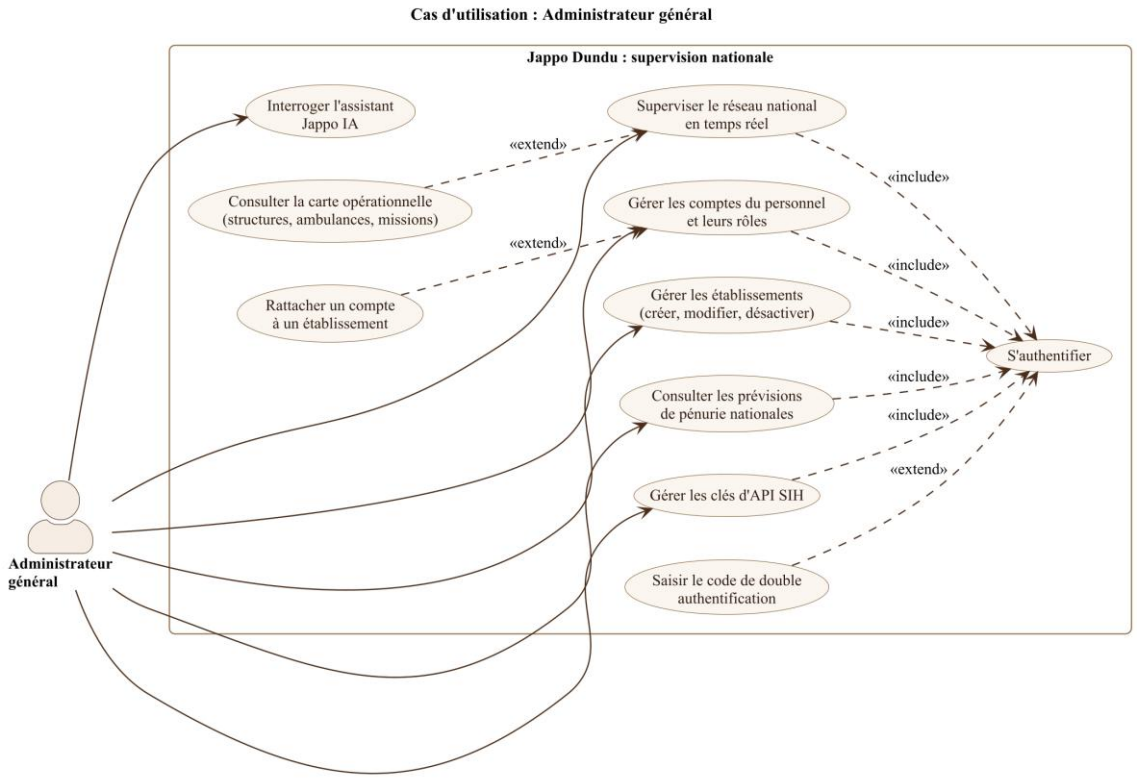


Figure 4 : Cas d'utilisation de l'administrateur général

Gestionnaire des lits (responsable de structure)

Rattaché à un hôpital, il tient à jour les capacités de ses services, enregistre admissions et sorties, réserve des lits et suit les ambulances en route vers son établissement. Chaque modification de capacité est diffusée en temps réel (`<<include>>`).

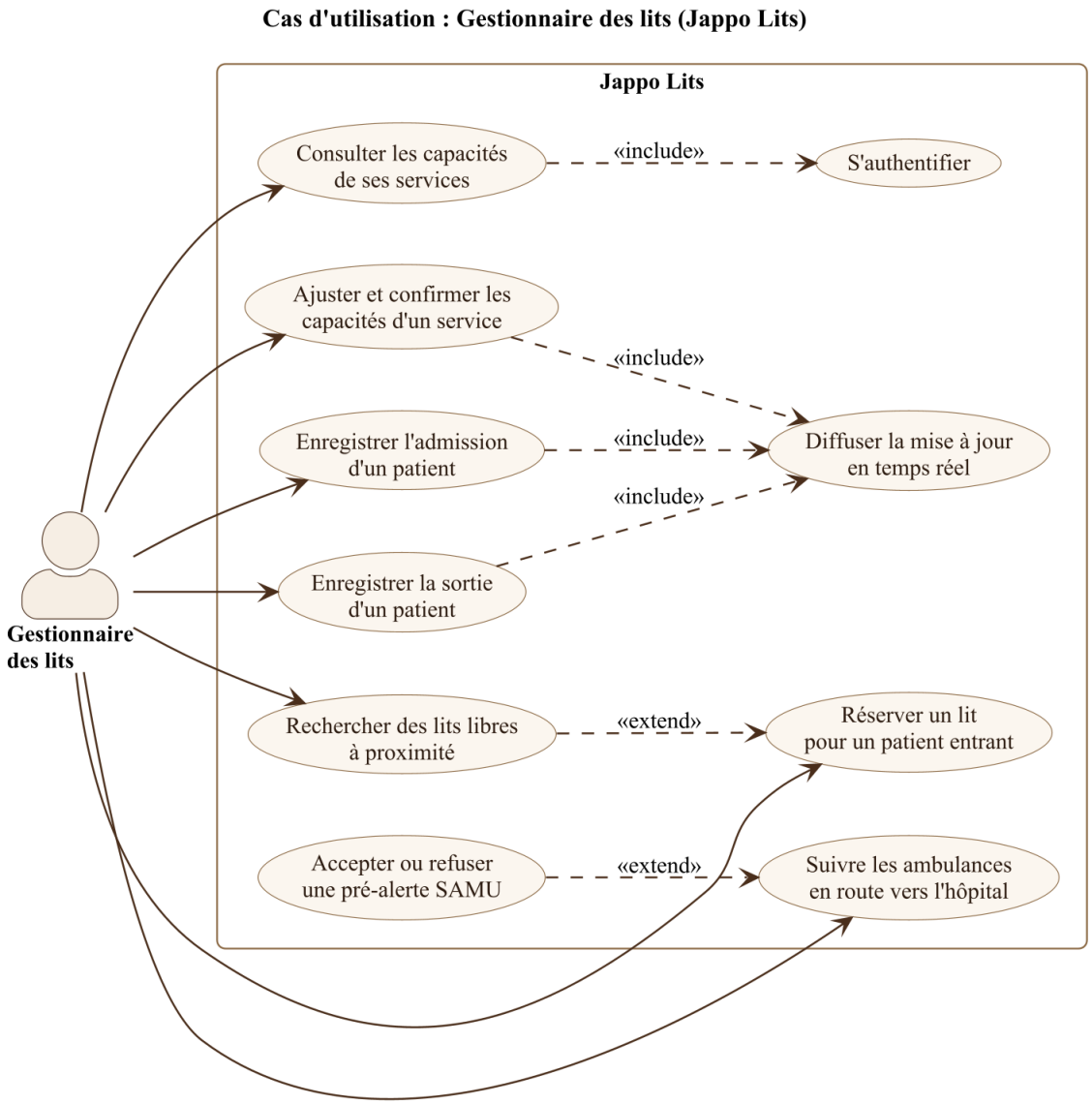


Figure 5 : Cas d'utilisation du gestionnaire des lits

Gestionnaire du sang (hôpital ou CNTS)

Il crée les demandes urgentes, qui incluent l'alerte automatique des donneurs compatibles ; il peut relancer une vague d'alertes (`<<extend>>`), suivre les réponses, confirmer les dons en retrouvant le donneur par sa carte, tenir les stocks et consulter les prévisions. Un gestionnaire rattaché au CNTS agit sur toute sa région.

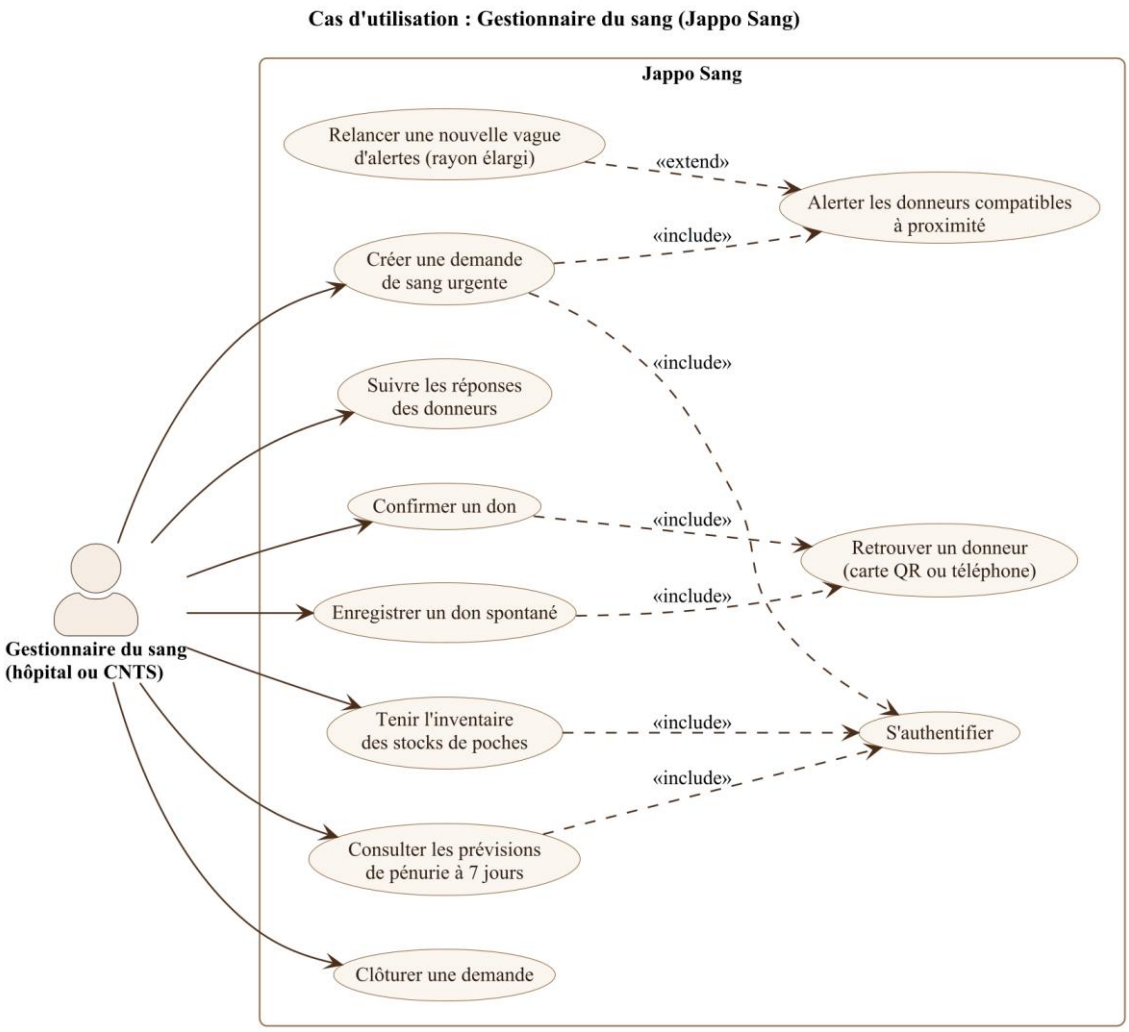


Figure 6 : Cas d'utilisation du gestionnaire du sang

Régulateur SAMU

Il crée une mission à partir d'un appel, ce qui inclut le tri de l'appel ; il affecte une ambulance parmi les plus proches, choisit l'hôpital de destination selon les lits libres, suit la flotte sur la carte, traite les alertes SOS des citoyens et peut annuler une mission en indiquant un motif.

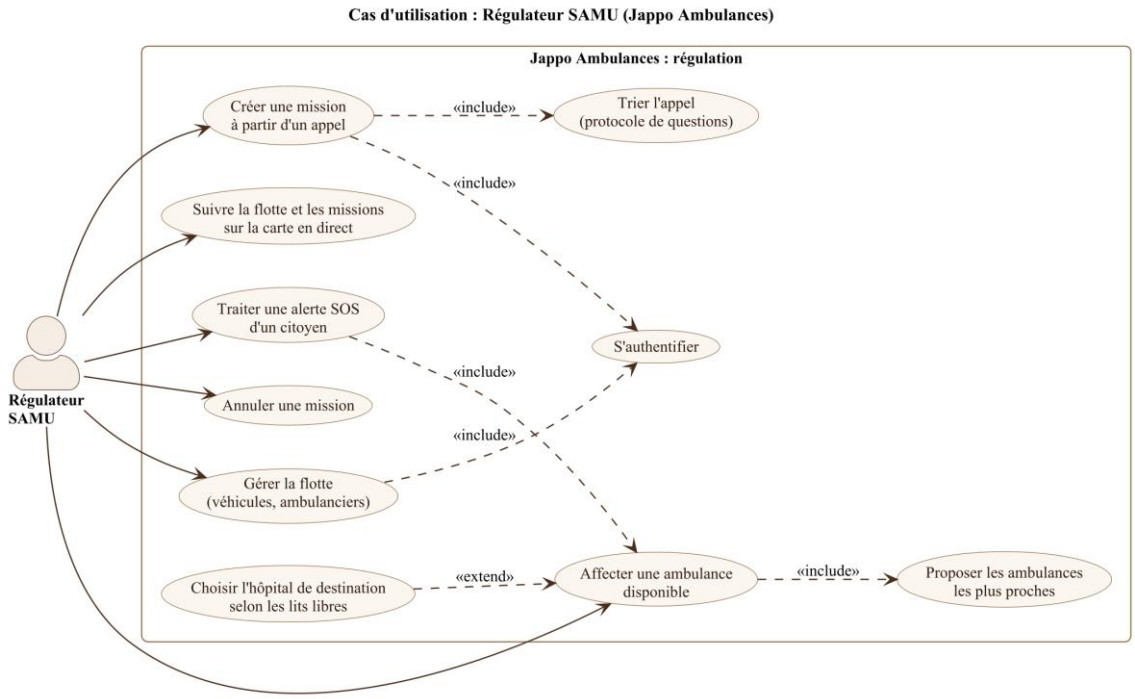
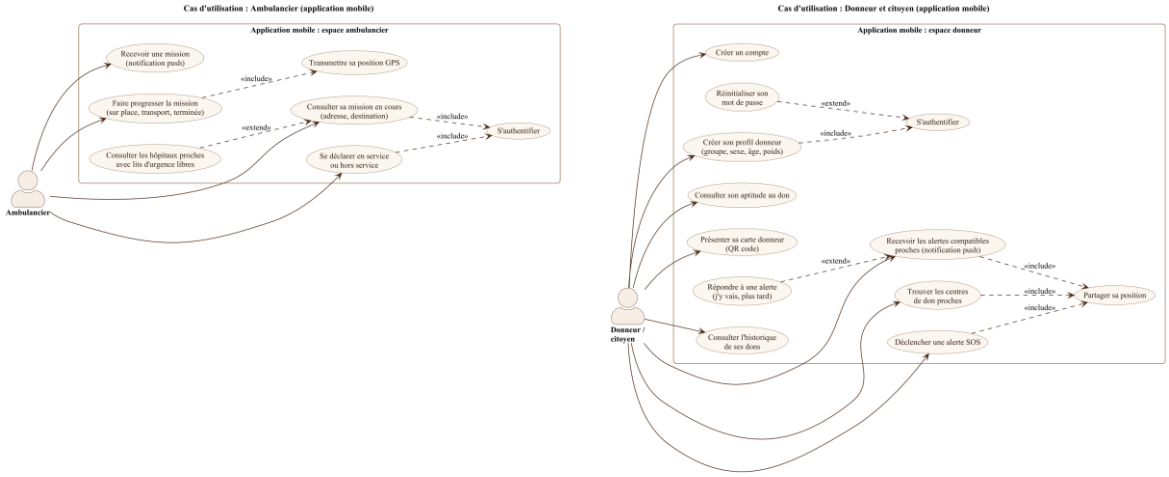


Figure 7 : Cas d'utilisation du régulateur SAMU

Ambulancier et donneur (utilisateurs mobiles)

L'ambulancier reçoit sa mission, en fait progresser les étapes (sur place, transport, remise du patient) en partageant sa position, et se déclare en service ou hors service. Le donneur crée son profil, reçoit les alertes compatibles proches, y répond, présente sa carte au centre de don, consulte son historique et peut déclencher une alerte SOS.



a) Ambulancier *b) Donneur et citoyen*

Figure 8 : Cas d'utilisation des utilisateurs mobiles

3.3 Diagramme de cas d’utilisation global

Le diagramme global rassemble l’ensemble des acteurs et leurs interactions principales : les acteurs web à gauche, les acteurs mobiles et le SIH à droite. Il met en évidence les points de rencontre entre modules : la demande de sang du gestionnaire inclut l’alerte à laquelle répond le donneur, et l’alerte SOS du citoyen inclut la création d’une mission que régule le SAMU.

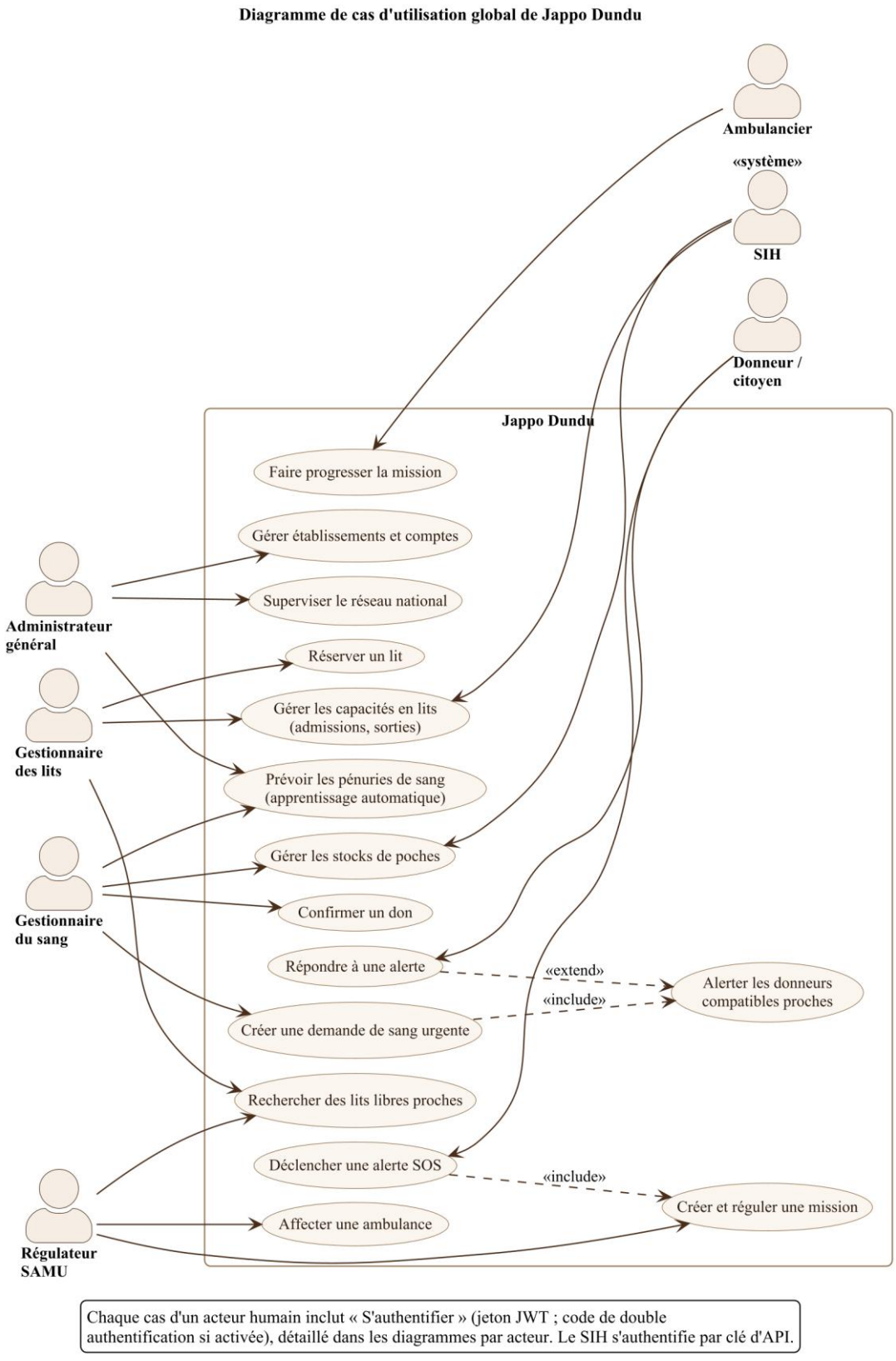


Figure 9 : Diagramme de cas d’utilisation global

3.4 Description textuelle des cas d’utilisation principaux

Tableau 7 : Fiche du cas « Créer une demande de sang urgente »

Rubrique	Description
Acteur principal	Gestionnaire du sang
Préconditions	L'utilisateur est authentifié, son rôle est « administrateur du sang » et son compte est rattaché à un établissement actif.
Scénario nominal	1. L'utilisateur ouvre « Nouvelle demande ». 2. Il saisit le groupe sanguin, le produit, le nombre de poches, l'urgence, le rayon de recherche et des précisions cliniques. 3. Le système valide la saisie et enregistre la demande « ouverte ». 4. Le système diffuse la demande aux tableaux de bord et alerte la première vague de donneurs compatibles, aptes et disponibles les plus proches. 5. Le système affiche la demande et son suivi.
Scénarios alternatifs	3a. Saisie invalide : le système signale les champs en erreur. 4a. Aucun donneur à portée : le rayon est doublé, jusqu'au maximum autorisé. 4b. Urgence « normale » la nuit : l'alerte attend le matin.
Postconditions	Une demande ouverte existe ; des alertes donneurs sont enregistrées et envoyées par push ou SMS.

Tableau 8 : Fiche du cas « Affecter une ambulance à une mission »

Rubrique	Description
Acteur principal	Régulateur SAMU
Préconditions	L'utilisateur est authentifié comme régulateur ; une mission « en attente » existe dans sa région ; au moins une ambulance est disponible.
Scénario nominal	1. Le régulateur déclenche la mission. 2. Le système recherche les ambulances disponibles dans le rayon et les classe par temps de trajet (OSRM, sinon estimation). 3. Le système affecte la plus proche, passe la mission à « affectée », calcule l'heure d'arrivée estimée et prévient l'ambulancier. 4. Le régulateur réserve un lit dans l'hôpital de destination choisi selon les lits libres. 5. L'hôpital est pré-alerté.
Scénarios alternatifs	2a. Aucune ambulance à portée : le système l'indique (erreur 409). 4a. L'hôpital refuse le patient : le régulateur choisit une autre destination. 5a. Annulation : un motif est exigé, l'ambulance et le lit sont libérés.
Postconditions	La mission est affectée, l'ambulance est « en mission » et un lit est réservé à destination.

Tableau 9 : Fiche du cas « Enregistrer l'admission d'un patient »

Rubrique	Description
Acteur principal	Gestionnaire des lits
Préconditions	L'utilisateur est authentifié comme administrateur des lits de cet hôpital.
Scénario nominal	1. L'utilisateur clique « Admettre un patient » sur un service. 2. Le système libère les réservations expirées. 3. Le système augmente le nombre de lits occupés par une mise à jour atomique conditionnelle. 4. Le système affiche la nouvelle disponibilité et la diffuse en temps réel au réseau.
Scénarios alternatifs	3a. Service complet (tous les lits occupés ou réservés) : le système refuse l'admission (erreur 409). 4a. Service vital devenu saturé : une alerte est diffusée à la région.
Postconditions	Les compteurs du service sont à jour sur tous les écrans connectés.

3.5 Diagrammes de séquence par acteur

Les diagrammes de séquence décrivent les échanges entre l'utilisateur, l'interface, l'API, la base de données, le canal temps réel et les services de notification, pour les scénarios principaux et critiques de chaque acteur.

Authentification d'un utilisateur (tous les acteurs humains)

L'utilisateur obtient une paire de jetons après vérification de son mot de passe et, si elle est activée, de son code de double authentification. Pour ouvrir le canal temps réel, l'interface échange son jeton contre un **ticket WebSocket à usage unique** : le jeton n'apparaît jamais dans une URL, où il pourrait être journalisé puis rejoué.

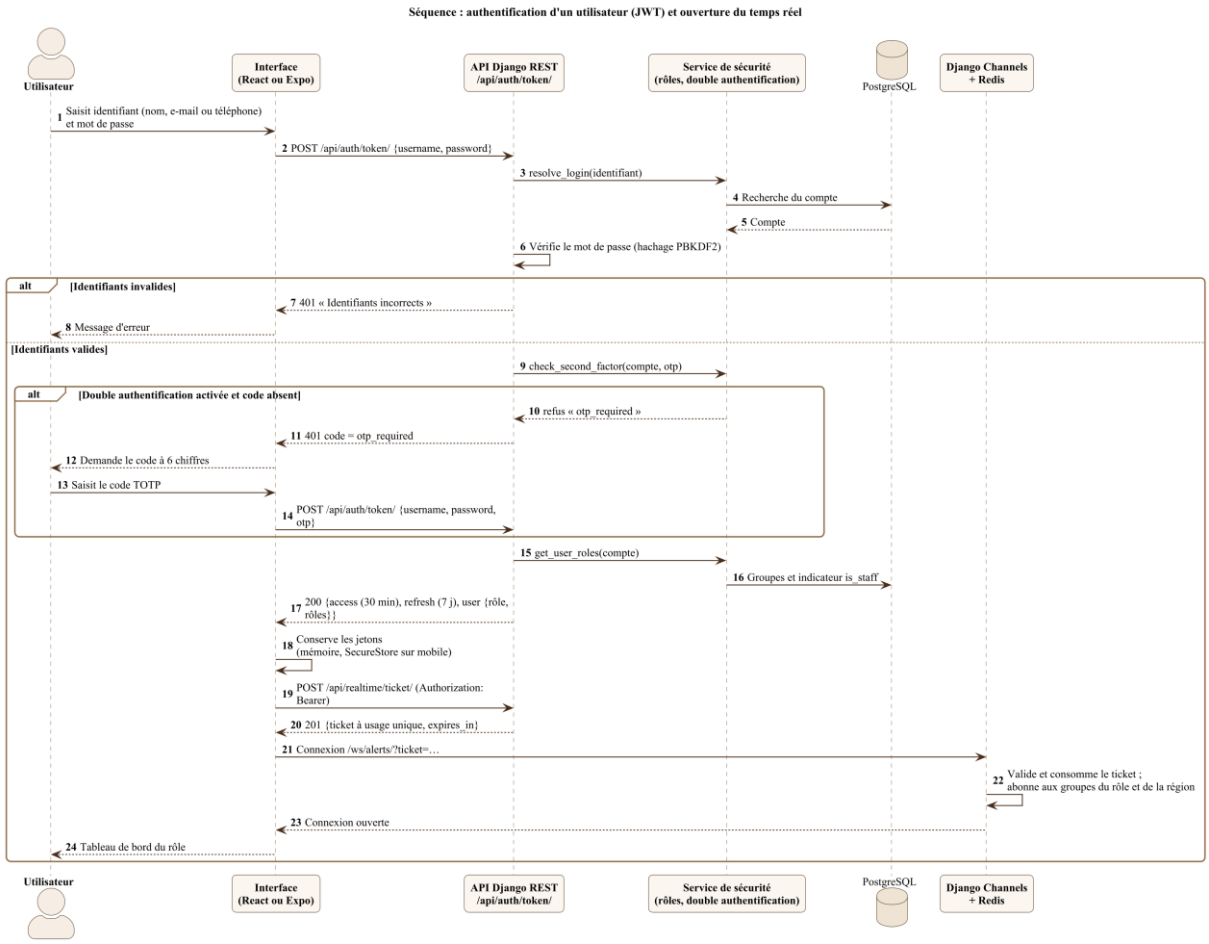


Figure 10 : Séquence d'authentification et d'ouverture du temps réel

Demande de sang urgente (gestionnaire du sang et donneur)

Le scénario critique du module sang traverse le web et le mobile : création de la demande, recherche spatiale des donneurs compatibles, alerte, réponse du donneur puis confirmation du

don au centre. Les diffusions et les notifications partent **après validation de la transaction**, pour qu'aucun écran n'affiche une donnée annulée.

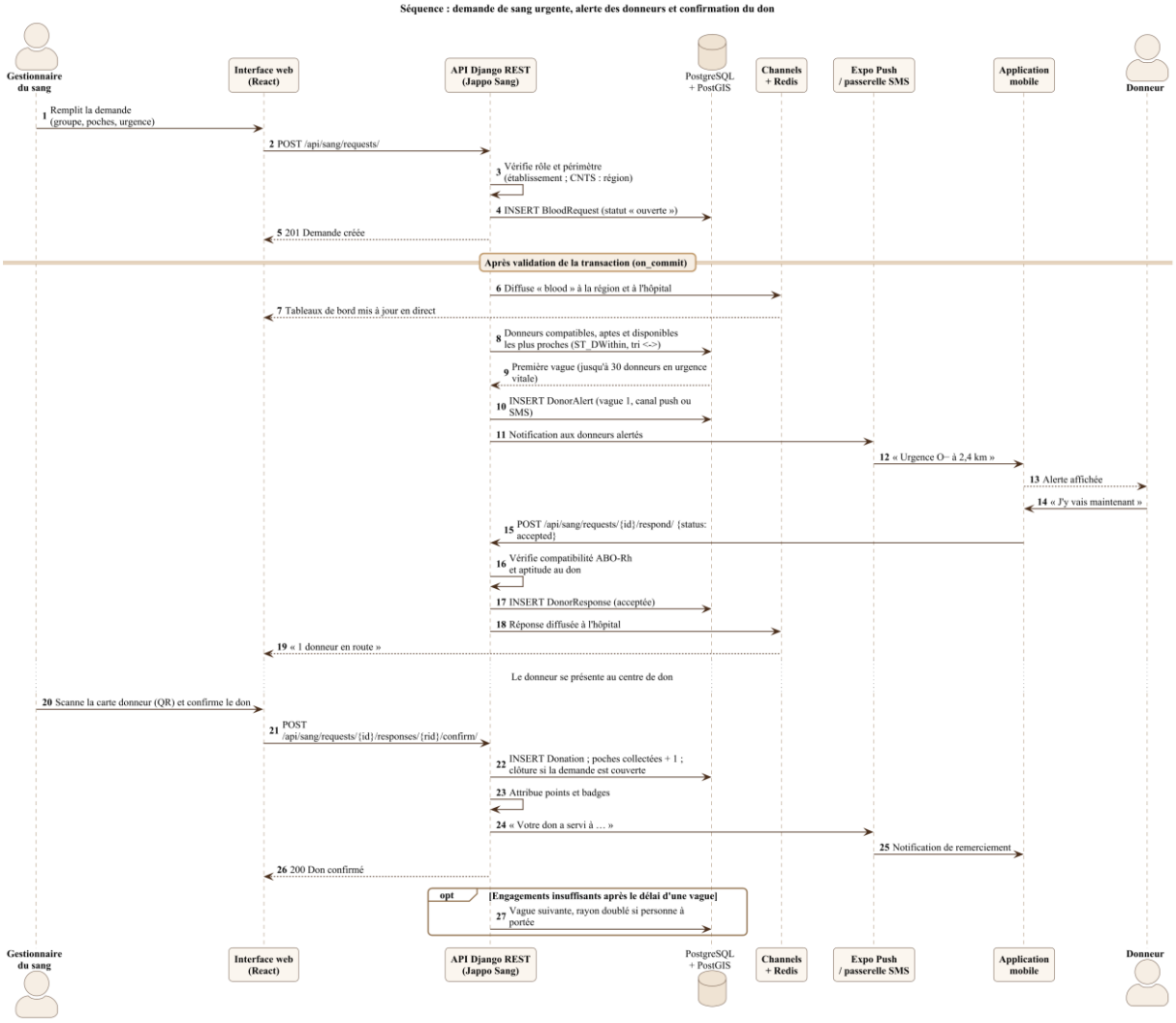


Figure 11 : Séquence de la demande de sang urgente jusqu'à la confirmation du don

Admission d'un patient (gestionnaire des lits)

L'admission repose sur une mise à jour conditionnelle unique : deux admissions simultanées sur le dernier lit libre ne peuvent pas réussir toutes les deux, sans verrou applicatif ni condition de course.

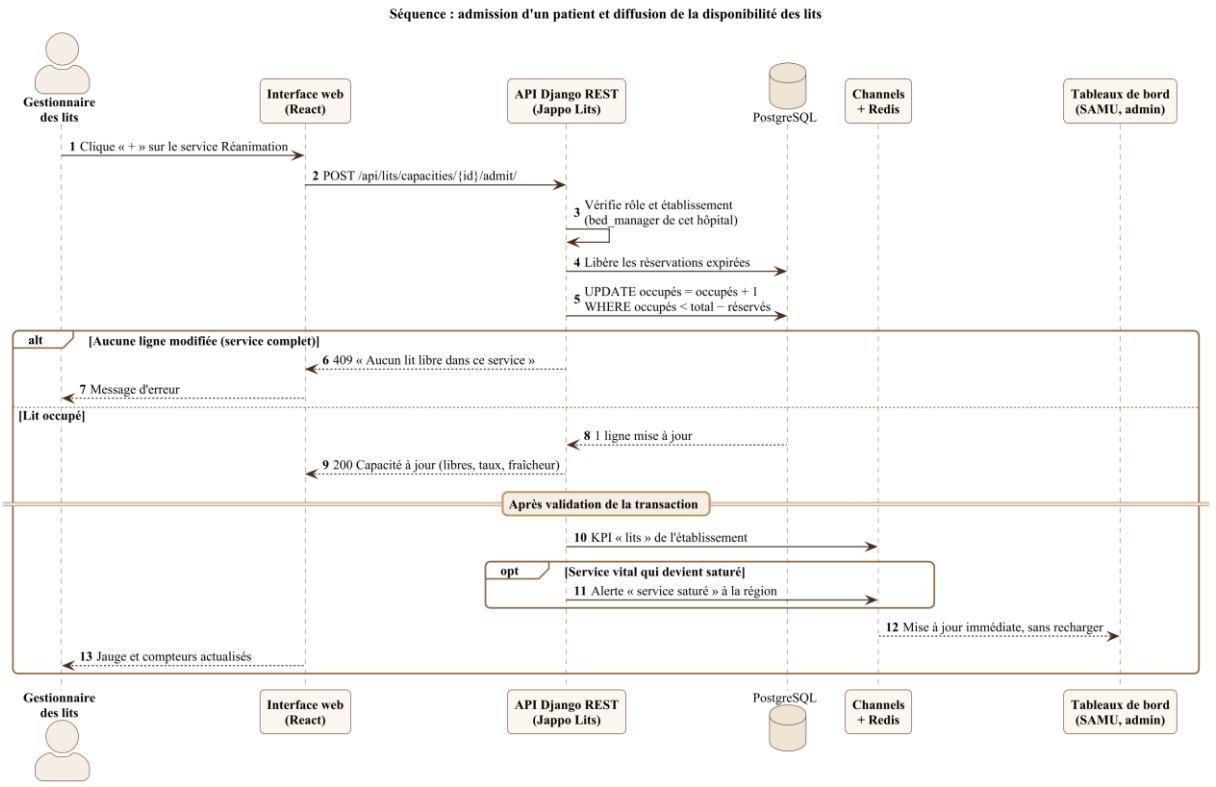


Figure 12 : Séquence de l'admission d'un patient

Régulation d'une mission SAMU (régulateur et ambulancier)

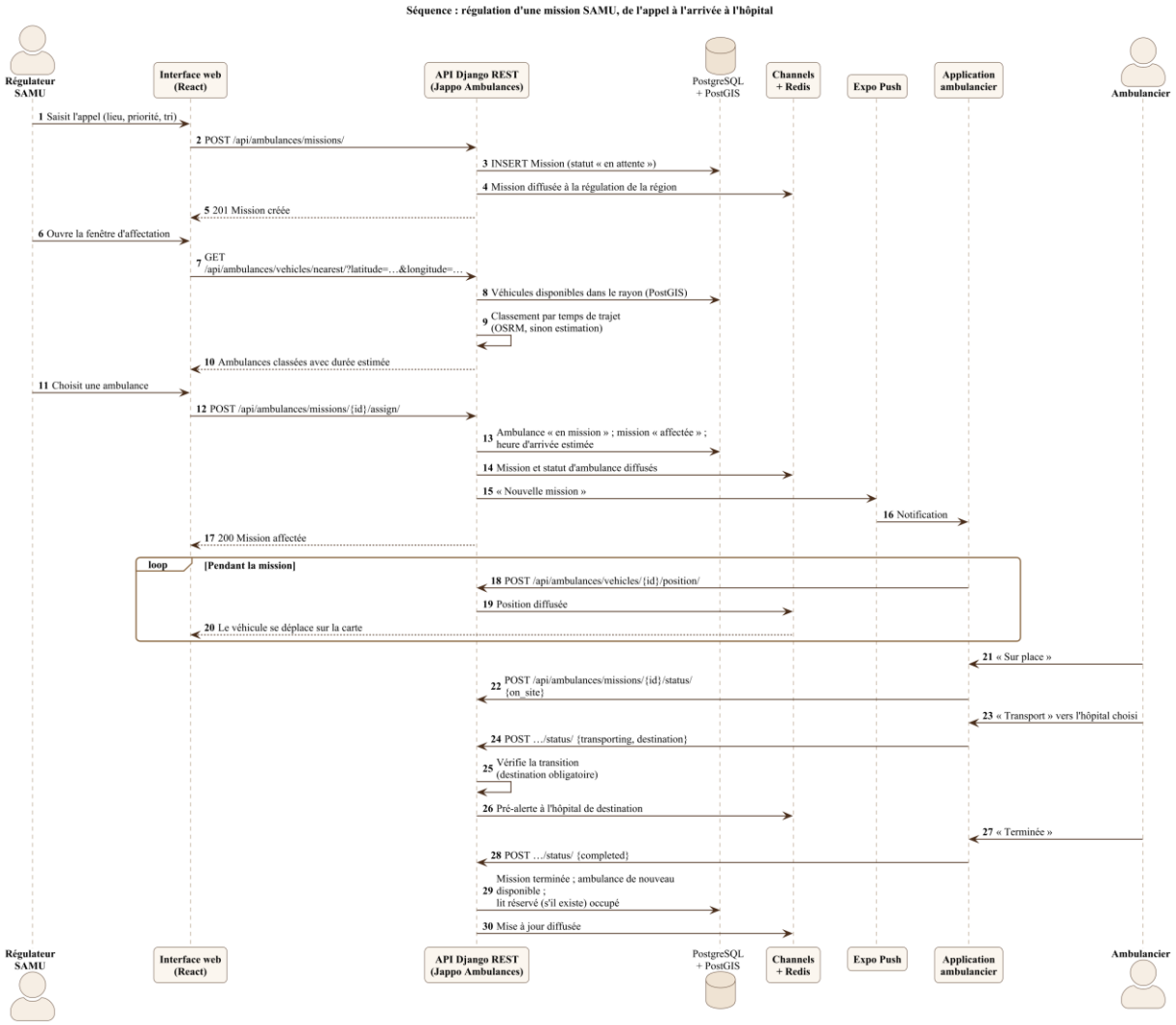


Figure 13 : Séquence de la régulation d'une mission, de l'appel à l'arrivée à l'hôpital

Prévision des pénuries à la demande (gestionnaire du sang)

Un calcul de prévision dure plusieurs secondes. Pour ne pas bloquer l'API, la requête répond immédiatement **202 Accepted** avec un identifiant de tâche ; le calcul s'exécute en arrière-plan et sa fin est annoncée en temps réel.

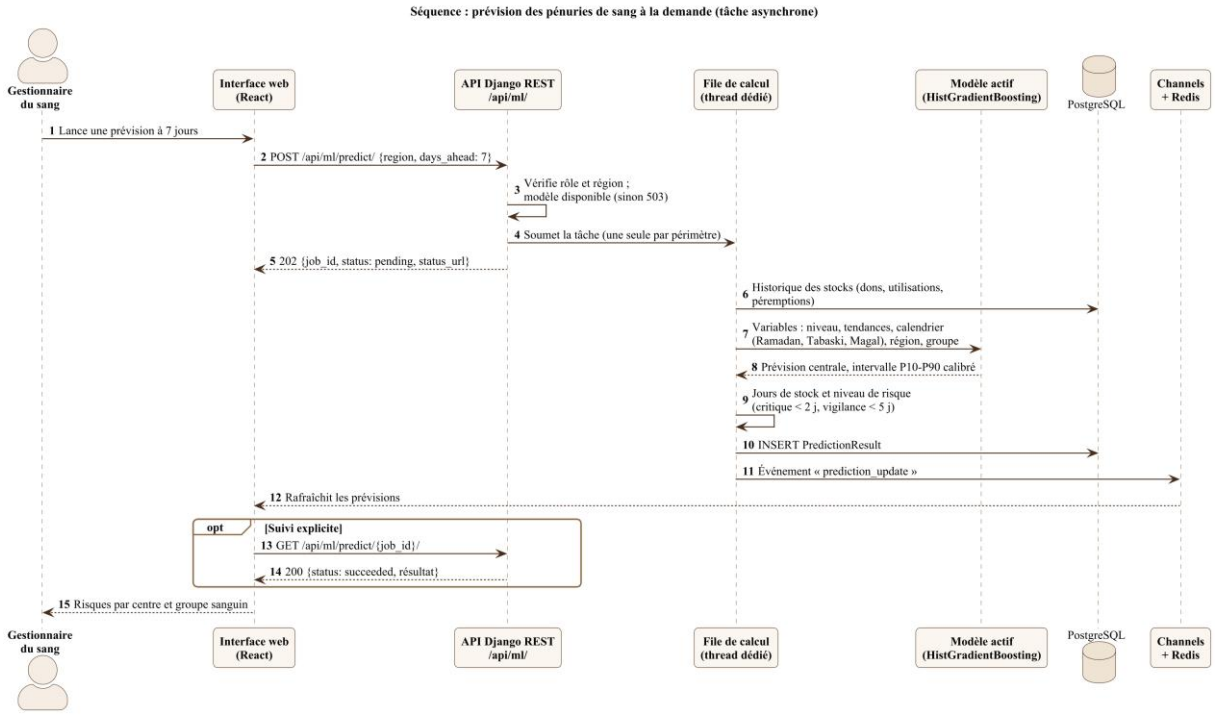


Figure 14 : Séquence de la prévision des pénuries à la demande

3.6 Diagramme de séquence global

Le diagramme global synthétise le fonctionnement commun à tous les modules : authentification et autorisation, validation, traitement métier transactionnel, réponse, puis diffusion temps réel et notifications.

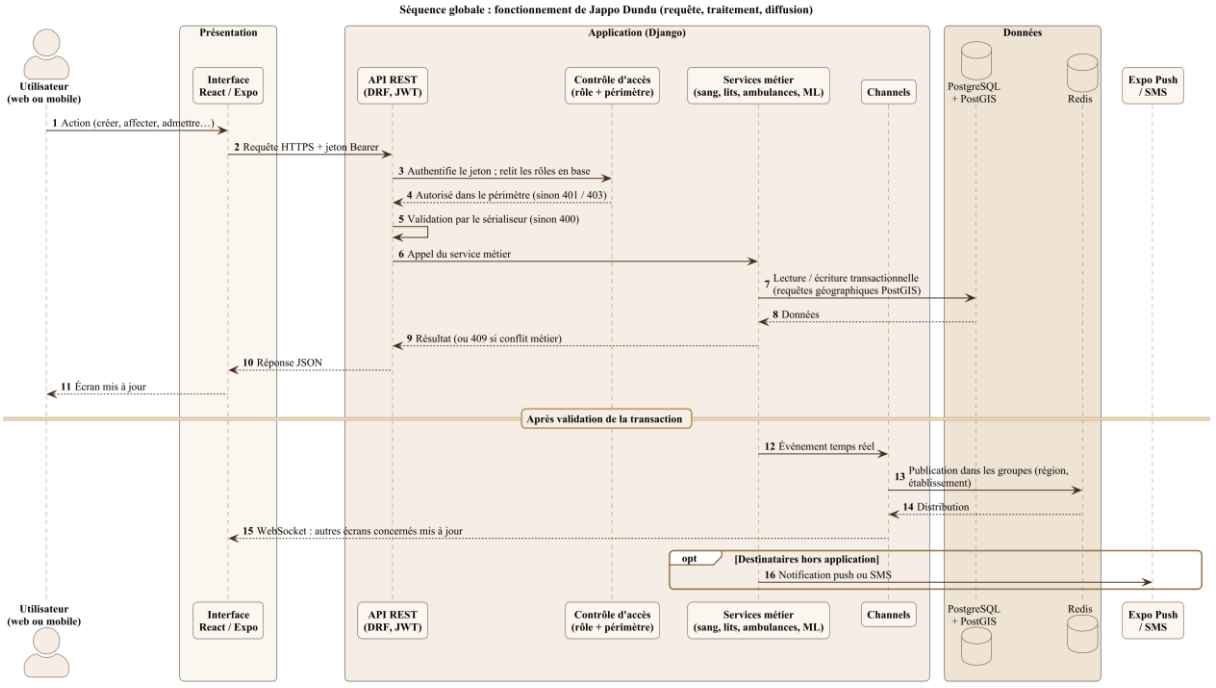


Figure 15 : Diagramme de séquence global de Jappo Dundu

3.7 Diagramme de classes

Le diagramme de classes est unique et complet : il reprend les **35 modèles** du système et le modèle `User` de Django, regroupés en paquets par module, avec leurs attributs métier, leurs méthodes et leurs associations. Les cardinalités correspondent aux contraintes réelles de la base : clé étrangère (0..* vers 1 ou 0..1), relation un-à-un ou plusieurs-à-plusieurs ; un losange plein indique une composition (la partie disparaît avec le tout).

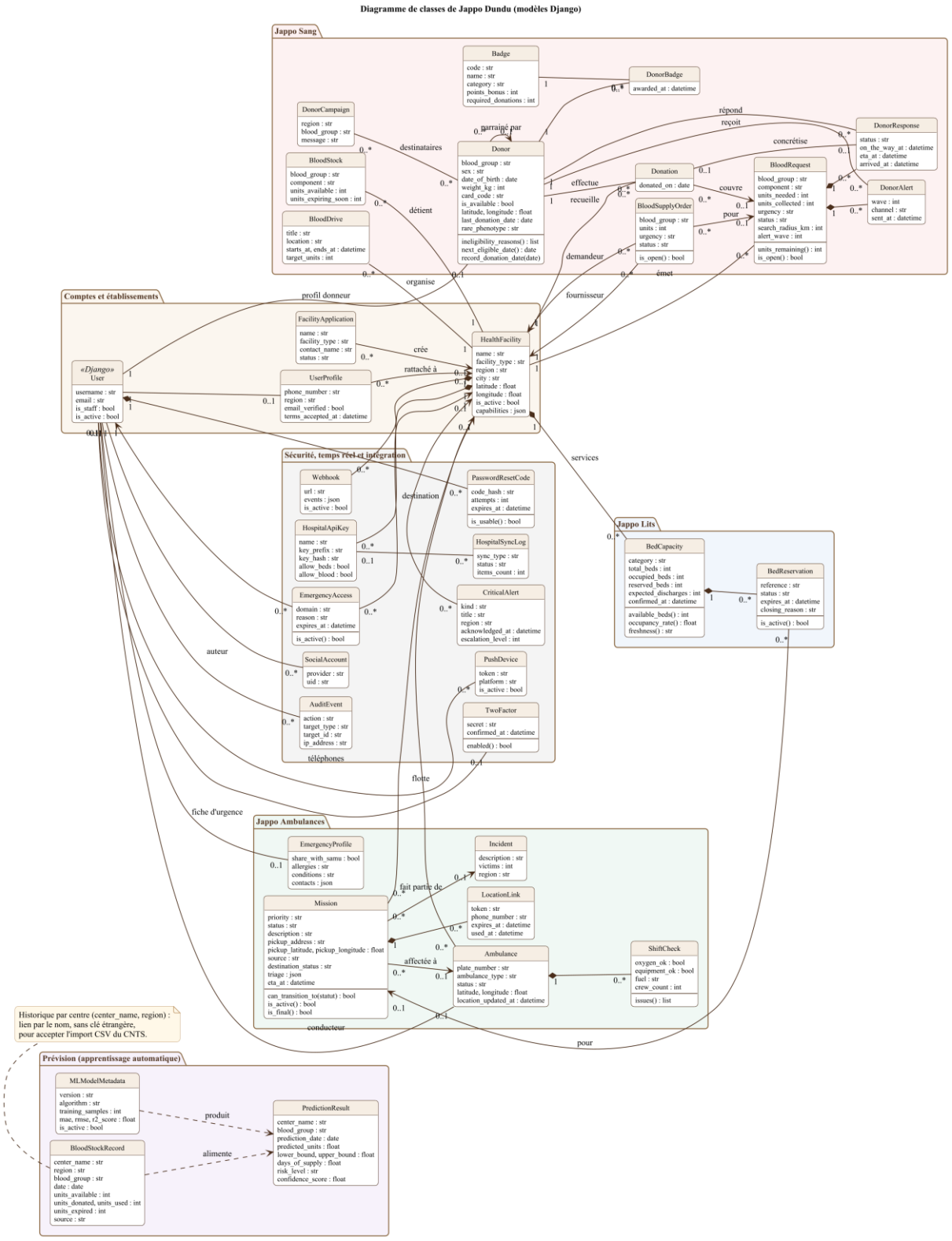


Figure 16 : Diagramme de classes de Jappo Dundu

L'entité pivot est **HealthFacility** (établissement de santé) : demandes de sang, stocks, services hospitaliers, ambulances, clés d'API et alertes s'y rattachent, ce qui permet de restreindre chaque donnée au périmètre de l'utilisateur. L'historique des stocks utilisé par la prévision

(`BloodStockRecord`) est rattaché aux centres par leur nom, afin d'accepter l'import d'un fichier CSV du CNTS sans correspondance préalable.

Tableau 10 : Dictionnaire des principales entités

Entité	Module	Rôle et attributs clés
HealthFacility	Comptes	Établissement (hôpital, CNTS, SAMU) : type, région, ville, coordonnées GPS, plateau technique.
UserProfile	Comptes	Complète le compte Django : téléphone, région, établissement de rattachement.
Donor	Sang	Profil donneur : groupe, sexe, date de naissance, poids, position, disponibilité, carte (JD-XXXX-XXXX) ; calcule son aptitude et sa prochaine date de don.
BloodRequest	Sang	Demande urgente : groupe, produit, poches demandées et collectées, urgence, statut, rayon et vague d'alertes.
DonorAlert / DonorResponse	Sang	Alerte envoyée à un donneur (vague, canal push ou SMS) et sa réponse (acceptée, déclinée, don effectué).
Donation / Badge	Sang	Don confirmé (date, centre) et distinctions de fidélisation attribuées au donneur.
BedCapacity	Lits	Capacité d'un service : lits installés, occupés, réservés, sorties prévues, date de confirmation ; calcule les lits libres et la fraîcheur des chiffres.
BedReservation	Lits	Lit bloqué pour un patient attendu, éventuellement lié à une mission, avec une échéance.
Ambulance	Ambulances	Véhicule : immatriculation, type, statut, conducteur, position et date de la position.
Mission	Ambulances	Intervention : priorité, lieu, statut (cycle de vie contrôlé), ambulance, destination, tri, heure d'arrivée estimée.
BloodStockRecord / PredictionResult	Prévision	Historique quotidien des stocks et prévisions (valeur, intervalle P10–P90, jours de stock, niveau de risque).
AuditEvent / TwoFactor	Sécurité	Journal des actions sensibles et secret de double authentification.

4. Choix technique

Cette section présente l'écosystème du projet : d'abord les outils de développement, puis les technologies et frameworks, par environnement technique. Chaque choix est justifié par son rôle exact dans Jappo Dundu ; les versions indiquées sont celles du projet.

4.1 Outils de développement



Git *Gestion de versions*

Git enregistre l'historique complet du code sous forme de commits. Chaque membre travaille sur sa partie, puis intègre ses modifications au tronc commun ; les conflits éventuels sont résolus avant fusion. Les messages de commit suivent une convention (`feat`, `fix`, `docs`, `chore`) qui rend l'historique lisible : c'est d'ailleurs à partir de cet historique que les itérations du projet ont été reconstituées.



GitHub *Hébergement du code et collaboration*

Le dépôt `jappo-dundu` est hébergé sur GitHub. Il centralise le backend, le frontend, l'application mobile, la configuration de déploiement et la documentation, et sert de point de synchronisation entre les quatre membres. GitHub conserve l'historique, permet de revenir à une version antérieure et héberge l'intégration continue du projet.



GitHub Actions *Intégration continue*

À chaque envoi de code, trois tâches s'exécutent en parallèle : le backend est testé sur une vraie base PostGIS (vérification des migrations, 590 tests), le frontend est typé, analysé, testé et construit, et l'application mobile est typée, testée et analysée. Une régression est ainsi détectée avant d'atteindre la branche principale.



Visual Studio Code *Environnement de développement*

Éditeur de code principal de l'équipe, utilisé pour Python, TypeScript et la configuration. Ses extensions (Python, ESLint, Tailwind CSS, Docker) apportent l'autocomplétion, le typage en temps réel et le débogage. Son terminal intégré sert à lancer les serveurs de développement, les tests et les commandes de gestion Django.



Docker et Docker Compose *Conteneurisation*

Docker exécute chaque composant dans un conteneur isolé et reproductible : PostgreSQL / PostGIS, Redis et l'API. En développement, `docker compose up` démarre la base et Redis en une commande, identiques pour toute l'équipe. En production, la même description lance les quatre services (base, Redis, API, proxy Caddy) sur un serveur Linux.



PlantUML et Graphviz *Modélisation UML*

PlantUML décrit les diagrammes UML sous forme de texte : cas d'utilisation, séquences, classes, architecture et déploiement. Ces fichiers sources sont versionnés avec le code (`docs/uml`), relus comme lui et régénérés à l'identique. Graphviz calcule la mise en page des diagrammes ; l'export se fait en image haute résolution, sans capture d'écran d'éditeur.



Swagger / OpenAPI *Documentation de l'API*

La spécification OpenAPI 3 de l'API est générée automatiquement par drf-spectacular à partir des vues et des sérialiseurs, puis présentée dans l'interface Swagger UI. Elle constitue le contrat formel entre le backend et les applications clientes : routes, méthodes, paramètres, formats de requête et de réponse, codes d'erreur et authentification, testables directement depuis le navigateur.



Playwright *Tests de bout en bout*

Playwright pilote un vrai navigateur pour rejouer des parcours complets sur la pile réelle (API Django, PostGIS, Redis, interface) : connexion et temps réel, affectation d'une ambulance, propagation d'une admission vers un autre poste. L'outil a aussi servi à produire les captures d'écran haute résolution de ce rapport, à partir de parcours réellement exécutés.

4.2 Technologies et frameworks

Frontend web



React 19 *Bibliothèque d'interface*

React construit l'interface web à partir de composants réutilisables : tuiles d'indicateurs, panneaux, graphiques, cartes, formulaires. Chaque écran (tableaux de bord par rôle, banque de sang, lits, flotte SAMU, prévisions, structures) assemble ces composants. Le rendu se met à jour automatiquement lorsqu'une donnée change, ce qui s'accorde naturellement avec le temps réel.



TypeScript 6 *Langage typé*

TypeScript ajoute un typage statique à JavaScript. Les types de chaque ressource de l'API (demande de sang, capacité, mission) sont décrits une fois et vérifiés partout : une faute de frappe sur un champ ou un appel incorrect est signalé avant même l'exécution. Le même langage est utilisé pour le web et pour le mobile.



HTML5 *Structure des pages*

HTML5 structure les pages avec des balises sémantiques (`header`, `nav`, `main`, `section`, `table`) et des attributs d'accessibilité (`aria-label`, rôles). Cette structure permet aux lecteurs d'écran de restituer les tableaux de bord et facilite les tests automatisés, qui retrouvent les éléments par leur rôle et leur libellé plutôt que par leur position.



CSS3 *Mise en forme*

CSS3 assure la mise en page en grilles et boîtes flexibles, les thèmes clair et sombre par variables de couleur, et les animations d'apparition, désactivées lorsque l'utilisateur a demandé à réduire les mouvements. Les couleurs portent un sens constant dans toute l'application : rouge pour le critique, ambre pour la vigilance, bleu nuit pour le nominal.



Tailwind CSS 4 *Framework CSS*

Tailwind fournit des classes utilitaires composées directement dans les composants. Il garantit des espacements, des tailles et des couleurs cohérents entre les écrans, sans feuilles de style divergentes. Les couleurs du thème sont définies une seule fois sous forme de jetons, ce qui rend le passage au mode sombre automatique.



Vite 8 *Outil de construction*

Vite sert l'application en développement avec un rechargement instantané à chaque modification et construit la version de production optimisée. En développement, il relaie aussi les appels `/api` et `/ws` vers le backend Django, si bien que l'interface et l'API partagent la même origine, comme en production.



Leaflet *Cartographie*

Leaflet affiche les cartes interactives : établissements, banques de sang, ambulances et lieux d'intervention, avec des calques que l'on peut masquer. Les positions des ambulances reçues par WebSocket sont appliquées directement sur la carte, sans nouvelle requête, ce qui permet de voir le véhicule progresser pendant la mission.



TanStack Query *Gestion des données serveur*

TanStack Query gère le chargement, la mise en cache et le rafraîchissement des données de l'API. Lorsqu'un événement temps réel arrive, seules les requêtes concernées sont invalidées et rechargées, regroupées par fenêtre d'une seconde : l'écran reste à jour sans surcharger le serveur.

Application mobile



Expo et React Native *Développement mobile multiplateforme*

L'application des donneurs et des ambulanciers est écrite une fois en React Native et produite pour Android et iOS grâce à Expo. Expo fournit l'accès natif au GPS, aux notifications push (relayées vers Firebase Cloud Messaging et Apple Push Notification service) et au stockage sécurisé du jeton de session ; Expo Router organise la navigation par onglets.

Backend



Python 3.14 *Langage serveur*

Python est le langage du backend et du module de prévision. Sa lisibilité facilite la relecture du code métier (règles d'aptitude au don, cycle de vie d'une mission, vagues d'alertes) et il donne accès à l'écosystème scientifique utilisé pour l'apprentissage automatique. Les tests du serveur sont écrits avec le framework de test de Django.



Django 6 *Framework web*

Django fournit l'ORM (correspondance entre classes Python et tables), les migrations du schéma, l'authentification, l'interface d'administration et les protections de sécurité standard. Les applications du projet (`users`, `sang`, `lits`, `ambulances`, `ml`, `security`, `realtime`, `sih`…) découpent le code par domaine. Django Channels et le serveur ASGI Daphne y ajoutent les WebSockets.



Django REST Framework *Framework d'API*

DRF expose les ressources en API REST : les sérialiseurs valident les entrées et formatent les sorties JSON, les ensembles de vues regroupent les opérations d'une ressource, et les classes de permission appliquent les rôles. Il gère l'authentification JWT (SimpleJWT), la pagination et la limitation du débit, par exemple 10 tentatives de connexion par minute.

Données et temps réel



PostgreSQL 16 et PostGIS 3.5 *Base de données relationnelle et géographique*

PostgreSQL stocke l'ensemble des données avec leurs contraintes d'intégrité (clés étrangères, contraintes de valeurs). L'extension PostGIS calcule les distances sur la sphère terrestre et, grâce à un index spatial, trouve en quelques millisecondes les donneurs, lits et ambulances les plus proches d'un point, triés par distance.



Redis 7 *Diffusion temps réel*

Redis sert de couche de messagerie à Django Channels : lorsqu'un service publie un événement (lit libéré, nouvelle demande, position d'ambulance), Redis le distribue à tous les processus serveurs, qui le poussent aux WebSockets abonnés au groupe concerné (région, établissement). Les tickets de connexion temps réel y sont aussi conservés.

Intelligence artificielle



scikit-learn 1.9 *Apprentissage automatique*

scikit-learn fournit les trois modèles `HistGradientBoostingRegressor` de la prévision : la valeur centrale et les quantiles 10 % et 90 %. Le projet y ajoute une calibration conformelle de l'intervalle et une évaluation chronologique stricte, comparée à une prévision de référence. Le modèle entraîné est sauvegardé avec joblib et versionné dans un registre.



pandas et NumPy *Traitement de données*

pandas et NumPy préparent les données de la prévision : séries quotidiennes de stocks par centre et par groupe, moyennes glissantes sur 7 et 30 jours, variables calendaires (jour de la semaine, part de Ramadan, de la Tabaski et du Grand Magal dans la fenêtre) et normalisation de la cible.

Tests



Vitest et Jest *Tests unitaires des interfaces*

Vitest exécute les 104 tests du frontend web (composants, pages, règles d'accès, règles métier affichées) et Jest les 73 tests de l'application mobile (alertes, carte donneur, mission, validation des formulaires). Ces tests simulent l'API et vérifient ce que voit réellement l'utilisateur.

Ces choix forment un ensemble cohérent : un seul langage côté serveur (Python) et côté clients (TypeScript), des briques open source éprouvées, et une base géographique qui porte la fonctionnalité centrale du projet, la recherche de la ressource la plus proche.

CHAPITRE 3 : REALISATION, TESTS ET DEPLOIEMENT

Ce chapitre présente les fonctionnalités réalisées, module par module, à travers des parcours utilisateurs réellement exécutés sur l'application. Chaque capture d'écran montre l'état obtenu après l'action décrite ; les parcours suivent le plan de tests fonctionnels présenté en section 2. Les données affichées sont des données de démonstration (établissements de Dakar et Thiès, comptes fictifs) ; l'historique des stocks utilisé par la prévision est simulé, faute d'historique réel du CNTS.

1. Réalisation technique

1.1 Authentification et contrôle d'accès

L'**objectif** de ce module est double : n'ouvrir l'application qu'aux personnes habilitées, et présenter à chacune uniquement ce qui relève de son rôle. Le visiteur arrive d'abord sur la page d'accueil publique, qui présente la plateforme et oriente vers l'espace professionnel ou le don de sang.



Figure 17 : Page d'accueil publique de Jappo Dundu

Étape 1 — Connexion. L'utilisateur saisit son identifiant (nom d'utilisateur, e-mail ou numéro de téléphone) et son mot de passe. En développement, des comptes de démonstration permettent de se connecter en un clic ; ils sont retirés du code livré en production.

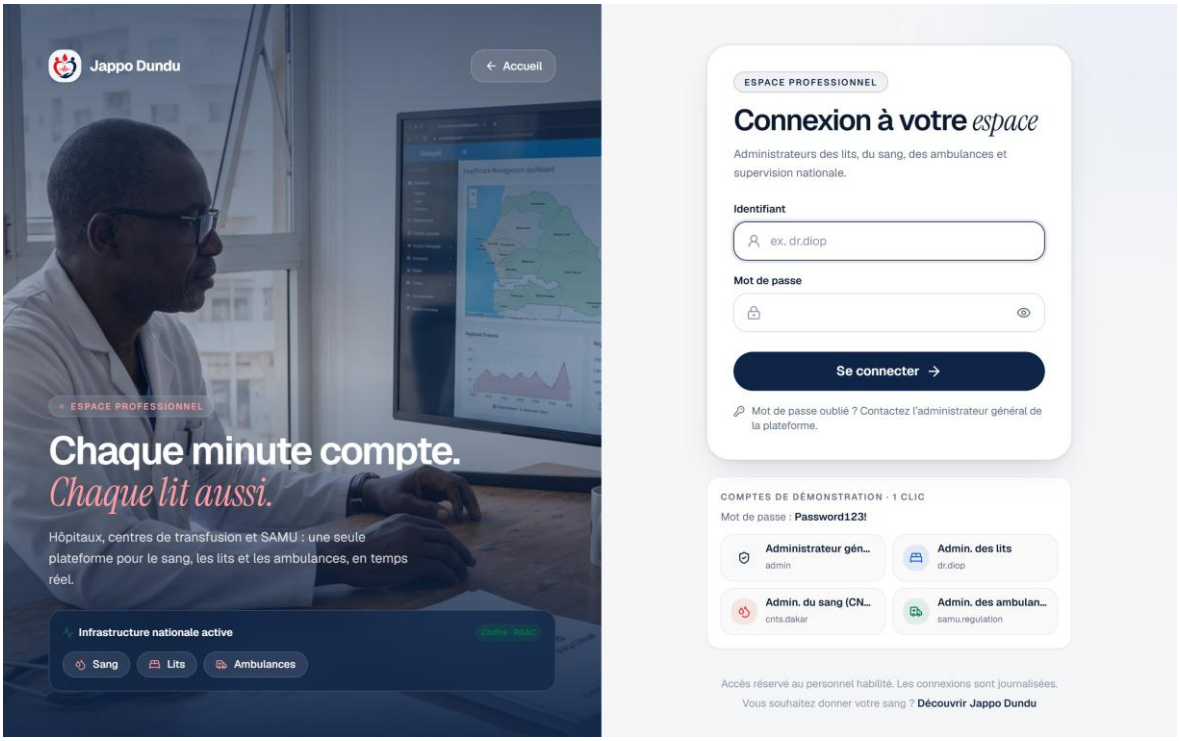


Figure 18 : Écran de connexion de l'espace professionnel

Étape 2 — Identifiants invalides (CT02). Avec un mot de passe erroné, l'accès est refusé et un message clair s'affiche, sans préciser si c'est l'identifiant ou le mot de passe qui est faux, pour ne pas renseigner un attaquant. Les tentatives sont limitées à dix par minute.

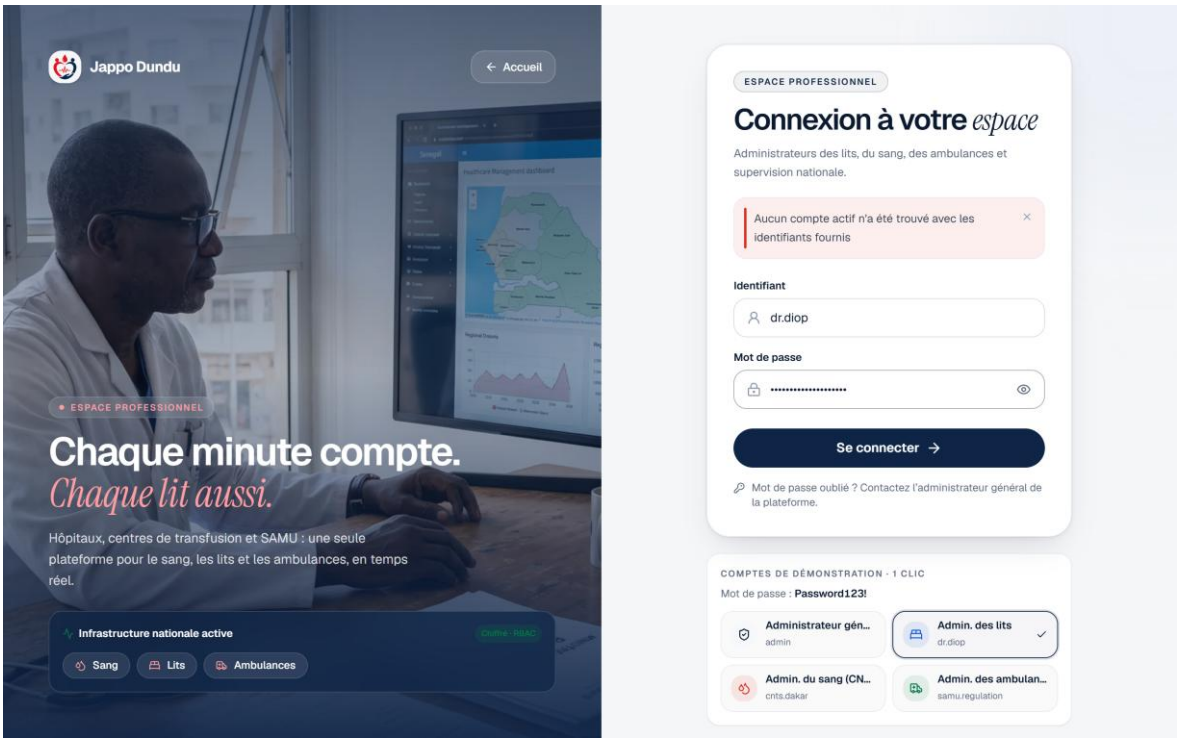


Figure 19 : Refus d'une connexion avec un mot de passe erroné

Étape 3 — Ouverture de l'espace du rôle (CT01). Avec des identifiants valides, l'utilisateur arrive directement sur le tableau de bord de son rôle ; la navigation latérale ne propose que les modules qui le concernent (ici, un administrateur des lits ne voit que « Lits »).

Étape 4 — Accès refusé (CT03). Si ce même utilisateur saisit à la main l'adresse d'un module qui ne relève pas de son rôle, l'interface affiche un écran explicite ; surtout, le serveur refuse de toute façon la requête (erreur 403), car le contrôle d'accès est appliqué côté API.

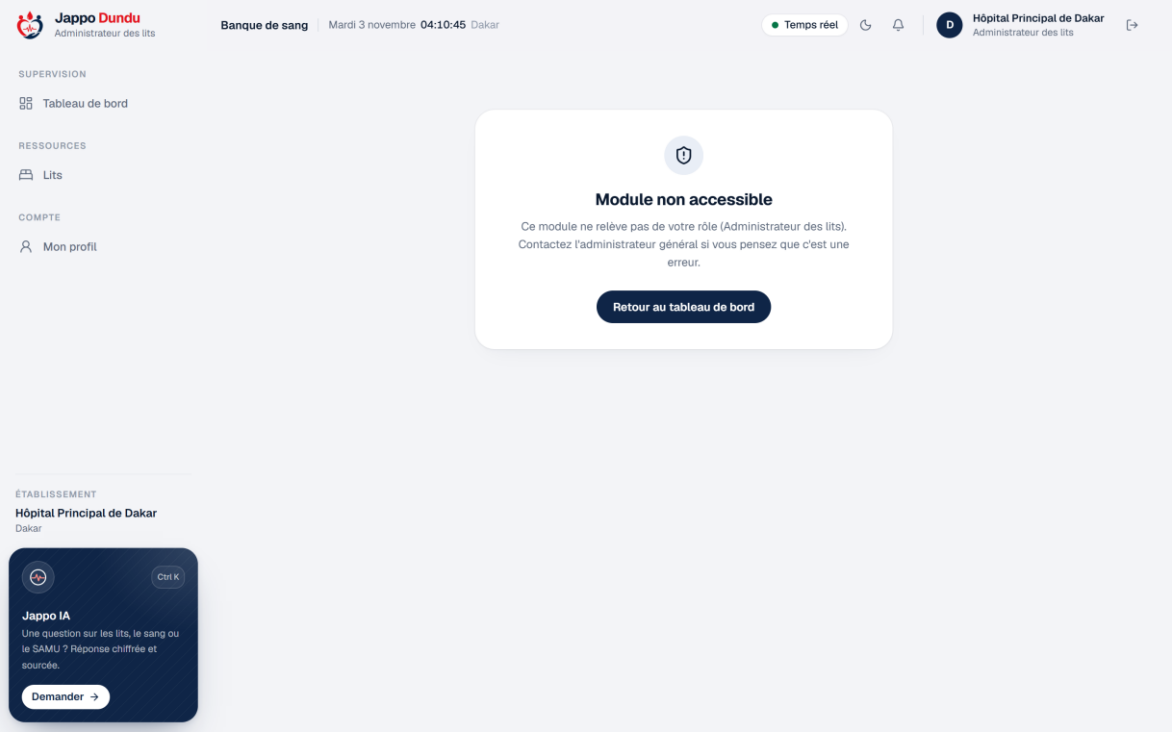


Figure 20 : Accès refusé à un module hors du rôle de l'utilisateur

1.2 Supervision nationale (administrateur général)

Le tableau de bord de l'administrateur général donne une vision consolidée du réseau en temps réel : lits disponibles et taux d'occupation, demandes de sang ouvertes, ambulances disponibles et risques de pénurie à sept jours. La liste « À traiter maintenant » classe toutes les situations du plus grave au moins grave, quel que soit le module ; chaque ligne mène à l'écran où agir.

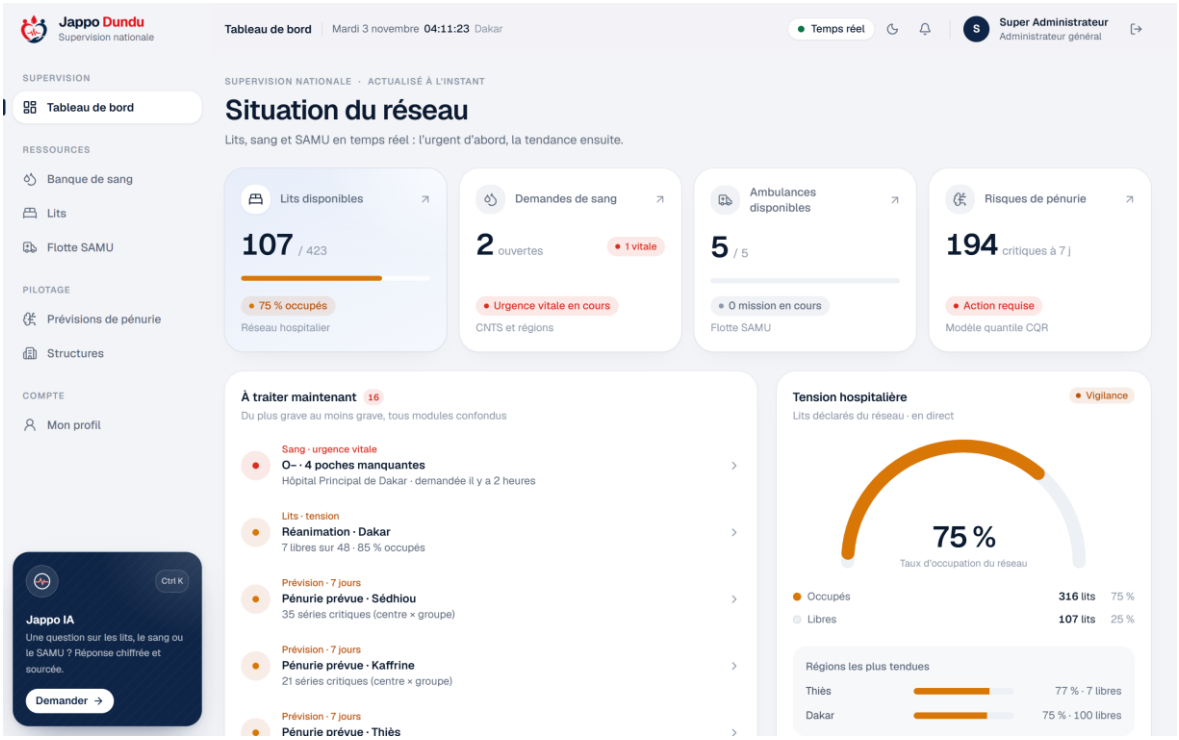


Figure 21 : Tableau de bord de supervision nationale

Plus bas, la page présente la tension hospitalière (jauge d'occupation et régions les plus tendues), le risque de pénurie par région, la carte opérationnelle avec ses calques (hôpitaux, banques de sang, ambulances, missions) et l'occupation par spécialité.

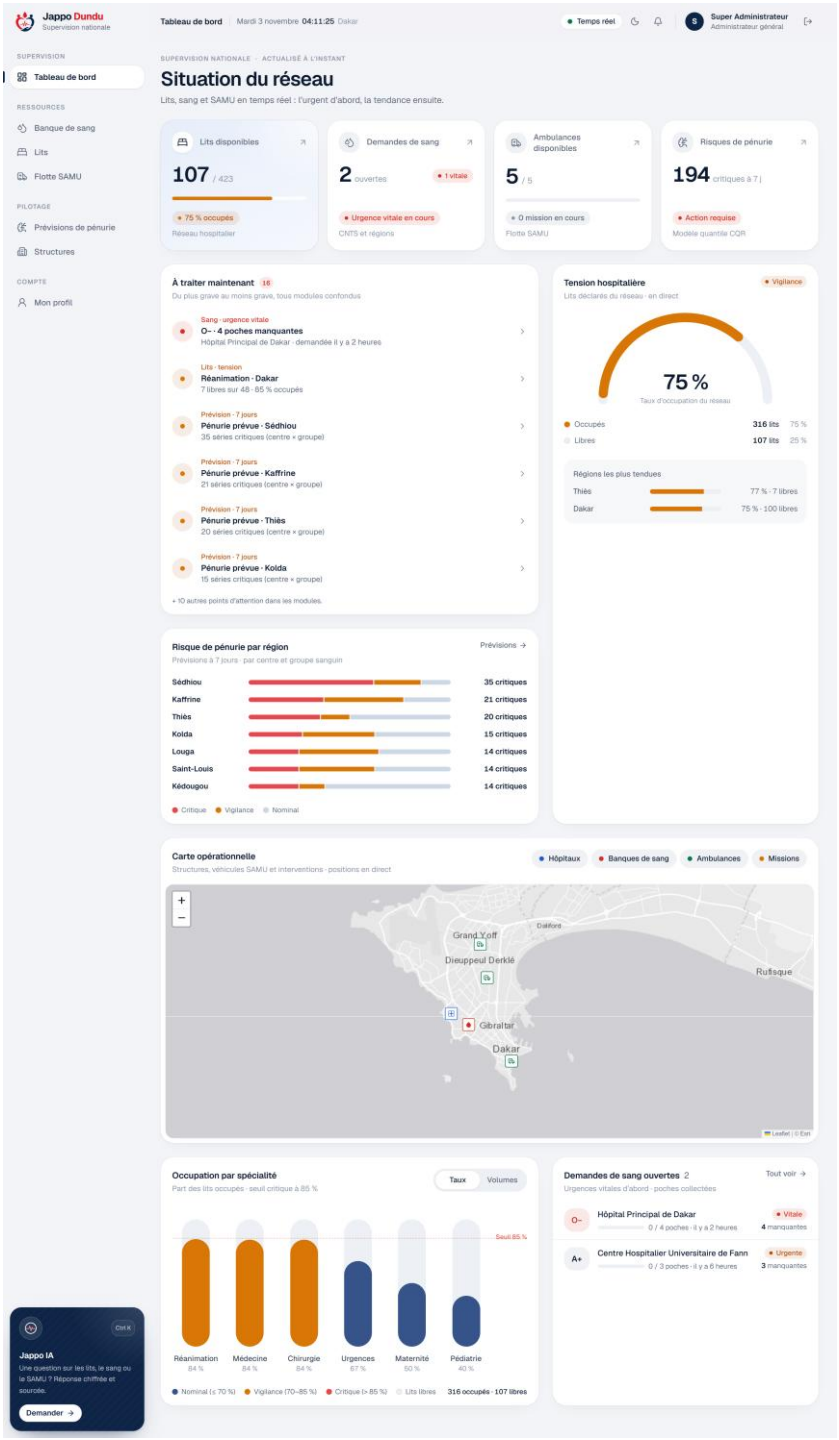


Figure 22 : Vue complète du tableau de bord national

Parcours : ajouter un établissement et créer le compte de son responsable

Étape 1. Depuis « Structures », l'administrateur ouvre « Nouvel établissement » et renseigne le nom, le type, la région, la ville, le téléphone, l'adresse et les coordonnées GPS, qui serviront aux recherches de proximité.

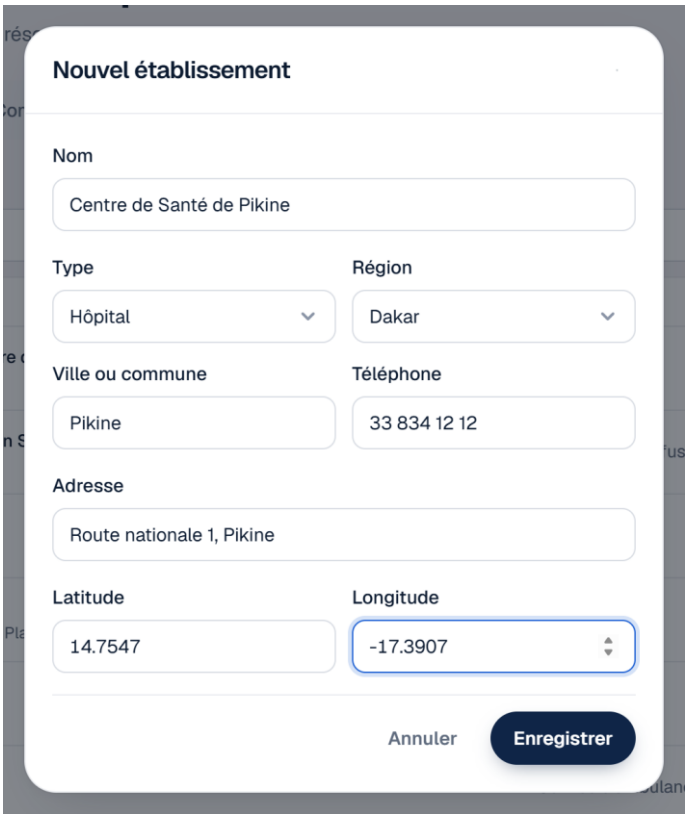
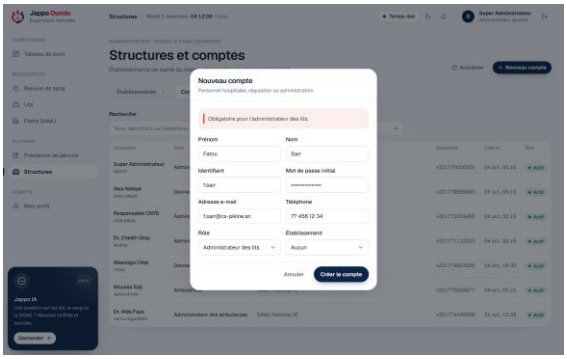
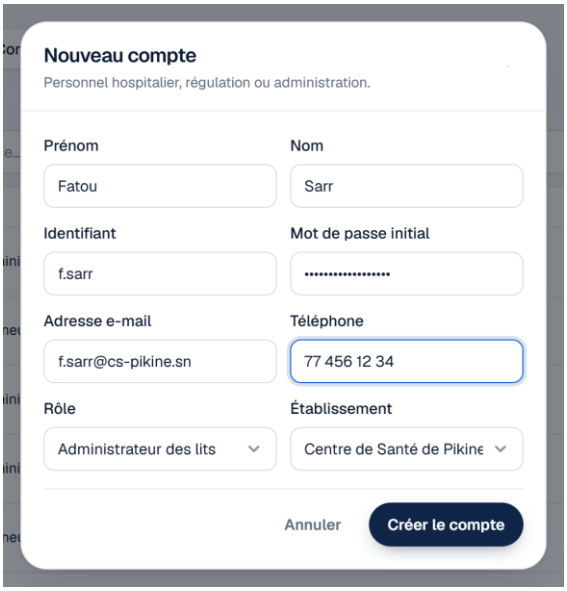


Figure 23 : Formulaire de création d'un établissement

Étape 2. Dans l'onglet « Comptes », il crée le compte de la responsable des lits. Le contrôle de saisie impose de rattacher un administrateur des lits à un établissement : sans ce rattachement, la création est refusée avec un message explicite.



a) Rattachement manquant : refus



b) Formulaire complet

Figure 24 : Création d'un compte : contrôle de saisie puis formulaire valide

Étape 3. Le compte apparaît aussitôt dans la liste avec son rôle et son établissement de rattachement. À sa première connexion, la nouvelle responsable ne verra que les lits du Centre de Santé de Pikine.

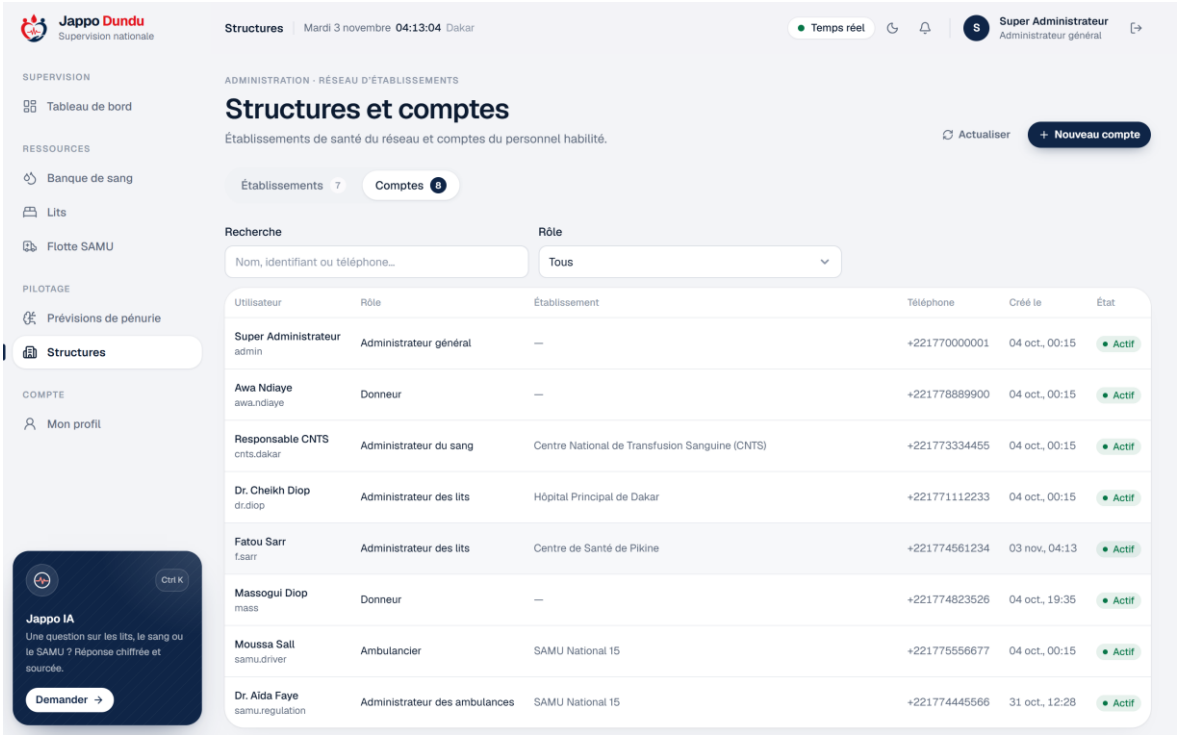


Figure 25 : Compte créé et rattaché à son établissement

Prévisions de pénurie et assistant Jappo IA

La page des prévisions présente les performances du modèle actif, puis le risque prévu pour chaque centre et chaque groupe sanguin, avec l'intervalle de prévision. Un bandeau rappelle que le modèle est entraîné sur des données simulées tant que l'historique réel du CNTS n'est pas importé.

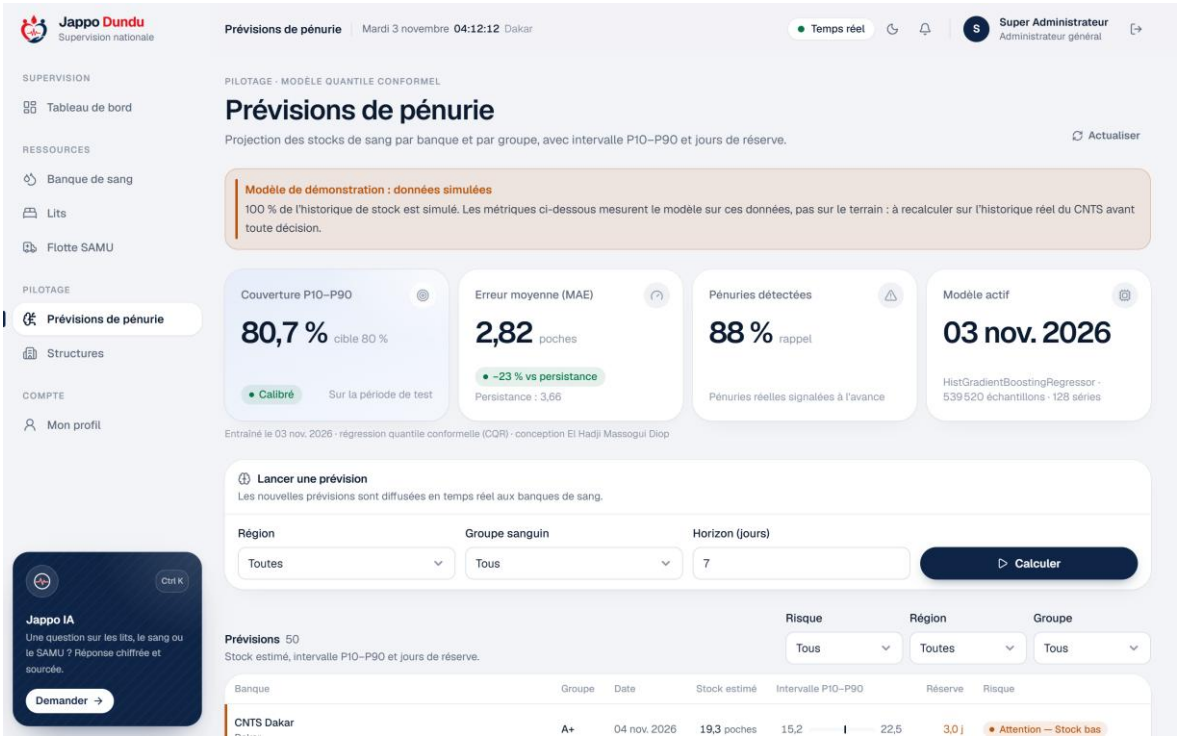


Figure 26 : Prévisions de pénurie de sang à 7 jours

L'assistant **Jappo IA** s'ouvre depuis la barre latérale ou par le raccourci Ctrl K. Il répond en langage naturel à une question comme « Où trouver un lit de réanimation à Dakar ? » par une synthèse chiffrée, un tableau des services concernés et la source des données avec son horodatage. Il ne s'agit pas d'un modèle de langage génératif : l'assistant interprète la question puis interroge l'API dans les droits de l'utilisateur, ce qui garantit des réponses exactes et vérifiables.

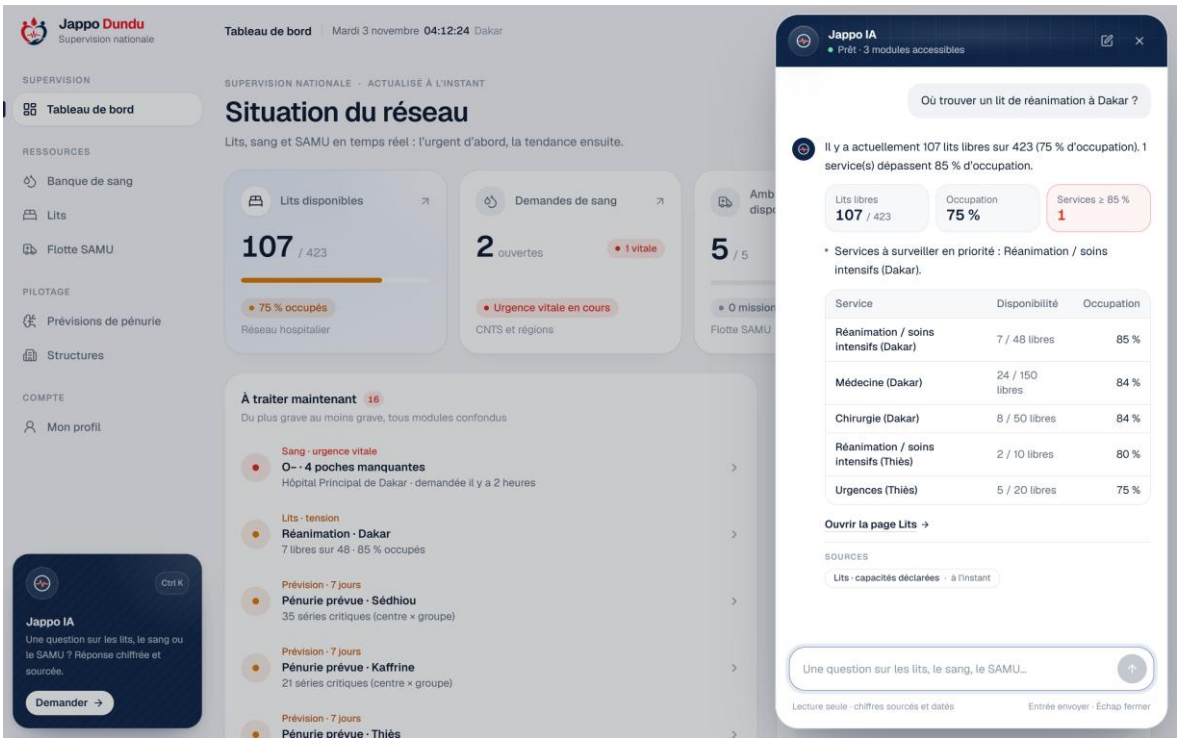


Figure 27 : Réponse de l'assistant Jappo IA, chiffrée et sourcée

1.3 Module Jappo Sang

Ce module couvre le parcours le plus critique du projet : de la demande urgente d'un établissement jusqu'à la confirmation du don, en passant par le téléphone du donneur. Le parcours ci-dessous a été exécuté de bout en bout avec le compte du CNTS de Dakar et celui d'une donneuse, Awa Ndiaye (groupe O+).

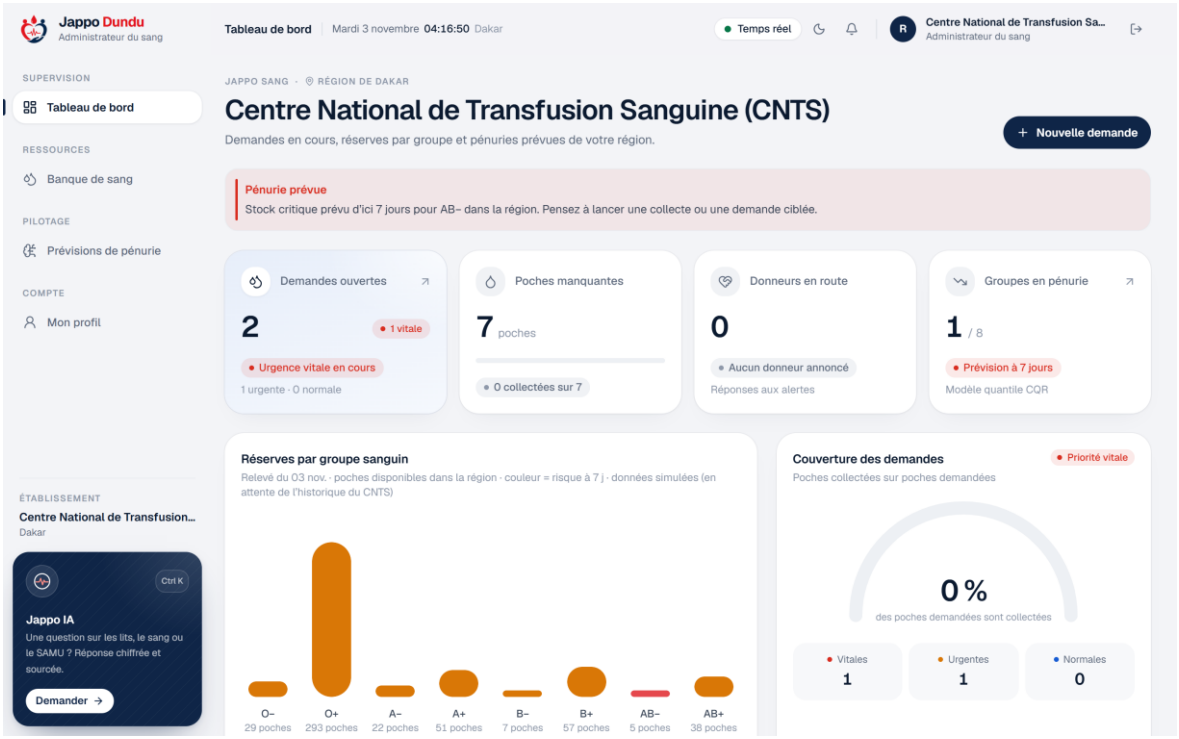


Figure 28 : Tableau de bord du gestionnaire du sang (CNTS)

Étape 1 — Créer la demande (CT04). Depuis la banque de sang, le gestionnaire ouvre « Nouvelle demande » et indique le groupe O+, deux poches, l'urgence critique, un rayon de recherche de 25 km et les précisions cliniques. Il clique sur « Diffuser la demande ».

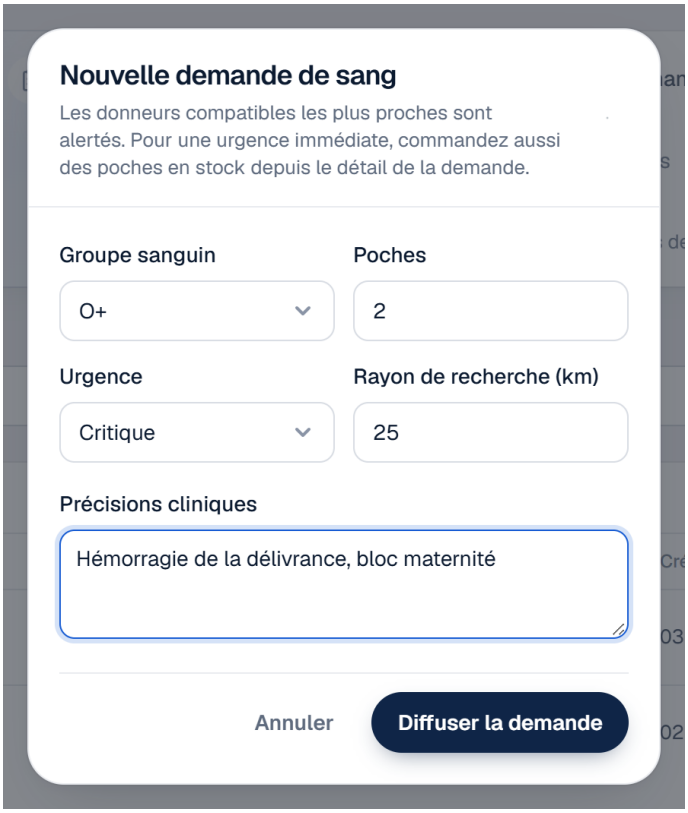


Figure 29 : Formulaire de demande de sang urgente

Étape 2 — Suivi de la demande. La demande apparaît en tête de liste avec la mention « Critique (pronostic vital engagé) ». Son panneau de détail indique les poches collectées, le rayon de recherche et la vague d'alerte envoyée ; le bouton « Alerter d'autres donneurs » permet de relancer sans attendre la vague automatique.

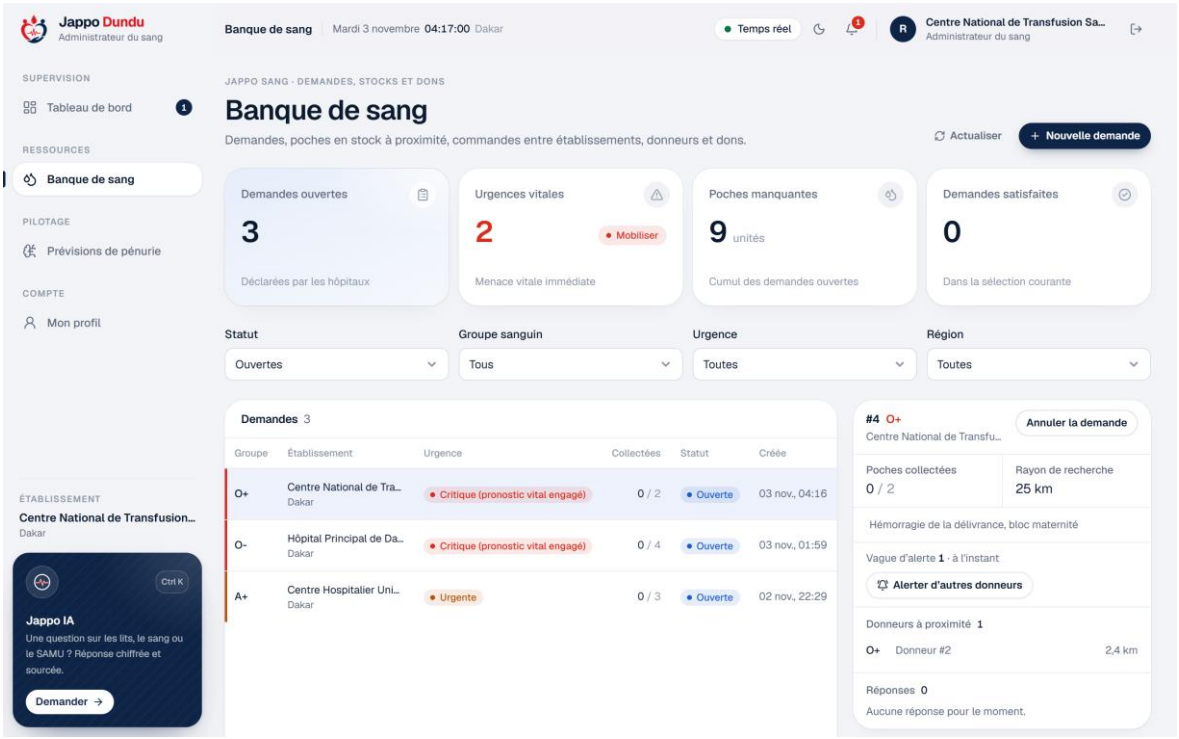
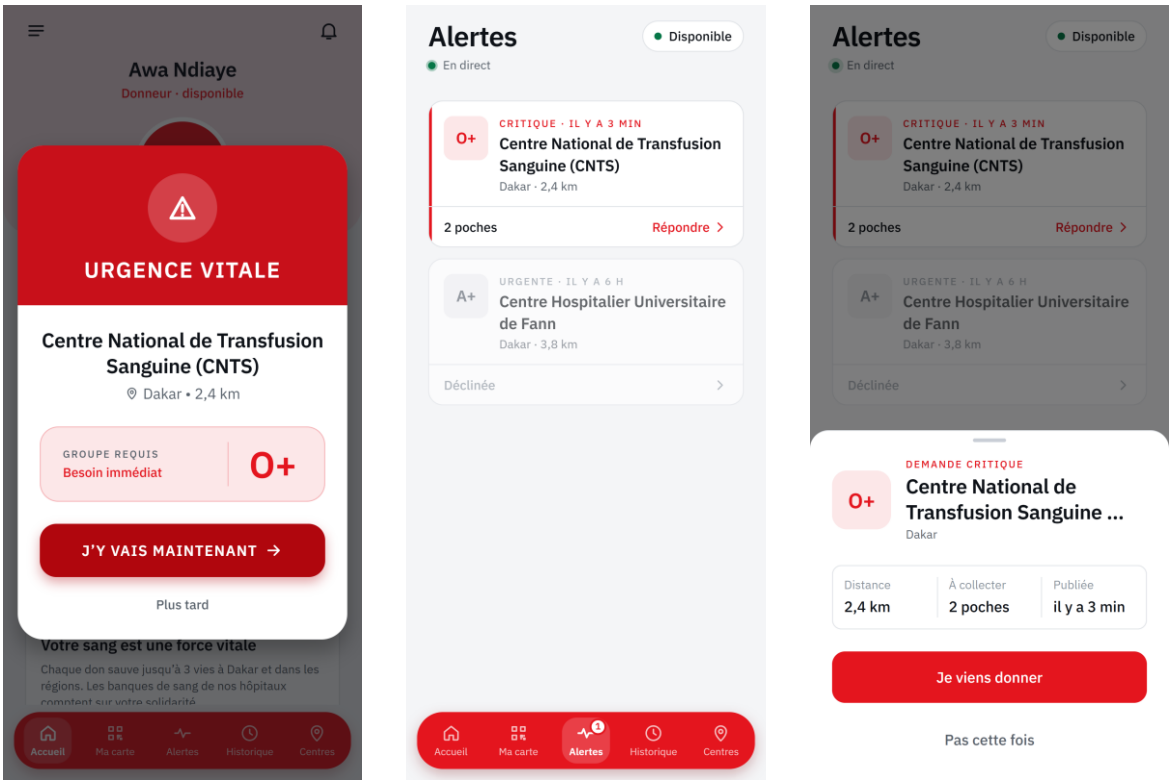


Figure 30 : Demande publiée et première vague d'alertes envoyée

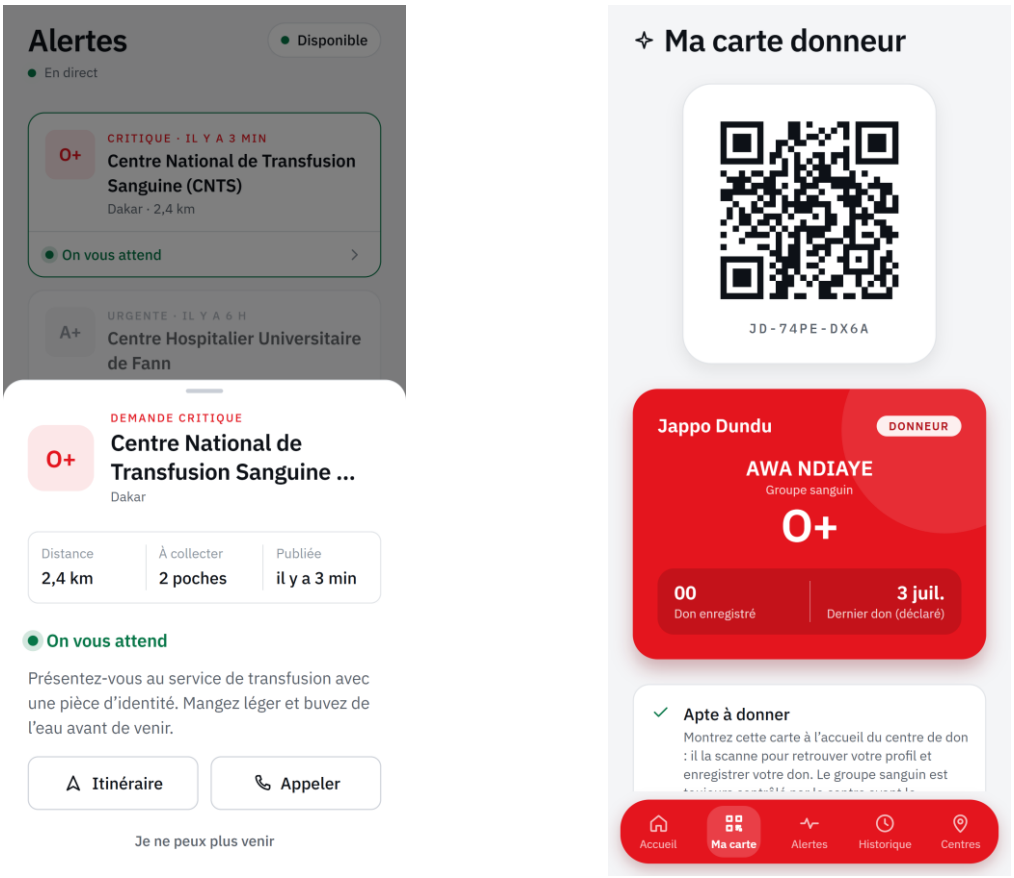
Étape 3 — Réception par la donneuse (CT05). Sur son téléphone, Awa Ndiaye voit l'alerte en tête de ses demandes proches : groupe, établissement, distance (2,4 km) et ancienneté. Seules les demandes **compatibles** avec son groupe et situées dans le rayon lui sont présentées ; sa position n'est jamais communiquée à l'hôpital.



a) Alerte en plein écran *b) Demandes proches* *c) Détail de la demande*

Figure 31 : Réception d'une demande urgente dans l'application mobile

Étape 4 — Réponse « Je viens donner » (CT06). En un geste, la donneuse s'engage. Le serveur vérifie la compatibilité ABO-Rh et son aptitude au don (âge de 18 à 60 ans, poids d'au moins 50 kg, 90 jours entre deux dons pour un homme et 120 pour une femme) avant d'enregistrer la réponse. L'application lui indique alors comment se préparer et lui propose l'itinéraire.



a) Engagement confirmé *b) Carte donneur à QR code*

Figure 32 : Engagement de la donneuse et carte donneur présentée au centre

Étape 5 — L'établissement voit venir la donneuse. Côté CNTS, sans rechargement de la page, la réponse apparaît dans le panneau de la demande : « Awa Ndiaye — Accepte de donner », avec son téléphone et le bouton « Confirmer le don ».



Figure 33 : Réponse de la donneuse reçue en temps réel par le CNTS

Étape 6 — Confirmation du don (CT07). Au centre, le gestionnaire retrouve la donneuse en scannant sa carte (code `JD-74PE-DX6A`) ou par son téléphone, puis confirme le don. Les poches collectées passent à 1 / 2, la réponse devient « Don effectué », des points et badges sont attribués et la donneuse reçoit une notification « Votre don a servi ».

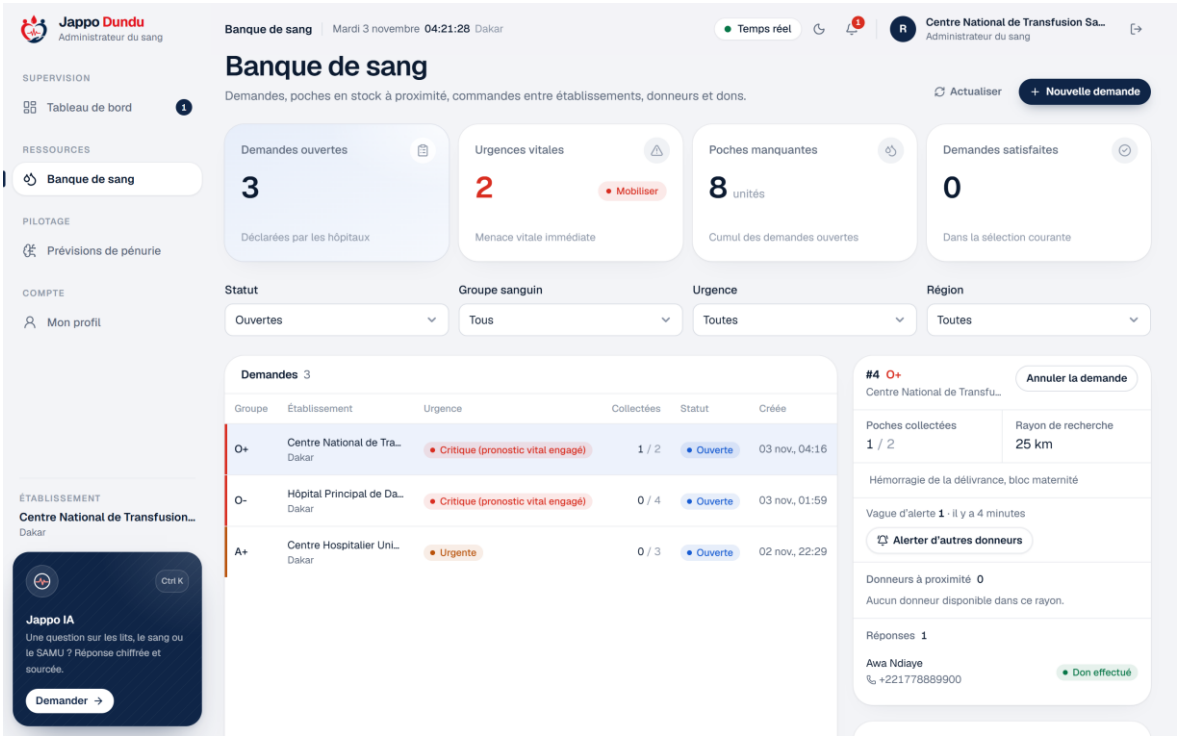
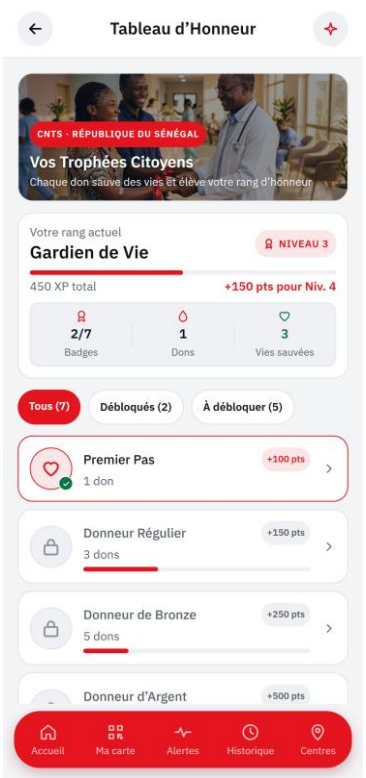


Figure 34 : Don confirmé : une poche collectée sur deux

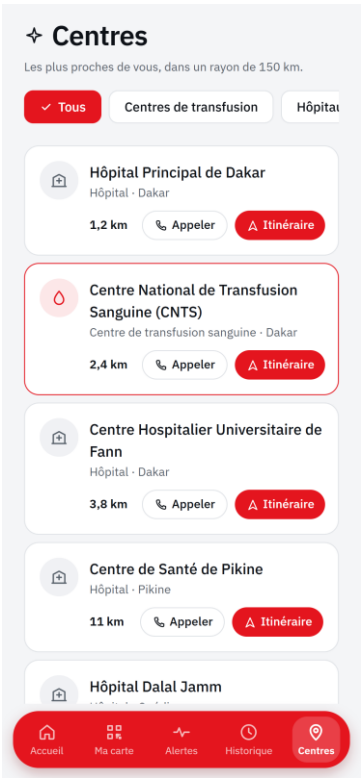
Étape 7 — Historique du donneur. Dans l'application, l'historique compte désormais un don validé et calcule la prochaine date de don possible (le 3 mars 2027, soit 120 jours plus tard pour une donneuse). Les badges valorisent la régularité des dons.



a) Historique et prochain don



b) Badges



c) Centres de don proches

Figure 35 : Suivi du donneur après le don

1.4 Module Jappo Lits

Le module Jappo Lits donne à chaque hôpital la maîtrise de ses capacités et au réseau une visibilité immédiate sur les lits libres. Le parcours est exécuté avec le compte de l'administrateur des lits de l'Hôpital Principal de Dakar.



Figure 36 : Tableau de bord de l'administrateur des lits

Le tableau de bord compare l'occupation de chaque service au seuil critique de 85 % et au taux de la région (trait blanc). La page « Mes lits » présente chaque service sous forme de carte : lits occupés, réservés et libres, et trois actions directes.

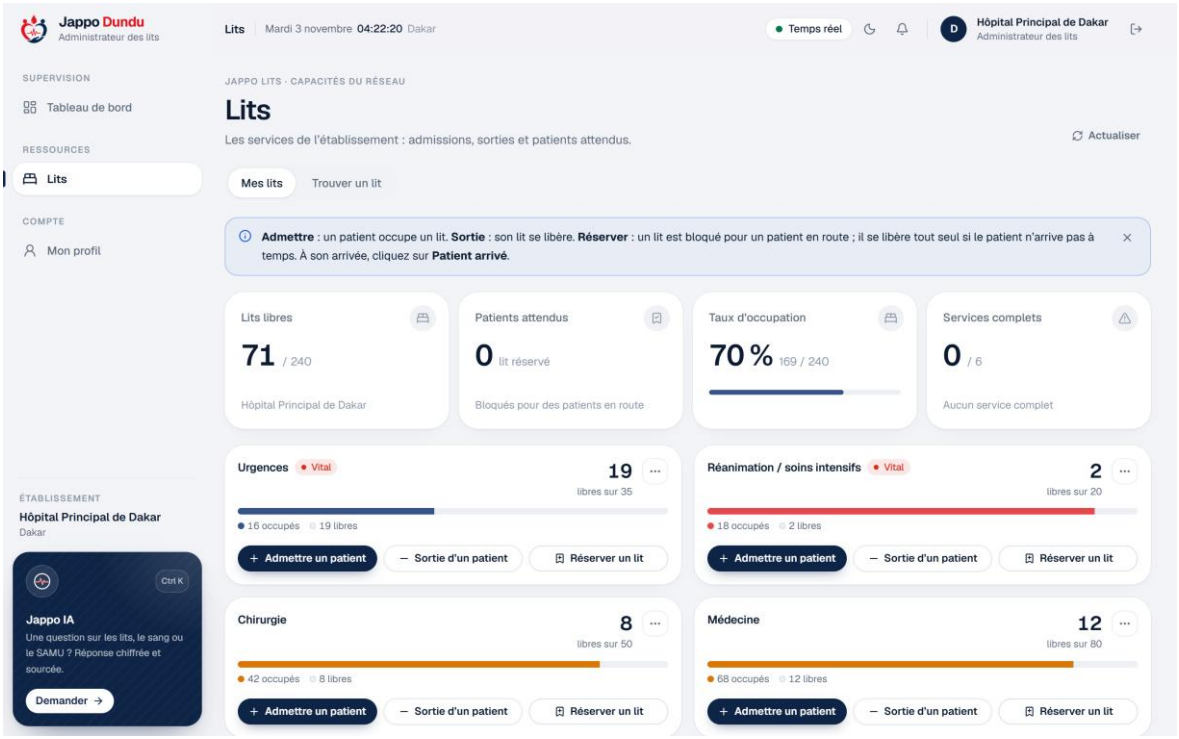


Figure 37 : Capacités des services de l'établissement

Étape 1 — Réserver un lit (CT09). Pour un patient attendu, le gestionnaire réserve un lit aux urgences : il peut le lier à une mission SAMU, indique une référence (jamais le nom du patient) et une durée de blocage. Le lit est libéré automatiquement à l'expiration du délai.

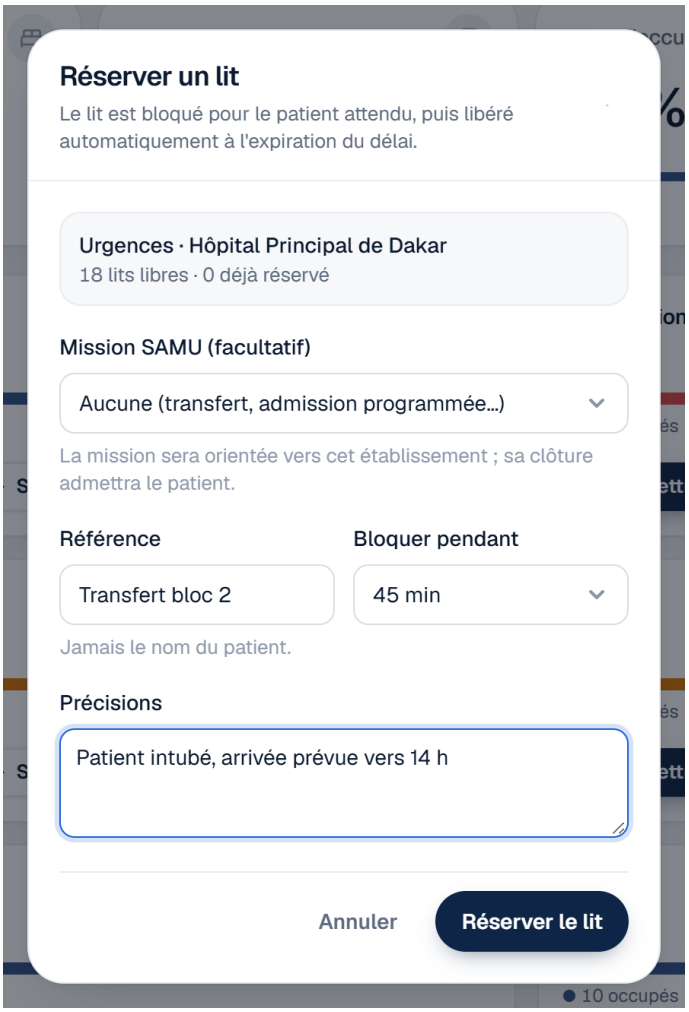
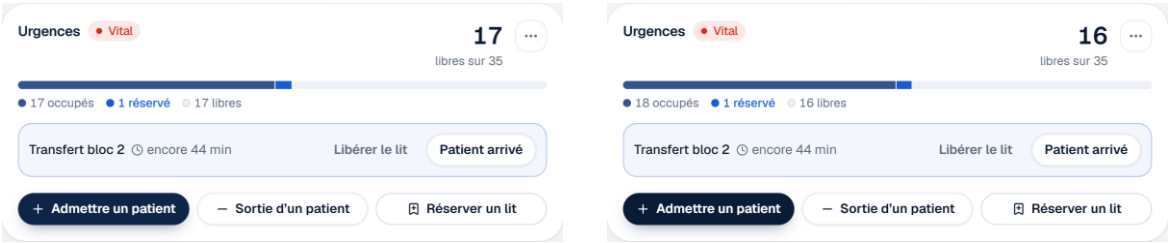


Figure 38 : Réservation d'un lit pour un patient attendu

Étape 2 — Admettre un patient (CT08). Un clic sur « Admettre un patient » met à jour le service : aux urgences, les lits libres passent de 17 à 16 et les lits occupés de 17 à 18, la réservation restant affichée avec son délai. La mise à jour est poussée en temps réel sur les écrans du réseau, ce que vérifie un test de bout en bout.

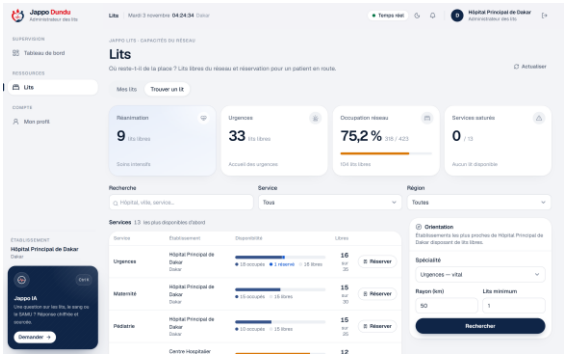


a) Avant l'admission *b) Après l'admission*

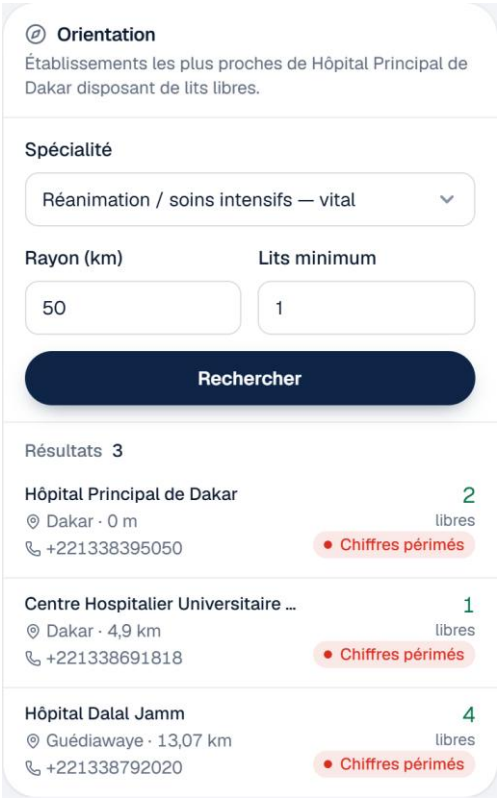
Figure 39 : Admission d'un patient aux urgences

Étape 3 — Trouver un lit dans le réseau (CT10). L'onglet « Trouver un lit » présente les lits libres de tout le réseau et un panneau d'orientation. Pour la réanimation, la recherche dans un rayon de 50 km renvoie les établissements classés par distance, avec leur téléphone et la

fraîcheur des chiffres : un badge « Chiffres périmés » signale qu'un service n'a pas confirmé ses capacités récemment, pour ne pas envoyer un patient vers un lit peut-être déjà occupé.



a) Lits libres du réseau



b) Orientation en réanimation

Figure 40 : Recherche de lits libres dans les établissements proches

1.5 Module Jappo Ambulances

Le module Jappo Ambulances accompagne une urgence de l'appel au SAMU jusqu'à la remise du patient à l'hôpital. Le parcours associe le régulateur (web), l'ambulancier (mobile) et l'hôpital de destination (web).

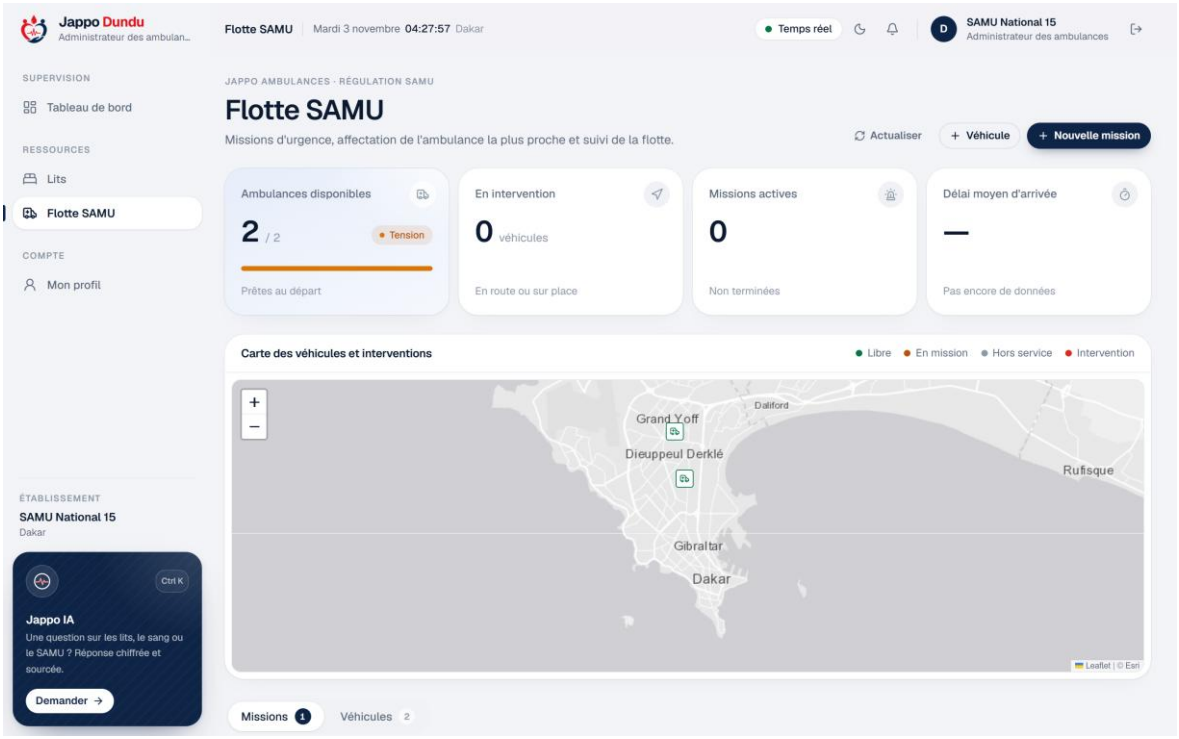


Figure 41 : Flotte SAMU : véhicules et interventions sur la carte

Étape 1 — Créer la mission (CT11). Le régulateur saisit la priorité, la région, le lieu, ses coordonnées, le téléphone de l'appelant et le motif. La destination peut rester « à décider selon les lits libres ». Un clic sur « Déclencher » crée la mission.

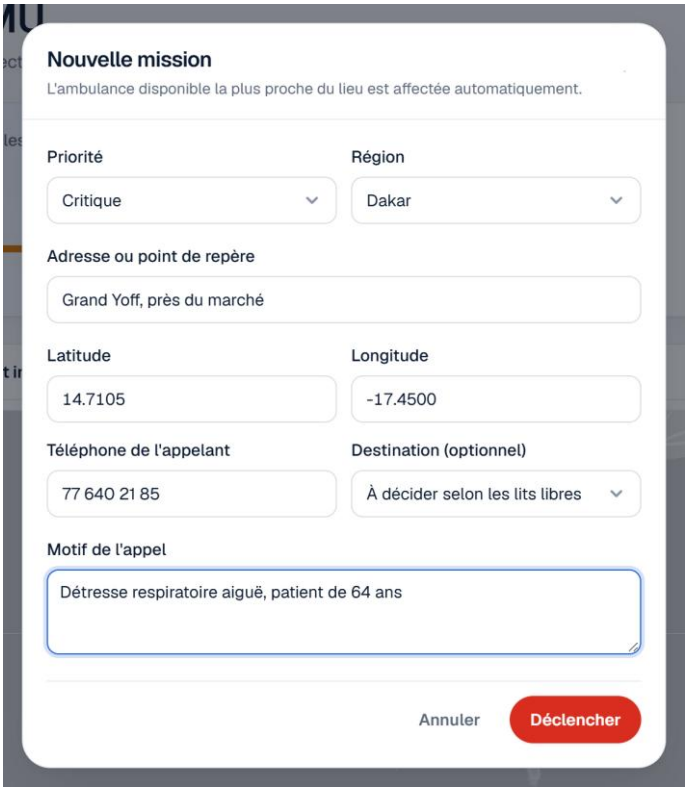
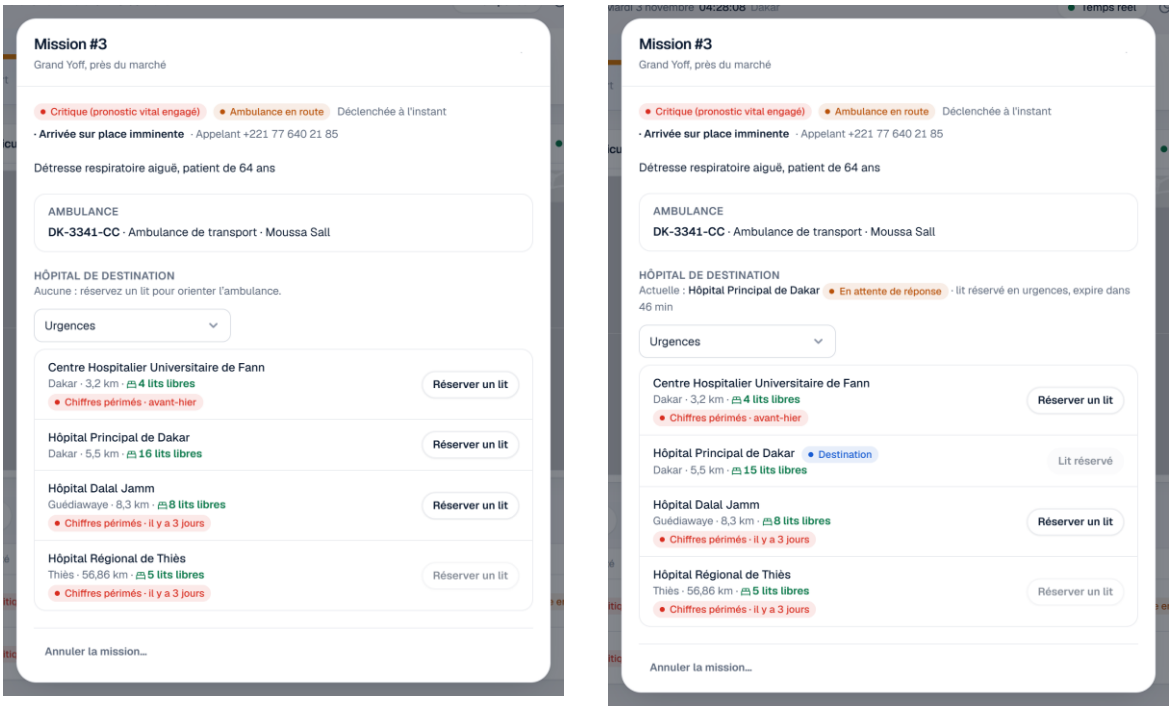


Figure 42 : Création d'une mission à partir d'un appel

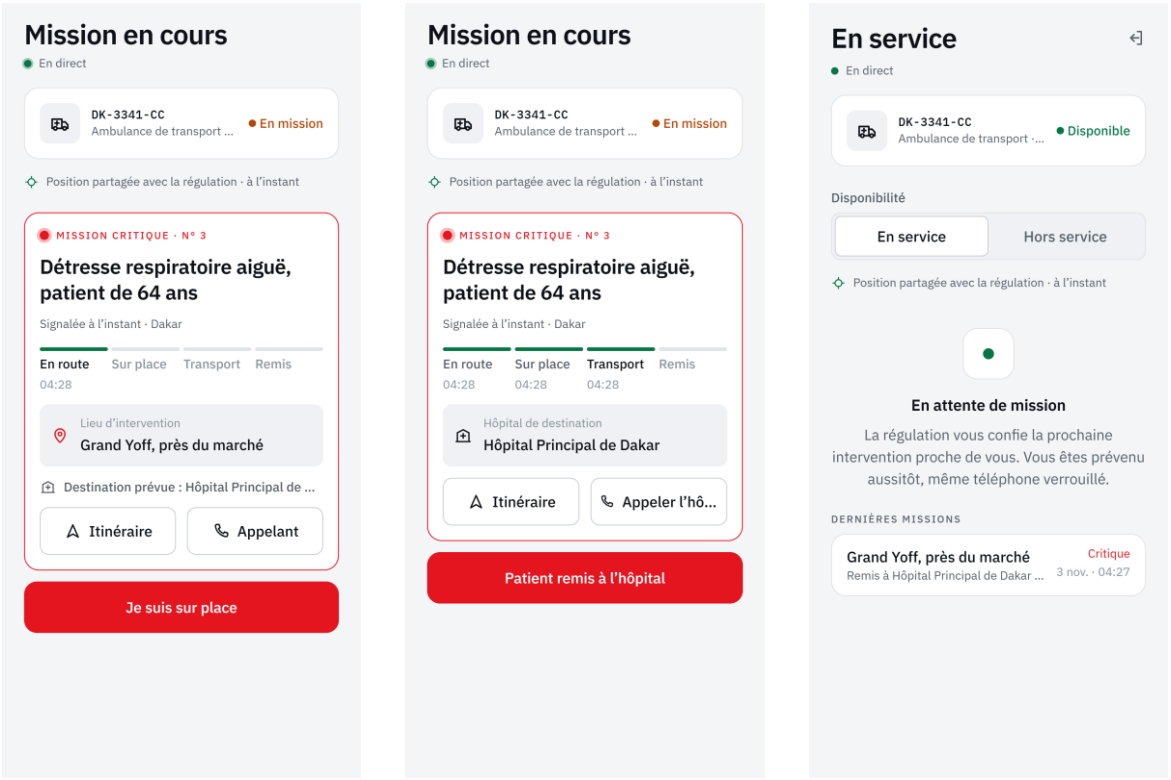
Étape 2 — Affectation automatique (CT12). Le système retient l'ambulance disponible la plus rapide par la route et l'affecte aussitôt : ici DK-3341-CC, conduite par Moussa Sall, avec une arrivée sur place imminente. La fenêtre de régulation propose ensuite les hôpitaux proches ayant des lits libres dans le service recherché ; le régulateur réserve un lit à l'Hôpital Principal de Dakar, qui devient la destination « en attente de réponse ».



a) Ambulance affectée *b) Lit réservé à destination*

Figure 43 : Régulation de la mission : affectation puis choix de l'hôpital

Étape 3 — L'ambulancier suit sa mission (CT13). Sur son téléphone, l'ambulancier voit la mission avec sa priorité, le lieu d'intervention et la destination prévue. Il fait progresser les étapes « En route », « Sur place », « Transport » puis « Remis », chacune horodatée ; sa position GPS est partagée avec la régulation pendant toute la mission.



a) Mission reçue *b) Transport vers l'hôpital* *c) Mission terminée*

Figure 44 : Déroulement de la mission dans l'application de l'ambulancier

Étape 4 — Pré-alerte de l'hôpital. Pendant le transport, l'Hôpital Principal de Dakar voit le patient en approche sur son tableau de bord : motif, priorité, étape de la mission et heure d'arrivée estimée. L'équipe peut accepter ou refuser le patient ; en cas de refus, le régulateur est prévenu et choisit une autre destination.

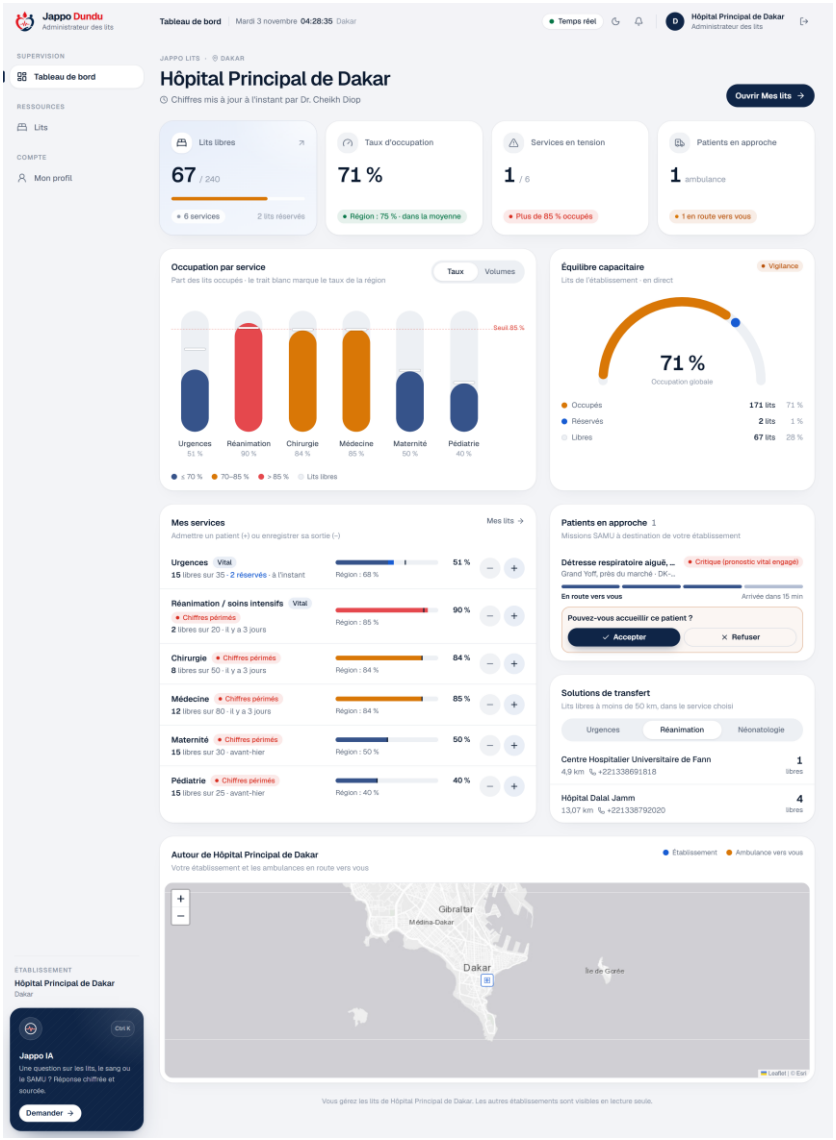


Figure 45 : Pré-alerte de l'hôpital : patient en approche

À la remise du patient, l'ambulance redevient disponible et le lit réservé devient occupé. Une mission peut aussi être annulée par la régulation ; un motif est alors obligatoire, l'ambulancier est prévenu et le lit réservé est libéré.

1.6 Backend et documentation de l'API (Swagger)

L'API REST est documentée par la spécification **OpenAPI 3**, générée par drf-spectacular et présentée dans Swagger UI à l'adresse `/api/docs/`. Elle couvre l'intégralité des points d'accès : **125 chemins et 163 opérations** (77 GET, 72 POST, 10 PATCH, 2 PUT et 2 DELETE), regroupés en treize sections par module. La génération du schéma ne produit aucune erreur ; un test automatisé vérifie à chaque modification qu'il reste valide et qu'il couvre les routes clés.

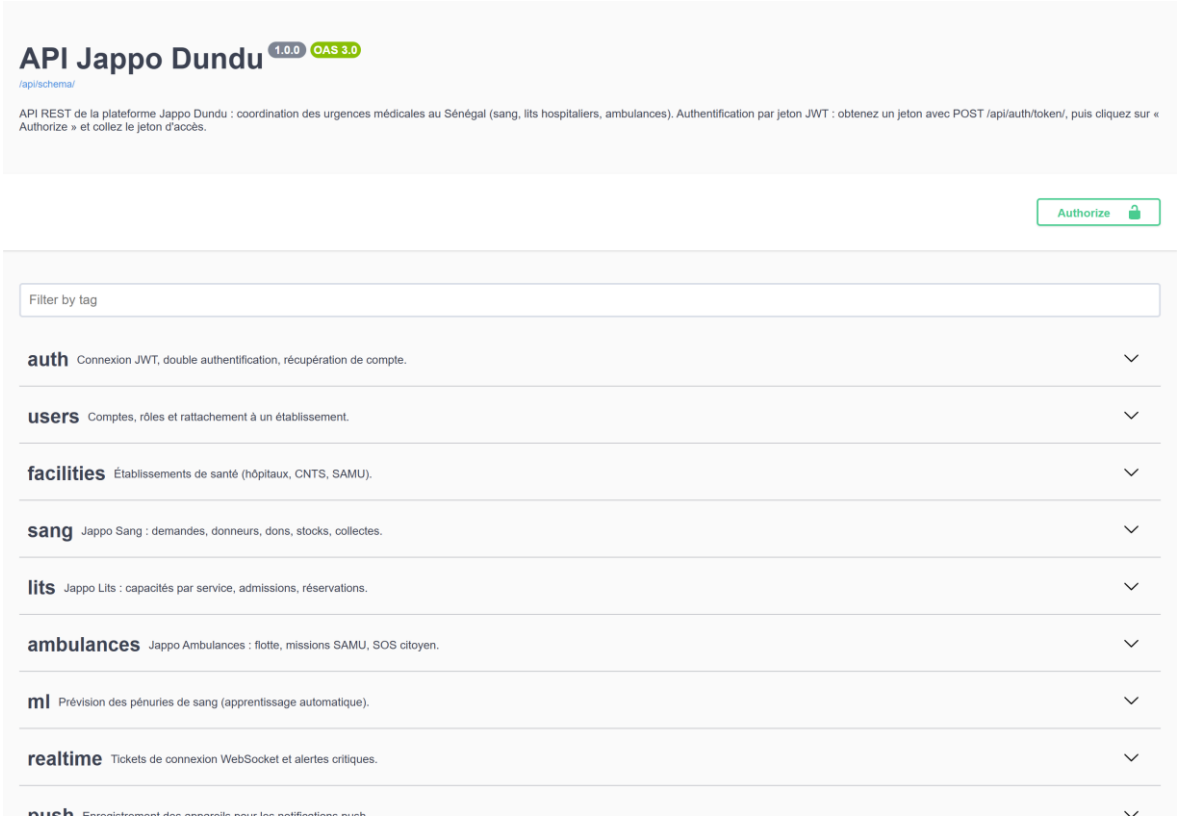
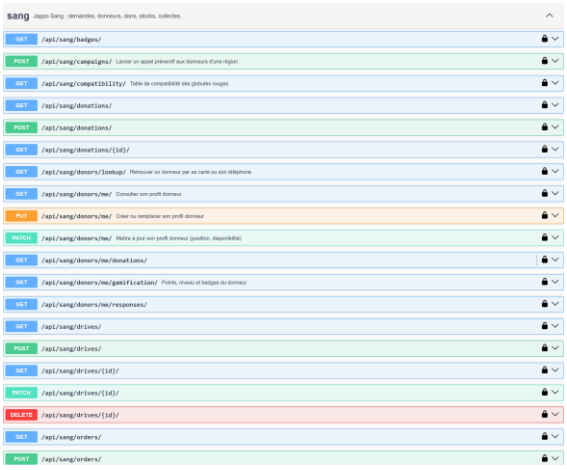
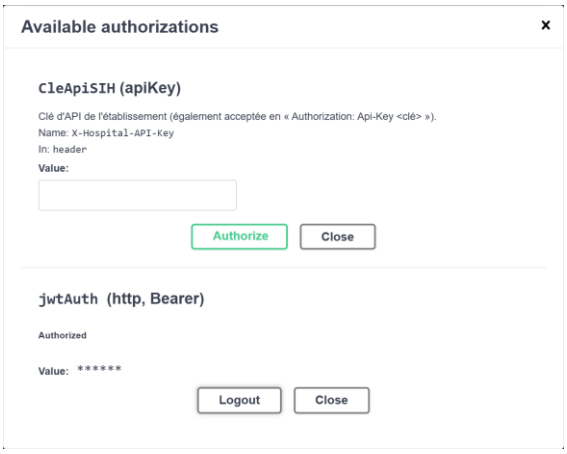


Figure 46 : Swagger UI : documentation de l'API Jappo Dundu

Cette documentation sert de **contrat technique** entre le backend et les applications clientes : pour chaque route, elle donne la méthode HTTP, les paramètres et leur type, le format du corps de requête et de chaque réponse, ainsi que l'authentification requise (cadenas). Deux schémas d'authentification sont décrits : le jeton JWT (`jwtAuth`) pour les utilisateurs et la clé d'API (`CleApiSIH`) pour les logiciels hospitaliers.



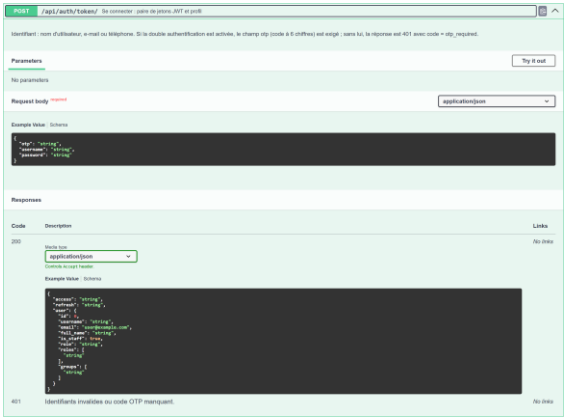
a) Routes du module sang



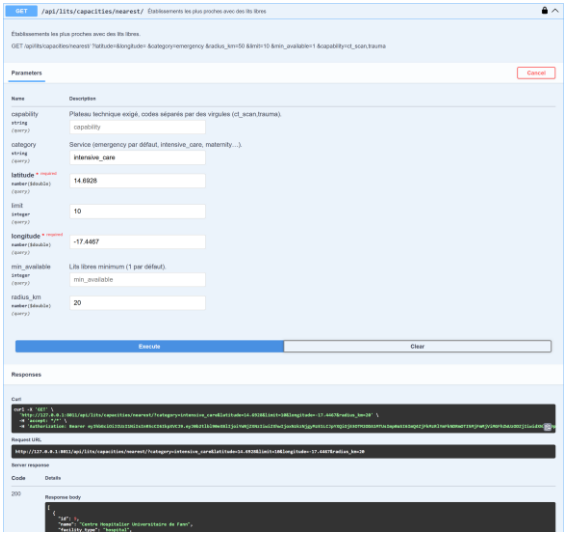
b) Autorisation par jeton JWT

Figure 47 : Liste des points d'accès et schémas d'authentification

Le contrat de la route de connexion précise le corps attendu (identifiant, mot de passe, code OTP facultatif) et la structure de la réponse (jetons et profil de l'utilisateur), ainsi que la réponse 401. Swagger permet enfin d'exécuter un appel réel : la recherche des établissements proches disposant de lits de réanimation renvoie, en 118 ms, la liste triée par distance.



a) Contrat de la connexion



b) Essai de la recherche de lits

Figure 48 : Contrat d'une route et exécution d'un appel depuis Swagger

Extrait de code : recherche des donneurs les plus proches

La recherche géographique est le cœur du système. Sous PostgreSQL, elle s'exprime en une requête PostGIS appuyée sur un index spatial ; la même fonction sert aux donneurs, aux lits et aux ambulances.

Figure 49 : Requête PostGIS de recherche de proximité (geo/postgis.py, simplifiée)

```
SELECT id,
       ST_Distance(geography(ST_MakePoint(longitude, latitude)),
                   geography(ST_MakePoint(%(lon)s, %(lat)s))) AS distance_m
FROM   sang_donor
WHERE  ST_DWithin(geography(ST_MakePoint(longitude, latitude)),
                  geography(ST_MakePoint(%(lon)s, %(lat)s)),
                  %(radius_m)s)                -- filtre par l'index GiST
ORDER  BY geography(ST_MakePoint(longitude, latitude))
          <-> geography(ST_MakePoint(%(lon)s, %(lat)s))   -- tri KNN
LIMIT  %(limit)s;
```

1.7 Module de prévision des pénuries

Pour chaque centre et chaque groupe sanguin, le modèle prévoit le stock des 1 à 30 jours suivants, avec un intervalle P10–P90 et un nombre de jours de stock ; le niveau de risque est **critique** sous 2 jours de stock et en **vigilance** sous 5 jours. Il a été entraîné sur deux ans d'historique simulé (93 568 relevés, 16 centres, 14 régions, 8 groupes), dont les hypothèses sont

documentées : saisons, Ramadan, Tabaski et Grand Magal, campagnes de collecte, péremption des poches.

L'évaluation respecte l'ordre chronologique (entraînement, calibration, puis test sur les 20 % les plus récents, sans chevauchement) et compare le modèle à une référence simple, « le stock ne change pas ». Les résultats ci-dessous ont été mesurés lors de l'entraînement réalisé pour ce rapport.

Tableau 11 : Performances du modèle de prévision (données simulées, 99 770 exemples de test)

Mesure	Modèle	Référence (persistance)
Erreur absolue moyenne (poches)	**2,82**	3,66
… horizon 1 à 3 jours	1,25	1,27
… horizon 15 à 30 jours	**3,44**	4,80
Coefficient de détermination R^2	0,96	—
Couverture de l'intervalle P10–P90 (cible 80 %)	**80,7 %**	—
Pénuries réelles détectées (rappel)	**88,1 %**	85,3 %
Alertes critiques fondées (précision)	97,0 %	—
Niveau de risque exact	64,1 %	65,6 %

Lecture : le modèle réduit l'erreur de 22,8 % et détecte davantage de pénuries ; la calibration porte la couverture de 74,4 % à 80,7 %. À très court terme, la persistance reste légèrement meilleure pour deviner la classe de risque exacte, car un stock varie peu en 1 à 3 jours. Ces chiffres devront être recalculés sur l'historique réel du CNTS.

2. Qualité et tests

2.1 Stratégie de tests

La stratégie suit la **pyramide des tests** : de nombreux tests unitaires et d'intégration rapides, complétés par quelques scénarios de bout en bout sur la pile réelle, puis une recette fonctionnelle manuelle des parcours. Tous les tests automatisés sont rejoués par l'intégration continue à chaque envoi de code.

Tableau 12 : Campagnes de tests automatisés et résultats

Niveau	Outil	Périmètre	Résultat
Backend	Django TestCase, sur PostGIS 16	Règles métier, permissions par rôle et par périmètre, API, temps réel, prévision, schéma OpenAPI	**590 / 590**
Frontend web	Vitest, Testing Library	Composants, pages, règles d'accès et règles métier affichées	**104 / 104**

Niveau	Outil	Périmètre	Résultat
Mobile	Jest (jest-expo)	Alertes, carte donneur, mission, SOS, validation, persistance	**73 / 73**
Bout en bout	Playwright (Edge)	Temps réel par ticket ; affectation SAMU ; admission propagée à un autre poste ; page inconnue	**4 / 4**
Analyse statique	TypeScript, ESLint, `manage.py check`	Typage, qualité du code, cohérence des migrations	Sans erreur

Exécutions du 3 novembre 2026 sur la branche de travail.

Les tests backend couvrent en priorité les risques les plus graves : qu'un utilisateur accède aux données d'un autre établissement, qu'un don soit confirmé pour un donneur inapte, ou que deux admissions simultanées occupent le même dernier lit. Par exemple, le test `test_patch_own_facility_only` vérifie qu'un administrateur des lits modifie les services de son hôpital, mais reçoit une erreur 403 sur ceux d'un autre établissement, tout comme un ambulancier.

2.2 Plan de tests fonctionnels

Le plan suivant a été exécuté sur l'application, dans l'environnement de démonstration ; chaque cas renvoie aux captures de la section 1.

Tableau 13 : Plan de tests fonctionnels et résultats observés

Cas	Action	Résultat attendu et observé	Statut
CT01	Connexion avec des identifiants valides	Accès au tableau de bord du rôle ; menu limité aux modules du rôle	Conforme
CT02	Connexion avec un mot de passe erroné	Accès refusé, message d'erreur	Conforme
CT03	Ouverture d'un module hors du rôle	Écran « Module non accessible » ; API en 403	Conforme
CT04	Création d'une demande de sang critique	Demande ouverte, première vague d'alertes envoyée	Conforme
CT05	Consultation des alertes par la donneuse	Demande compatible affichée avec la distance	Conforme
CT06	Réponse « Je viens donner »	Engagement enregistré, visible en temps réel par le CNTS	Conforme
CT07	Confirmation du don par la carte	Poches collectées 1 / 2, historique et prochaine date mis à jour	Conforme
CT08	Admission d'un patient aux urgences	Lits libres 17 → 16, diffusion temps réel	Conforme
CT09	Réservation d'un lit	Lit bloqué avec référence et échéance	Conforme

Cas	Action	Résultat attendu et observé	Statut
CT10	Recherche de lits de réanimation proches	Établissements triés par distance, fraîcheur signalée	Conforme
CT11	Création d'une mission SAMU	Mission créée et diffusée à la régulation	Conforme
CT12	Affectation de l'ambulance la plus proche	Ambulance affectée, arrivée estimée, lit réservé à destination	Conforme
CT13	Progression de la mission par l'ambulancier	Étapes horodatées, ambulance de nouveau disponible	Conforme
CT14	Création d'un compte sans établissement	Création refusée avec un message explicite	Conforme
CT15	Appel d'une route depuis Swagger	Réponse 200 conforme au schéma documenté	Conforme

La recette a aussi permis de corriger deux défauts : un chevauchement entre le téléphone du donneur et les boutons d'action dans le panneau d'une demande de sang, et des scénarios de bout en bout restés alignés sur d'anciens libellés de l'interface. Les deux corrections ont été vérifiées par une nouvelle exécution.

3. Déploiement

3.1 Gestion des environnements

Trois environnements sont distingués. Toute la configuration passe par des variables d'environnement (fichier `.env`, jamais versionné), ce qui permet d'utiliser le même code partout.

Tableau 14 : Environnements du projet

Environnement	Composition	Usage
Développement	PostgreSQL / PostGIS et Redis dans Docker ; API Django et interface Vite sur le poste	Développement quotidien, rechargement à chaud, comptes de démonstration
Test (CI)	GitHub Actions : service PostGIS 16, Python 3.13, Node 22	Tests, typage, analyse et construction à chaque envoi
Production	Serveur Linux, Docker Compose : base, Redis, API (Daphne), proxy Caddy	HTTPS automatique, seul Caddy est exposé ; démonstration et pilote

3.2 Conteneurisation et architecture de production

En production, quatre conteneurs sont orchestrés par Docker Compose. Le proxy **Caddy** est le seul point d'entrée : il obtient et renouvelle automatiquement le certificat HTTPS (Let's

Encrypt), applique les en-têtes de sécurité, sert l'interface React et relaie `/api`, `/admin` et `/ws` vers l'API. La base de données et Redis restent sur le réseau Docker interne, inaccessibles depuis Internet.

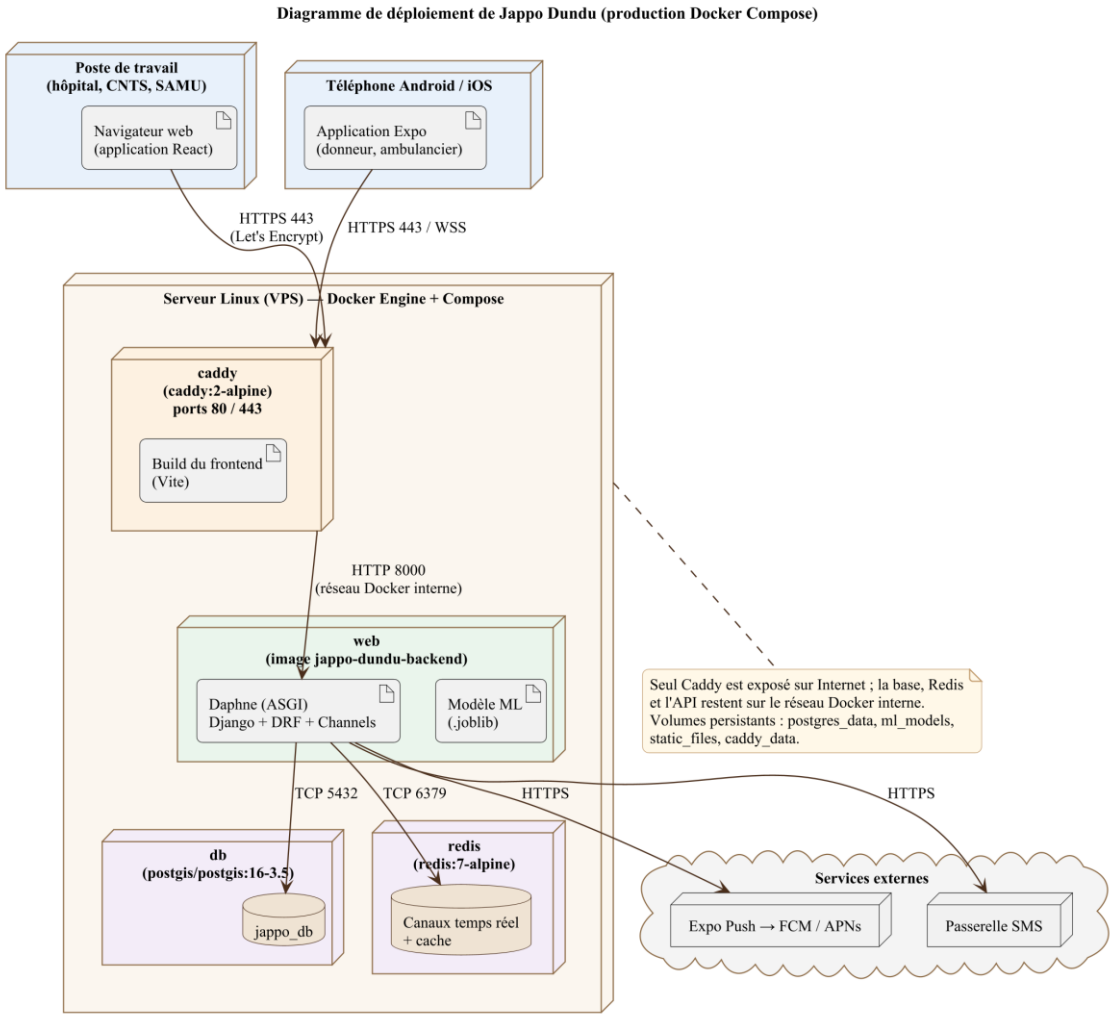


Figure 50 : Diagramme de déploiement en production

3.3 Intégration continue

Le fichier `.github/workflows/ci.yml` définit trois tâches indépendantes, exécutées à chaque envoi et à chaque demande de fusion.

Figure 51 : Extrait du flux d'intégration continue (.github/workflows/ci.yml)

```
jobs:
  backend:
    services:
      db: { image: postgis/postgis:16-3.5-alpine }
    steps:
      - run: pip install -r requirements.txt
      - run: python manage.py check
      - run: python manage.py makemigrations --check --dry-run
      - run: python manage.py test --noinput
  frontend:
    steps:
      - run: npm ci
      - run: npm run typecheck && npm run lint && npm test
      - run: npx vite build
  mobile:
    steps:
      - run: npm ci && npm run typecheck && npx jest --ci && npm run lint
```

La mise en production suit une procédure documentée : copie du fichier de variables, `docker compose -f docker-compose.prod.yml up -d --build`, application des migrations, puis exécution du script de tests de fumée qui rejoue les scénarios essentiels sur l'instance déployée. L'application mobile se construit pour Android et iOS avec les services d'Expo (EAS) ; leur configuration (identifiant de projet, clés Firebase) reste à finaliser avant une diffusion sur les magasins d'applications.

CONCLUSION

Ce projet est parti d'un constat simple : lors d'une urgence, savoir où se trouve la ressource compte autant que la ressource elle-même. Jappo Dundu y répond en réunissant, dans une même plateforme et en temps réel, le sang, les lits et les ambulances, et en mettant en relation les hôpitaux, le CNTS, le SAMU et les citoyens donneurs.

Bilan technique et fonctionnel

Les objectifs fixés en introduction sont atteints dans le périmètre d'un prototype fonctionnel :

Tableau 15 : Bilan des objectifs spécifiques

Objectif	Résultat obtenu	État
Jappo Sang	Demande urgente, alerte par vagues des donneurs compatibles et aptes les plus proches, réponse mobile, confirmation du don par carte QR, stocks	Atteint
Jappo Lits	Capacités par service en temps réel, admissions et sorties atomiques, réservations, orientation vers les lits libres proches	Atteint
Jappo Ambulances	Mission, affectation automatique par temps de trajet, suivi GPS, pré-alerte et réponse de l'hôpital, alerte SOS	Atteint
Pilotage	Tableau de bord par rôle, prévision des pénuries à 7 jours (évaluée sur données simulées), assistant Jappo IA	Atteint (données simulées)
Qualité et sécurité	Contrôle d'accès par rôle et périmètre, API documentée par Swagger, 590 + 104 + 73 tests et 4 scénarios de bout en bout, intégration continue	Atteint

Ces résultats doivent être lus avec honnêteté : ils démontrent le **fonctionnement technique** du système, pas encore son effet sur le terrain. La réduction des délais de prise en charge ne pourra être mesurée qu'au cours d'un pilote, avec des établissements partenaires et des données réelles ; de même, les performances du modèle de prévision devront être recalculées sur l'historique réel du CNTS.

Bilan personnel

Ce projet nous a fait passer de l'apprentissage de technologies isolées à la construction d'un système complet. Nous en retirons des compétences techniques (conception d'une API REST sécurisée, requêtes géographiques, communication temps réel, apprentissage automatique appliqué, développement mobile, tests automatisés, conteneurisation) et des compétences de

conduite de projet : découpage en itérations, répartition des responsabilités, travail sur un dépôt commun et relecture mutuelle.

Nous avons rencontré plusieurs difficultés, chacune résolue par une décision technique documentée :

Tableau 16 : Difficultés rencontrées et solutions apportées

Difficulté	Solution apportée
Absence d'historique réel des stocks de sang pour entraîner le modèle	Générateur de données simulées aux hypothèses documentées ; import CSV prêt pour l'historique du CNTS ; mention explicite « données simulées » dans l'interface
Un calcul de prévision de plusieurs secondes bloquait toute l'API	Exécution en tâche de fond : réponse immédiate 202, fin annoncée en temps réel
Risque d'exposer le jeton de session dans l'adresse du WebSocket	Ticket de connexion à usage unique, échangé contre le jeton par une requête authentifiée
Deux admissions simultanées sur le dernier lit libre	Mise à jour conditionnelle unique en base, sans condition de course
Coordination de quatre membres sur trois applications	Règle « API d'abord », un responsable par domaine, intégration continue sur chaque envoi

Perspectives

- **Pilote de terrain** avec un hôpital, un centre de transfusion et une équipe SAMU, pour mesurer les délais réels avant et après.
- **Historique réel du CNTS** : importer les données, ré-entraîner et réévaluer le modèle de prévision.
- **Connexion aux logiciels hospitaliers** existants grâce à l'API d'interopérabilité (FHIR, DHIS2), pour supprimer la double saisie.
- **Publication de l'application mobile** sur les magasins Android et iOS, et campagne d'inscription de donneurs avec le CNTS.
- **Passage à l'échelle** : file de calcul partagée (Celery ou RQ sur Redis) et plusieurs instances de l'API.
- **Conformité** : validation du traitement des données avec la Commission de Protection des Données Personnelles avant tout déploiement réel.

BIBLIOGRAPHIE ET WEBOGRAPHIE

Ouvrages et articles scientifiques

Beliën, J., & Forcé, H. (2012). Supply chain management of blood products: A literature review. *European Journal of Operational Research*, 217(1), 1-16. https://doi.org/10.1016/j.ejor.2011.05.026

Braa, J., & Sahay, S. (2017). The DHIS2 open source software platform: Evolution over time and space. In *Global health informatics* (pp. 451-462). MIT Press.

Dehnavieh, R., et al. (2019). The District Health Information System (DHIS2): A literature review and meta-synthesis of its strengths and operational challenges based on the experiences of 11 countries. *Health Information Management Journal*, 48(2), 62-75. https://doi.org/10.1177/1833358318777713

Faheem, H., & Dutta, S. (2023). Temie Giwa-Tubosun of LifeBank: Creating the future of health-care supply chain in Africa powered by technology. *The Case for Women*. Emerald Publishing. https://doi.org/10.1108/CFW-07-2022-0017

Haux, R., Winter, A., Ammenwerth, E., & Brigl, B. (2004). *Strategic information management in hospitals*. Springer.

Mamlin, B. W., et al. (2006). Cooking up an open source EMR for developing countries: OpenMRS. *AMIA Annual Symposium Proceedings*, 529-533.

Peleg, K., & Pliskin, J. S. (2004). A geographic information system simulation model of EMS: Reducing ambulance response time. *The American Journal of Emergency Medicine*, 22(3), 164-170. https://doi.org/10.1016/j.ajem.2004.02.003

Romano, Y., Patterson, E., & Candès, E. (2019). Conformalized quantile regression. *Advances in Neural Information Processing Systems*, 32.

Shortliffe, E. H., & Cimino, J. J. (Eds.). (2014). *Biomedical informatics: Computer applications in health care and biomedicine*. Springer.

Trelstad, B., Armerding, P. T., & Lawal, W. (2020). *LifeBank Nigeria*. Harvard Business School Case No. 321-082.

Verma, N., et al. (2021). OpenMRS as a global good: Impact, opportunities, challenges, and lessons learned from fifteen years of implementation. *International Journal of Medical Informatics*, 149, 104405. https://doi.org/10.1016/j.ijmedinf.2021.104405

Rapports et textes officiels

Agence Nationale de la Statistique et de la Démographie (ANSD). (2024). *Les principaux résultats du 5e Recensement général de la population et de l'habitat (RGPH-5)*. https://www.ansd.sn

Autorité de Régulation des Télécommunications et des Postes (ARTP). (2025). *Rapport trimestriel sur le marché des communications électroniques, 4e trimestre 2024*. https://artp.sn

Commission de Protection des Données Personnelles (CDP). (2008). *Loi n° 2008-12 du 25 janvier 2008 portant sur la protection des données à caractère personnel*. https://www.cdp.sn

Ministère de la Santé et de l'Action Sociale (MSAS). (2022). *Rapport de suivi de la carte sanitaire 2022*. https://www.esante.sn

Ministère de la Santé et de l'Action Sociale (MSAS). (2025). *Journée mondiale du donneur de sang*. https://www.sante.gouv.sn

Organisation mondiale de la Santé (OMS). (2020). *Blood safety and availability*. https://www.who.int/news-room/fact-sheets/detail/blood-safety-and-availability

Organisation mondiale de la Santé (OMS). (2021). *Global strategy on digital health 2020-2025*. https://www.who.int/publications/i/item/9789240020924

Documentation technique (consultée en octobre et novembre 2026)

Tableau 17 : Documentation technique consultée

Ressource	Adresse
DHIS2	https://dhis2.org/overview/
OpenMRS	https://openmrs.org/about/
LifeBank	https://lifebank.ng/
Django	https://docs.djangoproject.com/
Django REST Framework	https://www.django-rest-framework.org/
Django Channels	https://channels.readthedocs.io/
drf-spectacular	https://drf-spectacular.readthedocs.io/
Spécification OpenAPI 3.0.3	https://spec.openapis.org/oas/v3.0.3
PostgreSQL	https://www.postgresql.org/docs/
PostGIS	https://postgis.net/documentation/
Redis	https://redis.io/docs/
React	https://react.dev/
Vite	https://vite.dev/
Tailwind CSS	https://tailwindcss.com/docs
Leaflet	https://leafletjs.com/
Expo	https://docs.expo.dev/
React Native	https://reactnative.dev/docs/getting-started
scikit-learn	https://scikit-learn.org/stable/

Ressource	Adresse
Docker	https://docs.docker.com/
Caddy	https://caddyserver.com/docs/
Playwright	https://playwright.dev/
PlantUML	https://plantuml.com/

ANNEXES

Annexe 1 : Rôles et matrice des droits d'accès

Tableau 18 : Matrice simplifiée des droits par rôle

Ressource	Admin	Lits	Sang	SAMU	Ambul.	Donneur
Capacités en lits	L+É	L, É établ.	—	L	L	—
Admissions et sorties	L+É	É établ.	—	—	—	—
Demandes de sang	L+É	—	É établ. / région	—	—	L proches
Confirmation d'un don	É	—	É établ. / région	—	—	—
Stocks et prévisions	L+É	—	L+É	—	—	—
Flotte et missions	L+É	L (vers son établ.)	—	É région	Sa mission	—
Comptes et établissements	L+É	—	—	—	—	—

L = lecture ; É = écriture ; « établ. » = limité à l'établissement de rattachement ; « — » = refusé (403). Source : ROLES_ET_ACCES.md.

Annexe 2 : Extrait de code — contrôle d'accès par rôle

Figure 52 : Définition des rôles et des domaines (backend/security/roles.py, extrait)

```
class Role(StrEnum):
    ADMIN = 'admin'
    BED_MANAGER = 'bed_manager'
    BLOOD_MANAGER = 'blood_manager'
    AMBULANCE_MANAGER = 'ambulance_manager'
    HOSPITAL_STAFF = 'hospital_staff'
    AMBULANCE_DRIVER = 'ambulance_driver'
    DONOR = 'donor'


# Rôles qui ADMINISTRENT chaque domaine (lecture et écriture, dans leur périmètre).
DOMAIN_ROLES = {
    Domain.BEDS: frozenset({Role.ADMIN, Role.HOSPITAL_STAFF, Role.BED_MANAGER}),
    Domain.BLOOD: frozenset({Role.ADMIN, Role.HOSPITAL_STAFF, Role.BLOOD_MANAGER}),
    Domain.AMBULANCES: frozenset({Role.ADMIN, Role.HOSPITAL_STAFF, Role.AMBULANCE_MANAGER}),
}
```

Annexe 3 : Extrait de code — admission atomique d'un patient

Figure 53 : Admission sans condition de course (backend/lits/services.py, extrait)

```
def admit(capacity, *, updated_by):
    """Occupe un lit libre ; 409 si tous les lits sont occupés ou réservés."""
    release_expired(capacity=capacity)
    return _shift_occupancy(
        capacity,
        +1,
        # Condition vérifiée par la base dans la même requête UPDATE :
        {'occupied_beds__lt': F('total_beds') - F('reserved_beds')},
        "Aucun lit libre dans ce service (les lits restants sont occupés ou réservés).",
        updated_by,
    )
```

Annexe 4 : Extrait de configuration — documentation Swagger

Figure 54 : Configuration de drf-spectacular (backend/core/settings.py, extrait)

```
REST_FRAMEWORK = {
    'DEFAULT_SCHEMA_CLASS': 'drf_spectacular.openapi.AutoSchema',
    ...
}
SPECTACULAR_SETTINGS = {
    'TITLE': 'API Jappo Dundu',
    'VERSION': '1.0.0',
    'SERVE_PERMISSIONS': (['rest_framework.permissions.AllowAny'] if DEBUG
                          else ['rest_framework.permissions.IsAdminUser']),
    'SWAGGER_UI_DIST': 'SIDECAR',   # Swagger UI servi par le serveur : CSP stricte
}
# core/urls.py
path('api/schema/', SpectacularAPIView.as_view(), name='schema'),
path('api/docs/', SpectacularSwaggerSplitView.as_view(url_name='schema'), name='swagger-ui'),
```

Annexe 5 : Comptes de démonstration

Tableau 19 : Comptes de démonstration (mot de passe : Password123!)

Rôle	Identifiant	Établissement
Administrateur général	admin	National
Administrateur des lits	dr.diop	Hôpital Principal de Dakar
Administrateur du sang	cnts.dakar	CNTS (Dakar)
Administrateur des ambulances	samu.regulation	SAMU National 15
Ambulancier	samu.driver	SAMU National 15
Donneuse	awa.ndiaye	Citoyenne, groupe O+

Créés par `python seed_demo_data.py` ; le script refuse de s'exécuter en production.